

Gestione dell'Informazione

a.a. 2024/25

Indice

1	Introduzione	3
1.1	Obiettivi del Corso	3
1.2	Cosa sono i Big Data	3
1.3	Diversi Tipi di Informazione	4
2	Information Retrieval	5
2.1	Crescita dei dati non strutturati	5
2.2	Esempi di applicazioni basate sull'IR	5
2.3	Distinzione tra Data (DR) e Information (IR) Retrieval	6
2.4	Modelli di ranking e ricerca approssimativa	6
2.5	Obiettivi primari dell'IR	6
3	Query Languages	7
3.1	Tipologie di Query Languages	7
3.2	Due approcci per la gestione delle query	8
4	Document Preprocessing	9
4.1	Analisi Lessicale (Lexical Analysis)	10
4.2	Eliminazione delle Stop Words	10
4.3	Stemming & Lemmatization	11
4.4	Index Term Selection	12
5	Thesaurus	14
5.1	Thesaurus nel contesto dell'Information Retrieval	14
5.2	Componenti di un Thesaurus	14
5.3	Motivazioni per l'uso di un Thesaurus	15
5.4	Tipologie di Thesauri	15
5.5	Svantaggi e Limiti dei Thesauri	16
6	Similarità	17
6.1	Sinonimia	17
6.2	Similarità delle Parole	17
6.3	Due Classi di Misure di Similarità	17
6.4	Conclusioni	18
7	Word Sense Disambiguation	19
7.1	Due fasi principali del WSD	19
7.2	Metodi principali per la disambiguazione del significato	19
7.3	Disambiguazione dei sostantivi polisemi	19
7.4	Thesaurus e WSD	20
8	Query Expansion (QE)	22
8.1	Problemi principali della QE	22
8.2	Fonti di espansione	22
8.3	Metodi di Espansione	22
9	Indicizzazione - Trie, Inverted Indexes	23
9.1	Introduzione	23
9.2	Composizione	24
9.3	Trie	24
9.4	Inverted Index	26
9.5	Searching	30
9.6	Single-word Retrieval	31
9.7	Phrasal Retrieval	32
9.8	Boosting	32

10 Modeling	33
10.1 IR Models	33
10.2 Similarità	35
10.3 Rappresentazione Logica dei Documenti	36
10.4 Termini di Indicizzazione	37
10.5 Modello Booleano e Query Booleane	37
10.6 Modello Vector Space	39
10.7 Implementazione Modello Space Vector	43
10.8 Modello Probabilistico	43
10.9 Binary Independence Model (BIM)	44
10.10 Introduzione alla Fuzzy Set Theory (Teoria degli Insiemi Fuzzy)	52
10.11 Il Modello BM25 (Okapi BM25)	55
11 Evaluation	57
11.1 User Happiness	57
11.2 Precision e Recall	58
11.3 Livelli Standard di Recall	59
11.4 Precision Interpolata (a Soffitto)	60
11.5 Average Precision (Level) e Confronto tra Sistemi	61
11.6 Single Value Average Precision	62
11.7 Average Precision (Query) e Confronto tra Sistemi	62
11.8 R-Precision (a.k.a. Precision@R)	63
11.9 Problemi di precisione e recall	63
11.10 Harmonic Mean e E-Measure	64
11.11 Discounted Cumulative Gain (DCG)	64
11.12 Normalized Discounted Cumulative Gain (NDCG)	66
11.13 Benchmarks	67
12 Tolerant Retrieval	67
12.1 Wildcard Queries	68
12.1.1 Permutation Index	68
12.1.2 Q-grammi	70
12.2 Spelling Correction	71
12.3 Spelling Correction - Isolated Word Correction	72
12.3.1 Edit distance	73
12.3.2 Q-Gram Overlap	75
12.3.3 Advanced q-gram filtering	77
12.4 Spelling Correction - Context Sensitive Word Correction	78
12.4.1 Hit-Based Correction	78
12.5 Soundex	79
13 Ranking the Web	80
13.1 Link Analysis	80
13.2 PageRank	81
13.3 Hubs and Authorities	83
13.4 Spam	84

1 Introduzione

Il corso di Gestione dell'Informazione rappresenta il naturale proseguimento del corso di Basi di Dati e si focalizza sull'approfondimento delle tecniche avanzate per la gestione e l'elaborazione dell'informazione in contesti applicativi moderni.

Con l'esplosione di piattaforme come il **World Wide Web (WWW)**, i **Social Network**, i **Blog**, e i **Forum**, la capacità di **trattare grandi quantità di dati eterogenei e non strutturati** è diventata una competenza essenziale per chiunque operi nel campo della gestione dei dati e dell'informatica.

1.1 Obiettivi del Corso

Il corso ha come obiettivo principale l'insegnamento delle tecniche avanzate per la **manipolazione di dati testuali**, una delle forme di informazione più diffuse nei domini applicativi moderni.

1. **Modalità di estrazione e di manipolazione dell'informazione:** apprendere come **estrarre**, **processare** e **analizzare** informazioni da dati testuali non strutturati, includendo tecniche di pulizia, trasformazione e normalizzazione dei dati.
2. **Memorizzazione di grandi quantità di informazioni:** conoscere i modelli e le tecnologie per l'archiviazione efficiente di enormi volumi di dati, con un'attenzione particolare alle soluzioni scalabili per la gestione di dati testuali in ambienti distribuiti.
3. **Ricerca efficace basata sul contenuto:** esplorare modelli d'interrogazione che permettono di effettuare ricerche sui dati in modo basato su contenuti testuali, come il **recupero dell'informazione** e il **ranking dei risultati in funzione della rilevanza**.
4. **Ricerca efficiente con algoritmi e strutture dati:** studiare algoritmi e strutture dati avanzate, come **indici invertiti** e **alberi B**, che permettono una ricerca rapida e precisa su grandi quantità di dati testuali.
5. **Programmazione di applicazioni data-centric:** sviluppare competenze di programmazione ottimale per la creazione di applicazioni che accedono e manipolano grandi volumi di dati testuali, garantendo **prestazioni elevate** e **tempi di risposta rapidi**.

1.2 Cosa sono i Big Data

I **Big Data** sono insiemi di dati così vasti, complessi e in continua crescita che le tradizionali tecniche di elaborazione e analisi non sono in grado di gestirli efficacemente.

Questo concetto si riferisce non solo alla **quantità di dati**, ma anche alla loro **varietà** e **velocità con cui vengono generati**.

- **Volume:** la quantità di dati prodotti è enorme, e può provenire da molte fonti diverse.
- **Varietà:** i dati possono essere
 - strutturati (ad esempio, database relazionali)
 - semi-strutturati (HTML, XML, JSON)
 - o non strutturati (testo, immagini, video)

il che richiede strumenti differenti per la loro analisi.

- **Velocità:** i dati vengono generati continuamente e costituiscono un flusso continuo di informazioni che deve essere elaborato in tempo reale o quasi.

L'**80% dei dati digitali esistenti oggi sono dati non strutturati**, ovvero non organizzati in tabelle o schemi predefiniti come nei database relazionali tradizionali.

Questi dati includono testi, immagini, video, audio, documenti, post sui social media, email, e altre fonti di informazioni che non seguono un formato rigido.

L'informazione e il dato

Definizione 1 *L'informazione e il dato*

"L'informazione è associata ai dati, poiché i dati rappresentano valori attribuiti a parametri, e l'informazione è dei dati contestualizzati e con un significato attribuito."

Definizione 2 *L'informazione e la conoscenza*

"L'informazione è anche correlata alla conoscenza, poiché la conoscenza significa comprensione di un concetto astratto o concreto."

1.3 Diversi Tipi di Informazione

I dati possono assumere forme differenti, ognuna con le sue caratteristiche e modalità di utilizzo. Di seguito una descrizione dei principali tipi di informazione:

1. Informazione testuale

- Si riferisce ai dati rappresentati sotto forma di testo.
- Esempi: articoli, documenti, messaggi, post sui social network.
- Tecniche di gestione: ricerca full-text, analisi del contenuto, estrazione di informazioni.

2. Dati multimediali

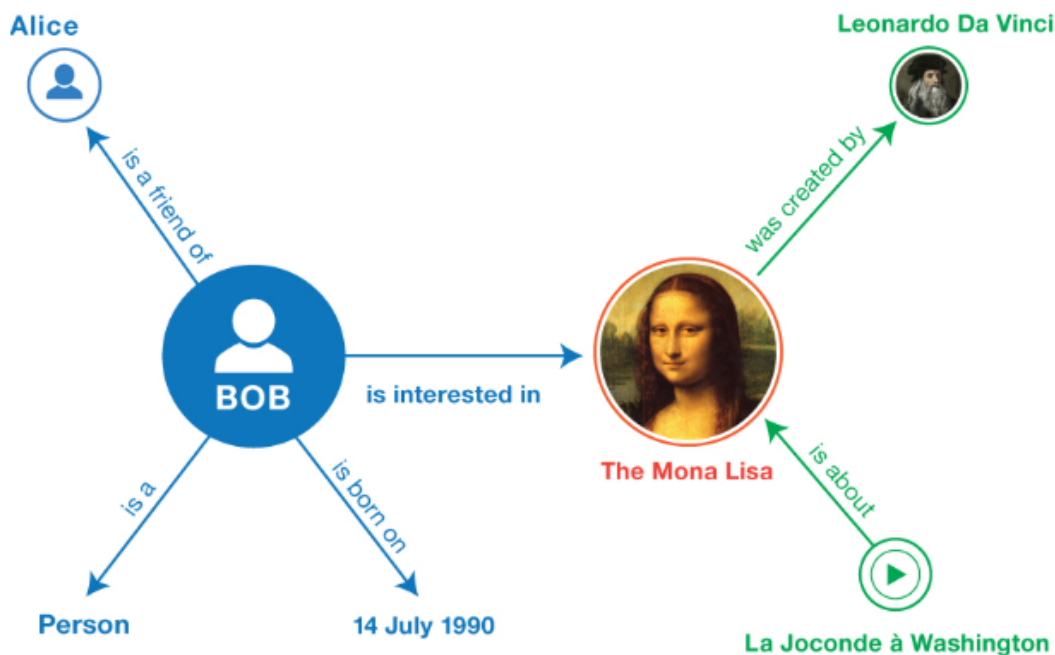
- Comprende formati come immagini, video, audio, e animazioni.
- Esempi: file JPEG, MP4, file audio MP3.
- Tecniche di gestione: elaborazione delle immagini, riconoscimento vocale, compressione video.

3. Informazioni semi-strutturate

- Dati che non seguono uno schema fisso, ma hanno una struttura interna.
- Esempi:
 - **HTML**: utilizzato per la visualizzazione di contenuti su pagine web.
 - **XML**: un formato utilizzato per rappresentare dati gerarchici.
 - **JSON**: un formato di scambio dati leggero, spesso utilizzato per le API e lo scambio di dati tra client e server.
- Tecniche di gestione: parsing, validazione, trasformazione (ad esempio, da XML a JSON), manipolazione tramite linguaggi di query specifici.

4. Informazioni basate su grafi

- Si tratta di informazioni strutturate come nodi e archi, che rappresentano entità e relazioni tra loro.
- Esempi:
 - **RDF (Resource Description Framework)**: un framework per rappresentare informazioni in forma di triplette (soggetto, predicato, oggetto) che descrivono risorse e relazioni tra esse. Usato spesso nel contesto del web semantico.
 - **OWL (Web Ontology Language)**: un linguaggio basato su RDF, usato per definire ontologie, che specificano le relazioni e le gerarchie tra concetti in un determinato dominio.
- Tecniche di gestione: navigazione dei grafi, query tramite SPARQL (linguaggio per interrogare i dati RDF), inferenza semantica.



Il **testo** è ovunque: nelle pagine web, nei documenti, nei giornali, negli articoli, nei blog, nei forum, nei social network, e persino in e-mail, SMS, e descrizioni di prodotti. Anche i dati biologici possono essere rappresentati come sequenze di caratteri testuali. Questa onnipresenza del testo sottolinea l'importanza di strumenti capaci di **gestire**, **organizzare** e **ricercare** informazioni in grandi collezioni di dati testuali.

2 Information Retrieval

Definizione 3 *L'Information Retrieval (IR) è una disciplina che si occupa della **rappresentazione**, **memorizzazione** e **organizzazione** delle informazioni per fornire un **accesso efficiente** all'utente. L'obiettivo primario dell'IR è recuperare elementi di dati che siano rilevanti per una query dell'utente e che aiutino a rispondere alla sua esigenza informativa (UIN - User Information Need).*

2.1 Crescita dei dati non strutturati

I dati **non strutturati**, come il testo in forma libera (documenti, email, post sui social, ecc.), crescono molto più rapidamente dei dati **strutturati** (che seguono schemi predefiniti come tabelle in un database). Mentre i dati strutturati hanno un formato ben definito e una semantica chiara, i dati non strutturati sono complessi da gestire, perché il loro contenuto è ambiguo e più difficile da interpretare.

2.2 Esempi di applicazioni basate sull'IR

Le tecniche di Information Retrieval vengono applicate in vari contesti:

- **Search Engines:** Motori di ricerca come Google sono esempi di sistemi che indicizzano enormi quantità di dati testuali per restituire pagine web rilevanti.
- **Biblioteche digitali:** Raccolte di articoli, libri e risorse accademiche dove gli utenti cercano informazioni specifiche.
- **Form di ricerca nei siti web:** Permettono agli utenti di cercare prodotti, articoli o documenti in base a parole chiave.
- **Banche dati biologiche:** Applicazioni che utilizzano l'IR per recuperare dati genetici o sequenze di DNA.
- **Sistema operativo:** Funzionalità come la ricerca di file utilizzano tecniche IR per trovare documenti basati sui nomi o contenuti testuali.

2.3 Distinzione tra Data (DR) e Information (IR) Retrieval

Esistono differenze chiave tra il Data Retrieval e l'Information Retrieval:

- **Data Retrieval (DR):**

- Si occupa di dati con una struttura ben definita, come i database.
- Si basa su condizioni chiaramente definite (ad esempio, query SQL) per recuperare oggetti che soddisfano tali condizioni.
- La risposta è precisa e non ambigua, restituendo esattamente i dati richiesti.

- **Information Retrieval (IR):**

- Si occupa di testi naturali non strutturati, che spesso hanno significati ambigui.
- Il recupero dell'informazione è **basato sul contenuto**, non solo sulla forma.
- La ricerca è **approssimativa**: non tutte le informazioni recuperate potrebbero essere esattamente pertinenti, ma l'obiettivo è massimizzare la rilevanza dei risultati con tecniche di ranking.

2.4 Modelli di ranking e ricerca approssimativa

Una delle caratteristiche distintive dei moderni sistemi di IR è la capacità di classificare i risultati in base alla loro **rilevanza** per l'utente. I motori di ricerca utilizzano complessi **algoritmi di ranking** per determinare quali documenti sono più pertinenti rispetto alla query dell'utente, anche in presenza di errori o differenze semantiche.

La ricerca testuale nei sistemi IR è spesso **imprecisa**. L'approccio utilizzato è di tipo **approssimativo**, il che significa che vengono tollerate piccole variazioni o errori nella query (es. errori di battitura), e i risultati vengono ordinati in base alla loro rilevanza.

2.5 Obiettivi primari dell'IR

L'obiettivo principale dell'Information Retrieval è recuperare **tutti** gli elementi di dati che sono rilevanti per la query dell'utente, riducendo al minimo il recupero di dati **non rilevanti**. Tuttavia, dato che il testo è spesso ambiguo, questo obiettivo può essere complesso da raggiungere. L'estrazione di **informazioni sintattiche e semantiche** dai dati testuali è cruciale per migliorare la rilevanza dei risultati.

3 Query Languages

Nel contesto della **Gestione dell'Informazione**, uno degli aspetti fondamentali è la capacità di tradurre le esigenze informative dell'utente (**User Information Need, UIN**) in una **query** comprensibile da un sistema di ricerca.

Definizione 4 *Query Languages*

*I **Query Languages** (linguaggi di interrogazione) sono lo strumento attraverso il quale l'utente può esprimere le sue necessità in modo che il sistema possa restituire risultati rilevanti.*

La scelta del query language da utilizzare dipende da diversi fattori, tra cui:

- **Il tipo di utente:** Un utente esperto (come un bibliotecario) avrà familiarità con query complesse, mentre un non esperto potrebbe preferire una ricerca più semplice.
- **Le capacità dell'utente:** La conoscenza del dominio informativo influenza il tipo di query formulata.
- **L'obiettivo della ricerca:** Una ricerca approfondita richiederà un linguaggio più sofisticato, mentre ricerche superficiali potrebbero richiedere query meno complesse.
- **Il tipo di testo da gestire:** Il contenuto della raccolta di informazioni (strutturato o non strutturato) influenza la scelta della query.

3.1 Tipologie di Query Languages

1. Query basate su parole chiave (**Keyword-based querying**)

- **Keyword-based querying** è il tipo di query più elementare, ampiamente utilizzato nei motori di ricerca e nelle applicazioni full-text.
- Le query possono essere composte da una singola parola o da più parole (ad esempio: "information retrieval").
- **Phrase query:** Restituisce documenti contenenti una specifica sequenza di parole nell'ordine esatto. Può permettere variazioni come l'intervento di stop words (es. "the") o l'uso di stemming (es. "enhance the retrieval" può essere trovato con "enhance retrieval").
- **Proximity query:** Consente di specificare una distanza massima tra le parole. Ad esempio, "retrieval" a una distanza di 4 parole da "enhance".

2. Query Booleane

- Le **Boolean queries** consentono di combinare parole chiave o frasi con operatori logici come **AND**, **OR**, e **NOT**.
- Questo tipo di query permette di raffinare i risultati specificando più dettagliatamente quali parole devono o non devono comparire nei documenti restituiti.

3. Pattern Matching

- Il **Pattern Matching** consente di cercare modelli nel testo che seguono certe regole sintattiche. Ad esempio, un **prefisso** può essere usato per trovare parole che iniziano con determinate lettere (es. "comput*" trova "computer" e "computation").
- **Pattern complessi** come le **espressioni regolari** possono essere utilizzati per creare query più avanzate, che permettono di cercare sequenze di caratteri specifiche all'interno di un testo.
- Tipi di pattern:
 - **Prefissi** (es. "comput*")
 - **Suffissi** (es. "*ters" per trovare "computers", "painters")
 - **Sottostringhe** (es. "tal" trova parole come "talk" o "metallic")
 - **Intervalli** (es. "250...500" per trovare numeri o parole tra due valori)

4. Query Strutturali

- Queste query presuppongono che i documenti abbiano una struttura che può essere sfruttata per cercare informazioni specifiche in sezioni definite, come **titoli di capitoli**, campi specifici nei metadati, ecc.
- Ad esempio, è possibile cercare la frase "nuclear fusion" solo all'interno del titolo di un documento.
- Molti database (come quelli di **Oracle** o **PubMed**) utilizzano query strutturali per fornire ricerche più mirate.

5. Query basate su concetti

- Questo approccio utilizza un **vocabolario controllato**, come un'**ontologia** o un **thesaurus**, per assistere l'utente nella formulazione delle query. E' particolarmente utile in domini specifici come la medicina (es. il sistema MESH di **PubMed**).
- Vantaggi:
 - Aiuta l'utente a selezionare termini corretti.
 - Consente di espandere o restringere una ricerca, aiutando a perfezionare la query iniziale.
- L'utente sceglie uno o più **termini predefiniti da cercare**, rendendo il processo di ricerca più preciso.

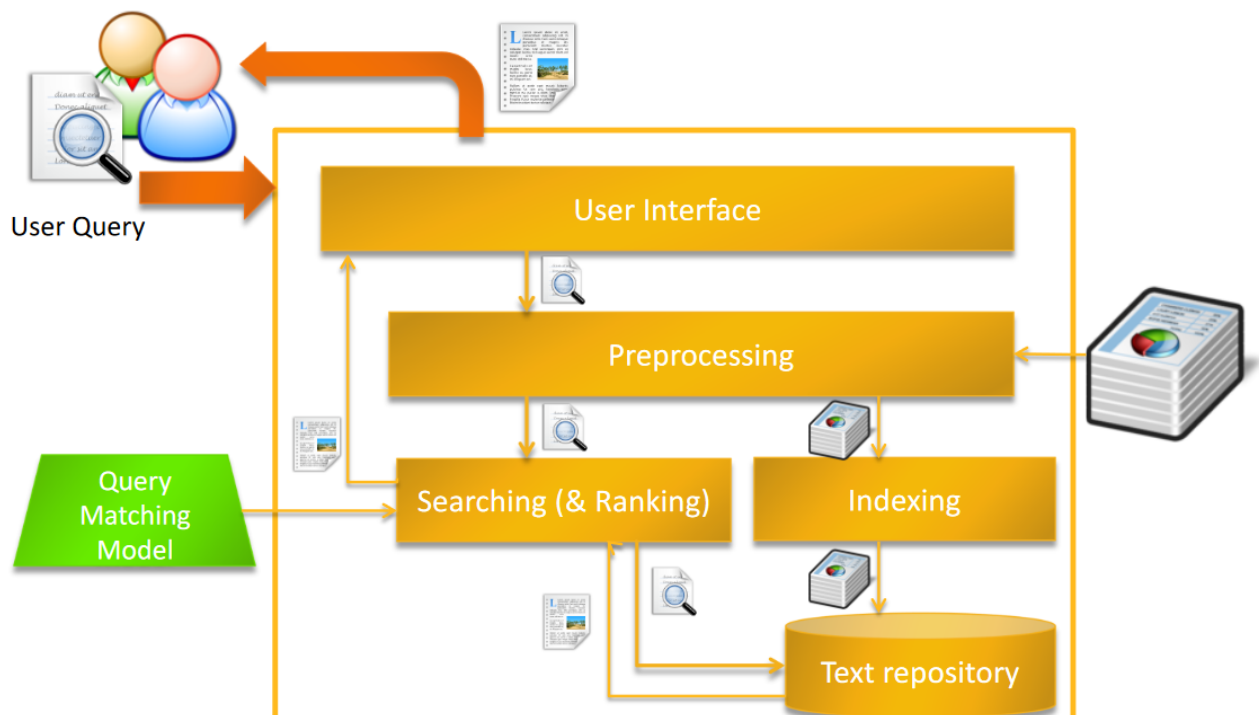
3.2 Due approcci per la gestione delle query

1. Online Query Processing

- Questo approccio viene utilizzato quando il testo da ricercare è **volatile** o **dinamico**, come nel caso dei sistemi di pubblicazione/abbonamento o di flussi di dati in tempo reale.
- Le query vengono elaborate direttamente al momento della richiesta, con un'**elaborazione leggera** e spesso **lato client**.

2. Store First-Query Later

- Questo approccio viene utilizzato quando il testo è di grandi dimensioni. I dati vengono **indicizzati e memorizzati in anticipo**, consentendo una ricerca più rapida e accurata quando l'utente formula una query.



4 Document Preprocessing

Il **Document Preprocessing** è una fase cruciale nell'Information Retrieval e nel Natural Language Processing (NLP), poiché prepara il testo per essere indicizzato e successivamente recuperato in modo efficiente. Lo scopo principale di questa fase è trasformare un documento testuale in un insieme di termini che rappresentano efficacemente il suo contenuto, rendendolo così adatto per un sistema di ricerca. La procedura si articola in vari passaggi, che partono dalla segmentazione del testo in unità significative (token) fino alla selezione di termini rilevanti per l'indicizzazione.

Le principali fasi della **pipeline** del Document Preprocessing:

1. **Lexical Analysis**

Conversione del testo in una sequenza di tokens, ossia gruppi di caratteri con significato collettivo.

2. **Stopword Elimination**

Rimozione delle parole prive di valore informativo per l'indicizzazione (es. articoli, congiunzioni).

3. **Stemming e Lemmatization**

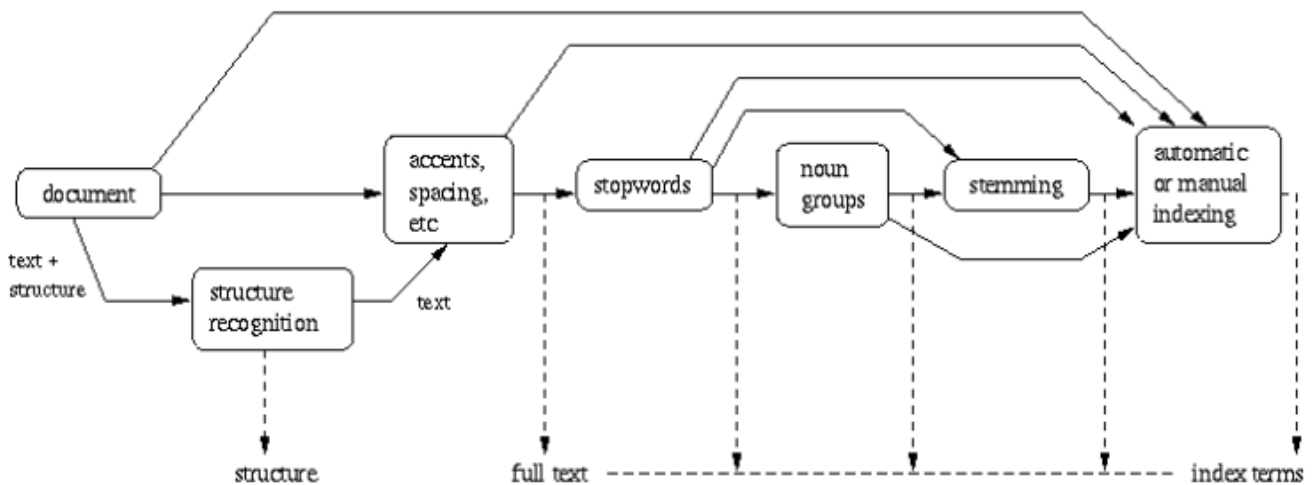
Riduzione delle parole alle loro radici (stemming) o alla forma base (lemmatization).

4. **Index Term Selection (Parser e Tagger)**

Selezione automatica o manuale dei termini rilevanti per l'indicizzazione, tramite analisi delle parti del discorso.

5. **Term Categorization (Espansione dei Termini)**

Costruzione di strutture di categorizzazione dei termini per espandere le query con termini correlati.



4.1 Analisi Lessicale (Lexical Analysis)

Il primo passo del document preprocessing è l'**analisi lessicale**, un processo che trasforma un flusso di caratteri (il testo del documento) in un flusso di token, ossia unità elementari di testo come parole, numeri o simboli con un significato collettivo. Questi token costituiscono i candidati per l'indicizzazione, che saranno usati nelle fasi successive.

- **Tokenizzazione:** si tratta di suddividere il testo in parole o gruppi di caratteri (token). Ad esempio, nella frase "He said that the chairs were enough", i token saranno: He, said, that, the, chairs, were, enough. Tuttavia, ci sono situazioni complesse in cui il riconoscimento dei token richiede decisioni particolari.
 - **Cifre:** Di solito, le cifre non sono buoni termini di indicizzazione a meno che non abbiano un significato specifico (es. B6 come vitamina).
 - **I trattini:** possono essere usati per unire o separare termini. Ad esempio, state-of-the-art può essere tokenizzato come state of the art, mentre MS-DOS non dovrebbe essere separato.
 - **Punteggiatura:** spesso viene eliminata, ma ci sono casi in cui può essere significativa (es. 510B.C diventa 510BC).
 - **Maiuscole/minuscole:** il testo viene spesso convertito tutto in minuscolo per evitare errori dovuti a differenze di capitale (es. Korea University diventa korea university).

Strumenti come i **lexers** (analizzatori lessicali) semplificano questo processo e gestiscono automaticamente questi casi complessi. Un esempio di lexer è **NLTK** (Natural Language Toolkit) in Python, una libreria molto usata per il pre-processing del testo, che offre strumenti per la tokenizzazione.

I principali **metodi di creazione** di un lexer:

- Utilizzo di un Lexical Analyzer Generator (es. **Lex su UNIX**)
- **Scrivere un Lexer manualmente ad hoc**
- **Scrivere un Lexer come Macchina a Stati Finiti (FSM)**

4.2 Eliminazione delle Stop Words

Una volta ottenuti i token, il passo successivo è l'eliminazione delle **stop words**, cioè parole che, pur essendo molto frequenti, non contribuiscono in modo significativo all'interpretazione o alla rilevanza del testo in un sistema di ricerca. Parole come *the, a, it, that* sono esempi di stop words.

- **Vantaggi dell'eliminazione delle stop words:**
 - Riduce la quantità di dati da processare, semplificando le fasi successive.
 - Evita che termini troppo comuni riducano la precisione delle query.

La rimozione delle stop words può avvenire in due momenti:

1. Durante la fase di tokenizzazione, rimuovendo immediatamente le stop words.
2. Dopo la tokenizzazione, filtrando i token attraverso un elenco predefinito di stop words.

In molti casi, vengono usate **strutture dati** come alberi di ricerca o tabelle di hashing per cercare efficientemente le stop words e rimuoverle. Librerie come **NLTK** forniscono liste di stop words pronte all'uso, che possono essere personalizzate in base alle esigenze.

4.3 Stemming & Lemmatization

La terza fase della pipeline del Document Preprocessing è lo **Stemming** e la **Lemmatization**, che si concentra sulla riduzione delle varianti morfologiche delle parole per migliorare la ricerca e l'indicizzazione dei documenti.

Stemming

Lo **stemming** è il processo di riduzione di una parola alle sue radici, rimuovendo i prefissi o i suffissi. Ad esempio, parole come “connected”, “connecting”, “connection” e “connections” possono essere ridotte al loro **stem comune** “connect”. Questo aiuta a raggruppare tutte le varianti morfologiche sotto una radice comune, consentendo un confronto più efficace tra i documenti.

Un **stemmer** è lo strumento utilizzato per eseguire questo processo. Gli stemmer sono spesso utilizzati perché forniscono una forma semplificata delle parole, riducendo la complessità e migliorando l'efficienza. Esistono diversi tipi di stemmer, come:

- **Stemmer di Porter**, che è uno degli algoritmi più noti e usati. Lavora su un insieme di regole per rimuovere le desinenze comuni delle parole. Stemmer di Lancaster, più aggressivo rispetto a Porter, tende a rimuovere maggiori parti della parola e quindi può causare più “overstemming”.
- **Stemmer di Lancaster**, più aggressivo rispetto a Porter, tende a rimuovere maggiori parti della parola e quindi può causare più “overstemming”.

Vantaggi dello Stemming:

- **Riduzione delle varianti**: Le parole che condividono la stessa radice vengono trattate come equivalenti, facilitando la ricerca.
- **Compressione**: Ridurre le parole alla radice permette di risparmiare spazio e rende l'indicizzazione più efficiente.
- **Semplicità**: Gli stemmer sono rapidi ed efficaci, utilizzando regole linguistiche semplici.

Svantaggi:

- **Overstemming**: Quando lo stemmer rimuove troppe parti della parola, si creano radici troppo corte o vaghe. Ad esempio, “university” e “universal” potrebbero essere ridotti erroneamente allo stesso stem.
- **Understemming**: Quando lo stemmer non rimuove abbastanza, lasciando varianti che non dovrebbero essere considerate separate.

Lemmatization

La **lemmatizzazione**, a differenza dello stemming, si concentra sul trovare la **forma base** o **lemma** della parola, come quella che si trova nel dizionario. Un lemmatizer non si limita a tagliare i suffissi, ma utilizza la conoscenza grammaticale della lingua (ad esempio, il tempo verbale o il numero) per restituire la forma corretta. Ad esempio:

- “am”, “are”, “is” → lemma: “be”
- “caring”, “cares”, “cared” → lemma: “care”
- “better” → lemma: “good”

Un **lemmatizer** è più accurato di uno stemmer perché restituisce la forma base reale della parola, ma è più complesso e richiede risorse linguistiche aggiuntive.

Vantaggi della Lemmatization:

- **Precisione:** A differenza dello stemming, che potrebbe produrre radici non esistenti, la lemmatizzazione garantisce la restituzione della forma corretta della parola.
- **Migliore interpretazione semantica:** Permette una comprensione più accurata del contesto e riduce il rischio di ambiguità.

Svantaggi:

- **Maggiore complessità:** La lemmatizzazione richiede più risorse computazionali e conoscenza linguistica.
- **Tempi di esecuzione più lunghi:** Essendo più sofisticata, la lemmatizzazione può rallentare il preprocessing.

Differenze tra Stemming e Lemmatization

- **Stemming** riduce le parole alla radice più semplice possibile, basandosi su regole prefissate.
 - “*connected*”, “*connecting*” → “*connect*”
- **Lemmatization** cerca la forma base corretta nel dizionario, utilizzando informazioni grammaticali.
 - “*connected*” → “*connect*”; “*better*” → “*good*”

Applicazione

Nella pipeline di preprocessing dei documenti, lo stemming è spesso preferito per la sua semplicità ed efficienza, soprattutto nei grandi volumi di dati. Tuttavia, in alcuni contesti come la linguistica computazionale o la ricerca di precisione, si potrebbe optare per la lemmatizzazione.

La scelta tra stemming e lemmatizzazione dipende dal tipo di applicazione: lo stemming è più rapido e leggero, mentre la lemmatizzazione offre maggiore precisione ma a un costo computazionale più elevato.

4.4 Index Term Selection

La quarta fase della pipeline del Document Preprocessing riguarda la selezione dei **index terms** (termini di indicizzazione). Questa fase è fondamentale perché non tutte le parole presenti in un documento hanno lo stesso **peso semantico**. Alcune parole sono più significative per rappresentare il contenuto e devono essere selezionate come index terms, mentre altre, come le stopwords, sono meno utili.

Selezione Manuale

Nella selezione manuale, specialisti (ad esempio, bibliotecari o esperti di informazione) decidono quali termini sono più significativi per rappresentare il documento. Gli specialisti vengono supportati da strumenti avanzati, che forniscono suggerimenti sui termini rilevanti. Un esempio è il lavoro di aziende come ExpertAI, che utilizzano modelli linguistici di grandi dimensioni (Large Language Models) per assistere nella selezione manuale.

Selezione Automatica

La selezione automatica è il metodo più comune nei sistemi di Information Retrieval su larga scala, dove è impossibile fare un’analisi manuale di ogni documento. Il problema principale nella selezione automatica dei termini è identificare correttamente i nomi, che solitamente portano più valore semantico rispetto ad altre parti del discorso. Qui entrano in gioco due strumenti chiave:

1. **Parser:** Un parser è un analizzatore che assegna una struttura sintattica a una frase.

- Il parser analizza la **struttura sintattica** di un testo.
- Scompone una frase in **componenti grammaticali** (soggetto, verbo, complemento, ecc.).
- Può costruire un **albero sintattico** per rappresentare la **gerarchia tra le parole**.

I parser si basano su grammatiche predefinite. Tuttavia, uno dei problemi principali nell'uso di un parser è la complessità del linguaggio naturale, che rende difficile ottenere una precisione superiore al 90%.

Esempio:

- **Frase:** *"Il gatto mangia il pesce."*
- Il parser può riconoscere che *"Il gatto"* è il soggetto, *"mangia"* è il verbo e *"il pesce"* è l'oggetto.

2. **Tagger:** A differenza dei parser, i tagger svolgono un'operazione più semplice.

- Il tagger assegna **etichette grammaticali** (Part-of-Speech, POS) a ogni parola.
- Non analizza la struttura completa della frase, ma **classifica le parole in categorie** (sostantivo, verbo, aggettivo, ecc.).

Il loro compito è identificare il ruolo (**Part-of-Speech o POS**) di ciascuna parola in una frase. I tagger moderni hanno una precisione più elevata rispetto ai parser, arrivando a oltre il 95%.

Esempio:

- Nella **frase** *"Il gatto mangia il pesce"*:
- *"Il"* → Det (determinante)
- *"gatto"* → Noun (sostantivo)
- *"mangia"* → Verb (verbo)
- *"il"* → Det (determinante)
- *"pesce"* → Noun (sostantivo)

Parser vs. Tagger

Mentre i parser sono più completi, fornendo una rappresentazione dettagliata della sintassi di una frase, i tagger sono più leggeri e focalizzati solo sulla classificazione delle singole parole in base alla loro funzione grammaticale.

Nei sistemi di Information Retrieval, il tagger viene spesso preferito per la sua maggiore efficienza e semplicità.

Limiti e Applicazioni

I parser e i tagger non sono perfetti e possono commettere errori, soprattutto con ambiguità sintattiche o termini specialistici. Tuttavia, questi strumenti sono essenziali per automatizzare l'estrazione dei termini significativi in documenti di grandi dimensioni, come articoli scientifici, pagine web o documenti legali.

5 Thesaurus

5.1 Thesaurus nel contesto dell'Information Retrieval

Il concetto di **thesaurus** è fondamentale nell'ambito del recupero dell'informazione (Information Retrieval). *"Si tratta di un vocabolario controllato utilizzato per l'indicizzazione e la ricerca di documenti. Questo vocabolario non solo fornisce un elenco di termini, ma stabilisce anche le relazioni tra i concetti rappresentati da quei termini, facilitando così l'organizzazione semantica delle informazioni"*.

Indicizzazione e Linguaggio di Indicizzazione

Un thesaurus si basa su un **linguaggio di indicizzazione**, cioè un insieme controllato di termini selezionati dal linguaggio naturale, che viene utilizzato per rappresentare in modo sintetico i soggetti di un documento. I termini di questo linguaggio sono scelti per garantire una rappresentazione precisa e uniforme dei contenuti, riducendo ambiguità e migliorando l'efficacia della ricerca.

5.2 Componenti di un Thesaurus

Un thesaurus è composto da diversi elementi che ne determinano la struttura e l'utilità:

1. Thesaurus Index Term

Il **termine di indicizzazione del thesaurus** è un concetto che rappresenta l'unità semantica di base. Questo può essere una parola singola, una frase o un gruppo di parole, e serve per rappresentare un concetto specifico all'interno di un dominio.

- **Esempi di termini di indicizzazione:** *Building, Teaching, Ballistic Missiles, Body Temperature.*

Questi termini sono selezionati per la loro capacità di riassumere concetti complessi, ma talvolta è necessario specificare ulteriormente il loro significato per **evitare ambiguità**. Ad esempio, la parola *Seal* può avere significati diversi, come "*foca*" o "*sigillo*". In questi casi, è utile aggiungere una definizione o un contesto che chiarisca il significato.

2. Relazioni tra i termini del Thesaurus

Un aspetto chiave di un thesaurus è che non solo definisce i termini, ma stabilisce anche le **relazioni tra di essi**. Queste relazioni possono essere *semantiche*, *gerarchiche* o *funzionali* e aiutano a comprendere come i concetti siano collegati:

- **Sinonimi (SYN)** (*Relazione semantica*): Termini con significati simili o intercambiabili. Ad esempio, *fotocamera* e *macchina fotografica* possono essere considerati sinonimi in certi contesti.
- **Broader Term (BT)** (*Relazione gerarchica*): Un termine più ampio che rappresenta un concetto di livello superiore. Ad esempio, *fotocamera* è un termine più ampio rispetto a *fotocamera digitale*, poiché include tutte le fotocamere, non solo quelle digitali.
- **Narrower Term (NT)** (*Relazione gerarchica*): Un termine più specifico che rappresenta una sotto-categoria del termine principale. Ad esempio, *fotocamera digitale* è un termine più specifico di *fotocamera*.
- **Related Term (RT)** (*Relazione funzionale*): Termini che hanno una relazione logica o funzionale, ma che non rientrano nelle categorie di "più ampio" o "più specifico". Ad esempio, *fotocamera* può essere collegata a *obiettivo* come termini correlati, poiché l'obiettivo è una parte essenziale della fotocamera.

Queste relazioni permettono di costruire una rete concettuale, dove i termini sono collegati in modo logico e semantico. Questo approccio aiuta a migliorare sia l'indicizzazione dei documenti sia il recupero dell'informazione durante le ricerche.

5.3 Motivazioni per l'uso di un Thesaurus

L'uso di un thesaurus nel contesto dell'Information Retrieval offre diversi vantaggi, sia per l'indicizzazione che per il processo di ricerca:

1. Indicizzazione

Un thesaurus fornisce un vocabolario controllato che facilita l'indicizzazione dei documenti. Questo approccio permette di:

- (a) **Normalizzare i concetti di indicizzazione:** Utilizzando termini standard per rappresentare i concetti, si evita la confusione derivante dall'uso di sinonimi o termini ambigui. Questo rende l'indicizzazione più coerente.
- (b) **Ridurre il rumore:** Il rumore si riferisce a risultati di ricerca irrilevanti. Grazie all'uso di un thesaurus, i documenti possono essere indicizzati con termini chiari e specifici, riducendo il numero di risultati non pertinenti.
- (c) **Identificare i termini di indicizzazione con una chiara semantica:** I thesauri aiutano a scegliere i termini che meglio rappresentano il contenuto di un documento, migliorando la precisione del processo di ricerca.

2. Ricerca

Un thesaurus è uno strumento utile anche per l'utente che effettua una ricerca. Permette di:

- (a) **Aiutare l'utente nella formulazione delle query:** Spesso, un utente può non conoscere esattamente i termini più appropriati per la sua ricerca. Il thesaurus può suggerire termini alternativi, sinonimi, o termini più specifici (*narrower*) o più ampi (*broadener*) per affinare la query e ottenere risultati migliori.
- (b) **Classificare le gerarchie:** Un thesaurus fornisce strutture gerarchiche che permettono all'utente di ampliare o restringere la portata della sua ricerca. Ad esempio, un utente può iniziare cercando *malattie cardiache* e poi ampliare la ricerca a *malattie cardiovascolari*.
- (c) **Particolarmente utile per domini specifici:** In settori come la medicina o l'ingegneria, l'uso di un thesaurus è essenziale per garantire precisione terminologica. Ad esempio, il thesaurus *MESH (Medical Subject Headings)* è ampiamente utilizzato nella medicina per indicizzare articoli in modo accurato. Per i domini generali, l'utilità del thesaurus può essere meno evidente, poiché i termini possono essere meno tecnici o specializzati.

5.4 Tipologie di Thesauri

Esistono diversi approcci alla costruzione di un thesaurus:

- **Costruzione Manuale:** Esperti del dominio costruiscono thesauri specifici per un particolare settore. Questo approccio garantisce l'accuratezza e la precisione dei termini, ma è molto dispendioso in termini di **tempo e risorse**.
- **Costruzione Automatica:** In alcuni casi, è possibile creare un thesaurus automaticamente, analizzando una collezione di documenti o integrando thesauri già esistenti. Tuttavia, la costruzione automatica può essere complessa, poiché richiede tecniche avanzate come la *disambiguazione del significato delle parole (Word Sense Disambiguation)*.
- **Costruzione Semi-Automatica:** Un approccio ibrido, dove gli esperti del dominio collaborano con sistemi automatici per costruire un thesaurus. Questo metodo combina l'efficienza dell'automazione con la precisione del controllo umano.

5.5 Svantaggi e Limiti dei Thesauri

Nonostante i vantaggi, l'uso di un thesaurus può presentare alcune difficoltà:

- **Selezione Manuale:** La costruzione manuale di un thesaurus è un processo lungo e complesso, che richiede una grande competenza da parte degli esperti del dominio. Gli utenti che usano un thesaurus devono anche essere sufficientemente formati per utilizzarlo correttamente.
- **Problemi di Coerenza:** Non tutti i thesauri sono coerenti o aggiornati, e ciò può portare a difficoltà nella selezione dei termini. In alcuni casi, l'uso di termini diversi per concetti simili può causare confusione.
- **Selezione Automatica:** La creazione automatica di un thesaurus è computazionalmente costosa e può richiedere sofisticate tecniche di disambiguazione per evitare errori semantici.
- **Copertura:** Alcuni thesauri automatici possono essere incompleti o inadeguati per coprire tutti i concetti di un particolare dominio o lingua, limitando così la loro utilità.

6 Similarità

La **similarità** delle parole è un concetto centrale sia nella gestione delle informazioni che nel processamento del linguaggio naturale (NLP). Per comprendere meglio questo concetto, è importante distinguere la **similarità** dalla **sinonimia**:

6.1 Sinonimia

Nel linguaggio comune, si considerano sinonimi due termini che hanno lo stesso significato o un significato molto simile. Questo significa che possono essere usati alternativamente senza alterare il senso di una frase. Ad esempio, "felice" e "contento" sono sinonimi perché possono essere scambiati in molte frasi senza alterarne il significato. La sinonimia è una **relazione binaria**: due parole sono sinonime o non lo sono. Tuttavia, nella pratica del Natural Language Processing (NLP), è importante tenere presente che la sinonimia perfetta è rara, e molte parole considerate "sinonime" possono avere sfumature leggermente diverse in contesti differenti.

6.2 Similarità delle Parole

A differenza della sinonimia, la **similarità** delle parole è una metrica più flessibile che misura quanto due parole abbiano in comune a livello di significato. La similarità non è binaria, ma è **continua**: due parole possono essere "più" o "meno" simili a seconda delle caratteristiche semantiche che condividono. Per esempio, "macchina" e "motorino" sono simili perché entrambi sono mezzi di trasporto, ma "macchina" e "benzina", pur essendo collegati, non sono semanticamente simili.

Esempi:

- *Macchina, motorino*: simili, entrambi mezzi di trasporto.
- *Macchina, benzina*: relazionati, ma non simili. La benzina è collegata alla macchina ma non condivide lo stesso significato.

In NLP, la similarità delle parole è utile in vari ambiti, tra cui:

- **Information Retrieval (IR)**: recupero dell'informazione rilevante basato su termini correlati.
- **Word sense disambiguation (WSD)**: E' il processo di determinare il significato corretto di una parola in base al contesto in cui viene utilizzata.

6.3 Due Classi di Misure di Similarità

1. **Similarità Basata sui Percorsi (Path-based Similarity)** In questa categoria, due concetti sono considerati simili se sono vicini tra loro nella gerarchia di un thesaurus, come nel caso di WordNet, che utilizza relazioni iperonimiche (is-a).
 - **Distanza del Percorso (Path Distance Similarity)**: misura la similarità basata sulla lunghezza del percorso più breve tra due concetti in una gerarchia tassonomica (ad esempio, una gerarchia iperonimo/iponimo). Più il percorso è breve, più i concetti sono simili.

$$\text{sim}_{\text{path-distance}}(c_1, c_2) = \frac{1}{\text{shortest-path}(c_1, c_2) + 1}$$

- **Similarità di Wu-Palmer**: si basa sulla profondità dei due concetti nella tassonomia e sul loro "Least Common Subsumer" (LCS), ovvero l'antenato comune più specifico.

$$\text{sim}_{\text{Wu-Palmer}}(c_1, c_2) = \frac{2 * \text{depth}(\text{LCS}(c_1, c_2))}{\text{depth}(c_1) + \text{depth}(c_2)}$$

Più il LCS è profondo, più i concetti sono simili.

2. **Similarità Basata sul Contenuto Informativo (Information-Content Similarity)** Questa misura di similarità si basa sulla quantità di informazioni condivise tra due concetti. L'idea è che due concetti sono più simili se condividono più informazioni comuni. Il **contenuto informativo** di un concetto può essere misurato come:

$$\mathbf{IC}(c) = -\log P(c)$$

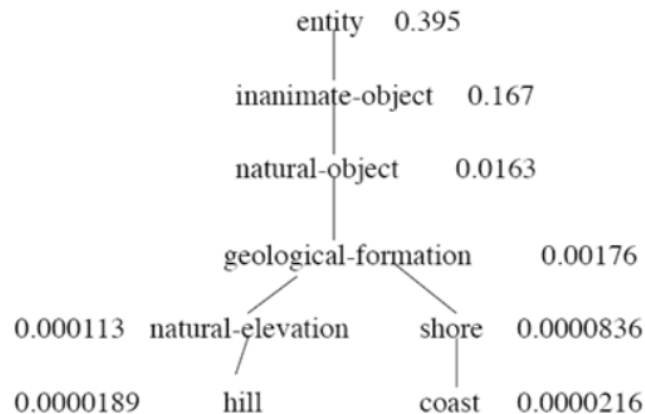
Dove $P(c)$ è la probabilità che un termine nel corpus rappresenti il concetto c . Concetti rari hanno un IC elevato, mentre concetti comuni hanno un IC basso.

- **Similarità di Resnik:** si basa sul contenuto informativo del "Lowest Common Subsumer" (LCS), il nodo comune più specifico nella tassonomia. La formula è:

$$\mathbf{sim}_{\mathbf{Resnik}}(c_1, c_2) = \mathbf{IC}(\mathbf{LCS}(c_1, c_2))$$

Questo approccio misura la quantità di informazioni che due concetti condividono.

Nella **gerarchia dei significati**, i concetti più specifici (vicini alle foglie dell'albero) sono più rari e hanno un Information Content (IC) più alto. Al contrario, i concetti più generali (vicini alla radice dell'albero) sono più comuni e hanno un IC più basso.



Altri metodi basati sul contenuto informativo includono:

- **Similarità di Jiang-Conrath**
- **Similarità di Lin**

Altri ambiti in NLP nei quali similarità delle parole è utile:

- *Machine Translation*: traduzione automatica, dove i concetti simili possono essere tradotti con termini adatti.
- *Question Answering*: risposte automatiche a domande.
- *Language Modeling*: modellazione del linguaggio per la predizione del testo.
- *Plagiarism Detection*: rilevamento di plaghi attraverso somiglianze semantiche tra testi.
- *Document Clustering*: raggruppamento di documenti simili.

6.4 Conclusioni

La similarità delle parole è un concetto fondamentale nel campo della gestione delle informazioni e del processamento del linguaggio naturale (NLP). Mentre la sinonimia è un concetto binario, la similarità è una metrica continua che permette di misurare il grado di somiglianza tra due termini. Le due classi principali di metodi, basate sui percorsi e sul contenuto informativo, offrono modi differenti di calcolare la similarità semantica tra concetti.

7 Word Sense Disambiguation

Il **Word sense disambiguation (WSD)** è il *"processo di determinare il significato corretto di una parola in base al contesto in cui viene utilizzata"*. Molte parole hanno **più significati** (polisemia), e la WSD è cruciale per **evitare ambiguità** e migliorare l'accuratezza in attività come traduzione automatica, analisi semantica e recupero delle informazioni.

La similarità delle parole viene utilizzata per scegliere quale tra i diversi significati di una parola sia il più vicino alle altre parole nel contesto. Ad esempio, per la parola *"banca"*, in un contesto finanziario verrà identificata come *"istituto di credito"*, mentre in un contesto naturale potrebbe essere *"sponda di un fiume"*.

7.1 Due fasi principali del WSD

1. **Determinazione di tutti i sensi di una parola:** Il primo passo nel WSD consiste nell'identificare tutti i possibili significati di una parola. Questo processo si basa solitamente sull'adozione di una risorsa lessicale come un dizionario o un thesaurus (es. WordNet), che offre una lista dei significati associati a ciascuna parola.
 - *WordNet*: Spesso utilizzato come riferimento, è un vasto database lessicale che organizza i significati delle parole in base a una gerarchia di iperonimi, meronimi e relazioni semantiche, facilitando la determinazione dei sensi.
2. **Assegnazione del senso appropriato a ogni occorrenza della parola:** Una volta determinati i significati, il passo successivo è assegnare ad ogni occorrenza della parola il senso più appropriato, basandosi sull'analisi del contesto in cui appare.

7.2 Metodi principali per la disambiguazione del significato

1. Approccio Bag of Words:

Il contesto della parola viene definito come una finestra di parole vicine al termine da disambiguare. L'idea è che le parole vicine possano fornire indizi utili per scegliere il significato corretto.

2. Approccio basato sulle relazioni:

Questo metodo non si limita solo a considerare le parole circostanti, ma utilizza anche altre informazioni contestuali, come la distanza semantica tra le parole e le relazioni sintattiche tra di esse. Questo può includere relazioni di iperonimia o similarità semantica.

7.3 Disambiguazione dei sostantivi polisemi

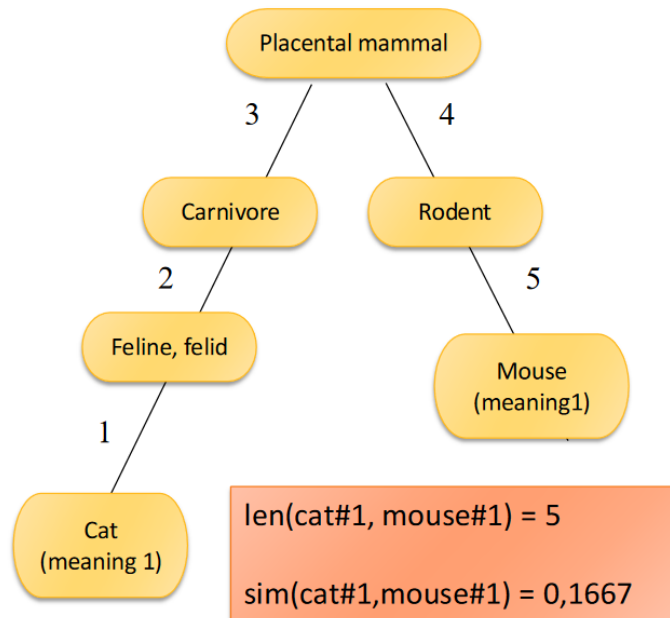
Per *disambiguare un sostantivo polisemo*, si utilizza il contesto rappresentato dagli altri sostantivi presenti nella frase. L'idea chiave è che, se due parole poliseme nel contesto sono simili, i loro concetti correlati possono fornire informazioni utili per scegliere i significati corretti.

- **Confidence C_{s_N} :** Il grado di fiducia nella scelta del senso s_N come significato corretto per N dipende dalla similarità tra s_N e i sensi degli altri sostantivi presenti nel contesto.

Esempio

Consideriamo la frase: *"The cat is hunting the mouse"*.

- Il contesto della parola *"cat"* è influenzato dalla presenza di *"mouse"* nella frase.
- Dopo aver analizzato la distanza gerarchica tra i significati di *"cat"* e *"mouse"*, il sistema di disambiguazione assegnerà il significato più adatto a entrambi, considerando la similarità tra i loro sensi.

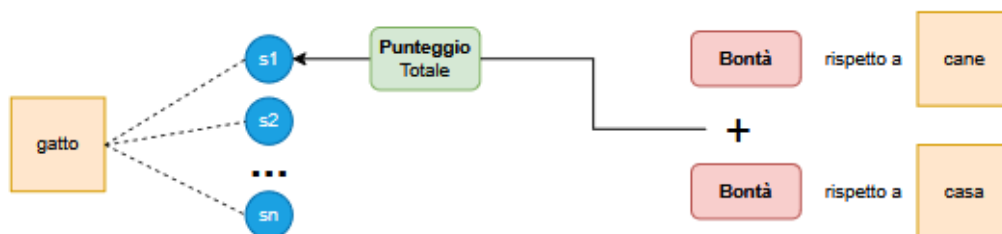
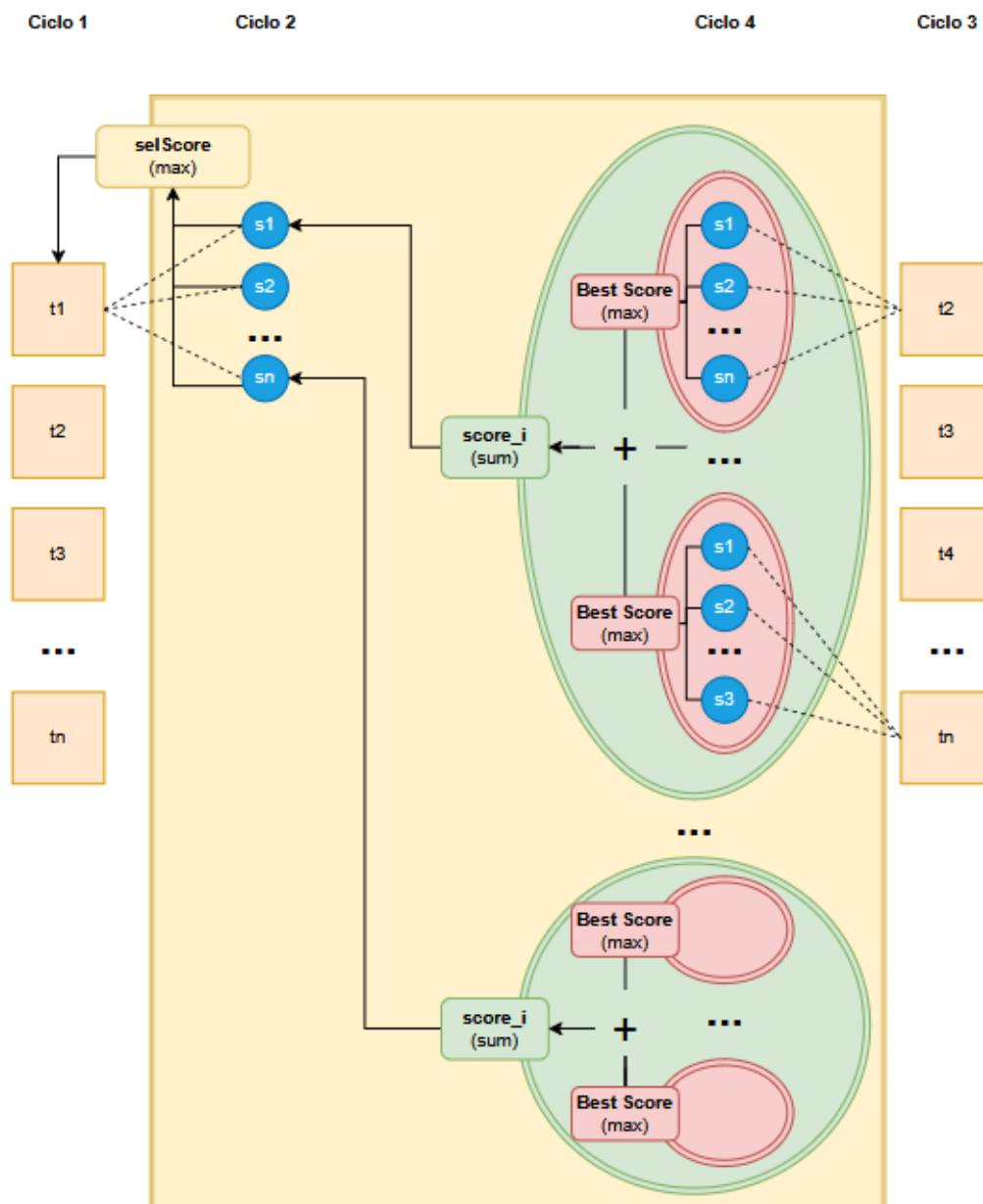


7.4 Thesaurus e WSD

La WSD è *fondamentale* per la progettazione automatica di un thesaurus, poiché permette di risolvere le ambiguità semantiche tra termini polisemi. In un thesaurus, ogni parola deve essere associata al **significato corretto** per evitare confusione tra **concetti simili**.

La WSD assegna a ciascun termine il suo senso appropriato, basandosi sul contesto, migliorando così la coerenza e l'efficacia del thesaurus, soprattutto quando è generato automaticamente da testi o altre fonti.

Semplice Algoritmo WSD Mostratoci dalla Prof - $O(n^4)$



Quanto il primo significato di gatto sta bene con cane? +
 Quanto il primo significato di gatto sta bene con casa? =
Punteggio totale del 1° significato di gatto

8 Query Expansion (QE)

La **Query Expansion** è una tecnica utilizzata nei sistemi di recupero dell'informazione per migliorare le performance della ricerca. L'idea alla base è di ampliare una query originale aggiungendo termini correlati, come sinonimi o varianti semantiche, al fine di ottenere un set di risultati più rilevante.

8.1 Problemi principali della QE

Il problema principale nella Query Expansion è la **selezione dei termini di espansione**, ovvero come scegliere parole che migliorino davvero i risultati della ricerca, evitando al contempo l'inclusione di termini che potrebbero ridurre la precisione (ad esempio aggiungendo rumore).

8.2 Fonti di espansione

Una delle tecniche più comuni per espandere una query è utilizzare risorse come i **thesauri** o database lessicali come **WordNet**, che offrono sinonimi e altre relazioni tra le parole. WordNet fornisce collegamenti semantici come sinonimi (parole con lo stesso significato), iponimi (sottocategorie), e iperonimi (categorie generali).

8.3 Metodi di Espansione

1. **Sinonimi:** Una delle tecniche più semplici consiste nell'aggiungere tutti i sinonimi delle parole della query originale.
2. **Parole correlate:** Oltre ai sinonimi, possono essere usate parole strettamente correlate come iponimi e iperonimi.
3. **Espansione basata sull'analisi delle co-occorrenze:** Tecniche statistiche possono individuare parole che spesso appaiono insieme nella stessa informazione, suggerendo un'espansione basata su queste relazioni.

9 Indicizzazione - Trie, Inverted Indexes

9.1 Introduzione

Gli **Inverted Index** (**indici invertiti**) rappresentano una struttura dati chiave per l'indicizzazione e la ricerca all'interno di grandi collezioni di documenti, come quelle utilizzate nei motori di ricerca e nei database. La funzione principale di un indice invertito è quella di mappare velocemente i documenti che contengono una determinata parola, migliorando drasticamente i tempi di ricerca rispetto a una scansione sequenziale dei testi.

Un indice invertito si compone principalmente di due parti:

1. **Vocabolario**: un insieme di termini (parole) presenti all'interno della collezione di documenti.
2. **Occorrenze**: per ogni termine, una lista di documenti (o delle posizioni nei documenti) in cui quel termine appare.

Una delle sfide principali nella gestione di un Inverted Index è la sua scalabilità ed efficienza, specialmente in grandi collezioni di dati.

Per facilitare la ricerca e l'aggiornamento del vocabolario, viene spesso impiegata una **Trie**, una struttura dati ad albero che consente di organizzare il vocabolario in modo efficiente. La Trie permette ricerche rapide sui termini e garantisce un ordinamento lessicografico, che è cruciale quando si deve accedere velocemente alle liste di occorrenze associate a ciascun termine.

Grazie a questa **combinazione di Inverted Index e Trie**, è possibile costruire un sistema di indicizzazione e ricerca altamente ottimizzato.

Trie	Inverted Index	
	Vocabulary	Posting Lists
/	+-----+	+-----+
// ----->	term1 --->	doc1, doc2
/// ----->	term2 --->	doc1, doc3
\\ ----->	... --->	...
\\ ----->	termN --->	doc2, doc4
\\	+-----+	+-----+

9.2 Composizione

1. **Trie:** Questa struttura dati viene utilizzata per organizzare il vocabolario (l'insieme dei termini unici) in modo efficiente. Il Trie permette di gestire la ricerca di parole in modo rapido e di mantenere l'ordinamento dei termini. Questo è particolarmente utile perché l'ordinamento dei termini è cruciale per eseguire rapidamente ricerche e query. Ogni nodo del Trie contiene parti dei termini, e il percorso verso le foglie rappresenta il termine completo.
2. **Vocabulary:** Ogni termine ($\text{term1}, \text{term2}, \dots, \text{termN}$) è una voce nel vocabolario. Ogni termine viene cercato nel Trie, e se trovato, viene associato a una lista di occurrences (le cosiddette posting lists) che puntano ai documenti dove appare quel termine. La presenza di questo vocabolario organizzato migliora l'efficienza del recupero delle informazioni, in quanto consente di accedere ai documenti correlati al termine con un'operazione relativamente veloce, sfruttando strutture come array ordinati, alberi B+ o, come in questo caso, il Trie.
3. **Posting Lists:** Queste liste contengono l'elenco dei documenti in cui un determinato termine compare. Possono includere anche informazioni aggiuntive, come la frequenza del termine (tf) o la posizione esatta del termine nel documento, utile per query di prossimità (ad esempio, ricerche che verificano se due termini compaiono vicini tra loro). Le posting list sono il cuore dell'Inverted Index, perché consentono di rispondere a query complesse in modo rapido, una volta che il termine è stato identificato nel vocabolario.

Strutture dati di base e notazioni

- n : dimensione della collezione di testi.
- m : lunghezza del pattern di ricerca.
- M : quantità di memoria principale disponibile.

Queste strutture si basano su concetti fondamentali come:

DATA STRUCTURE	ACCESS TIME	SPACE	MEMORY	Notes
<i>Sorted Array</i>	$O(\log(n))$	$O(n)$	<i>PRIMARY</i>	
<i>Binary Search Tree</i>	$O(\log(n))$	$O(n)$	<i>PRIMARY</i>	<i>Deve essere mantenuto Bilanciato</i>
<i>B-tree</i>	$O(\log(n))$	$O(n)$	<i>SECONDARY</i>	
<i>Hash Table</i>	$O(1)$ but $O(k)$	$O(n)$	<i>SECONDARY</i>	$k = \text{records in single memory block}$
<i>Trie</i>	$O(m)$	$O(n)$	<i>PRIMARY</i>	$m = \text{lunghezza della ricerca}$

9.3 Trie

La **Trie** (detta anche **prefix tree**) è una struttura ad albero utilizzata per memorizzare stringhe. Le sue caratteristiche principali sono:

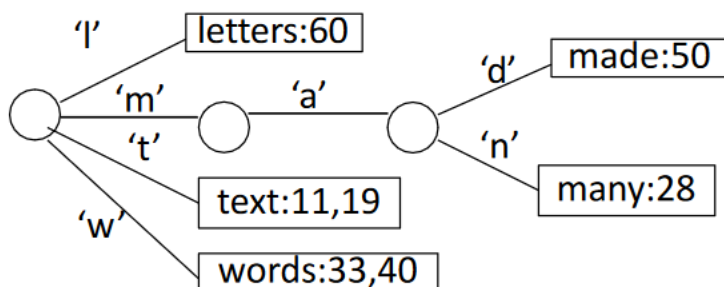
- Ogni nodo rappresenta un prefisso comune tra le stringhe memorizzate.
- Le stringhe si trovano nelle foglie dell'albero.
- Le lettere lungo un percorso dal nodo radice alla foglia rappresentano la stringa memorizzata.

Costruzione della Trie

- Il nodo radice rappresenta il primo carattere.
- I figli rappresentano i caratteri successivi.
- Se una parte della Trie contiene una sola stringa, questa viene memorizzata in un nodo foglia.

Esempio

1 6 9 11 17 19 24 28 33 40 46 50 55 60



Strings to be stored and the corresponding starting positions

letters: 60
made:50
many: 28
text: 11, 19
words: 33,40

Indicare la fine di una parola tramite carattere speciale

Nelle Trie, quando una parola è un prefisso di un'altra, è importante segnalare chiaramente dove termina la parola prefisso. Per farlo, si introduce un carattere speciale o un **marker** che rappresenta la fine della parola (indicato solitamente un simbolo distinto, ad esempio: # o *).

Consideriamo le parole "car" e "cart". Nella Trie, "car" è un prefisso di "cart". Per segnalare il termine della parola "car", aggiungiamo un carattere speciale dopo il nodo "r":

```
(root)
|
c
|
a
|
r -- (* indica la fine di "car")
|
t -- (fine di "cart")
```

Complessità computazionale della Trie

Una delle principali caratteristiche che rende la Trie estremamente efficiente è che la sua complessità computazionale dipende esclusivamente dalla **lunghezza della stringa** che stiamo cercando, e non dalla dimensione totale del vocabolario o della collezione di stringhe memorizzate.

- **Complessità di accesso e inserimento:** Entrambi hanno un costo di $O(m)$, dove m è la lunghezza della stringa.
 - Durante la ricerca, per ogni carattere della stringa, percorriamo un singolo nodo nell'albero, seguendo il percorso corrispondente al carattere.
 - Se la stringa ha una lunghezza di m , il costo sarà direttamente proporzionale a m , indipendentemente dal numero totale di parole memorizzate nella Trie.

Questo rende la Trie molto più efficiente rispetto a strutture dati come alberi di ricerca binari o tabelle hash, dove il tempo di ricerca dipende dalla dimensione del vocabolario o dalla funzione hash.

La **Trie** non è influenzata dalla quantità di parole nel vocabolario. Non importa se la Trie contiene 10 parole o 1 milione di parole, il tempo di accesso rimane $O(m)$, dove m è la lunghezza della stringa.

9.4 Inverted Index

Matrice di Incidenza

La **matrice di incidenza** associa i termini estratti durante la fase di pre-processing ai documenti in cui compaiono. Essa è composta da una riga per ogni termine (o **index term**) e da una colonna per ogni documento. Gli elementi della matrice sono valori binari:

- **1**: indica che il termine è presente in un documento.
- **0**: indica che il termine non è presente in quel documento.

Esempio semplificato:

Termini	Doc1	Doc2	Doc3
<i>Antony</i>	1	1	0
<i>Brutus</i>	1	0	1
<i>Cleopatra</i>	0	0	1

Query booleane come **Brutus AND Caesar AND NOT Calpurnia** possono essere risolte con operazioni bitwise sui vettori dei termini corrispondenti, riducendo la matrice di incidenza ai documenti rilevanti. Tuttavia, la matrice di incidenza ha un problema di **sparsezza**: se i documenti e i termini sono molto numerosi, la maggior parte degli elementi della matrice sono 0, occupando molto spazio senza essere informativi.

Ottimizzazione tramite Inverted Index

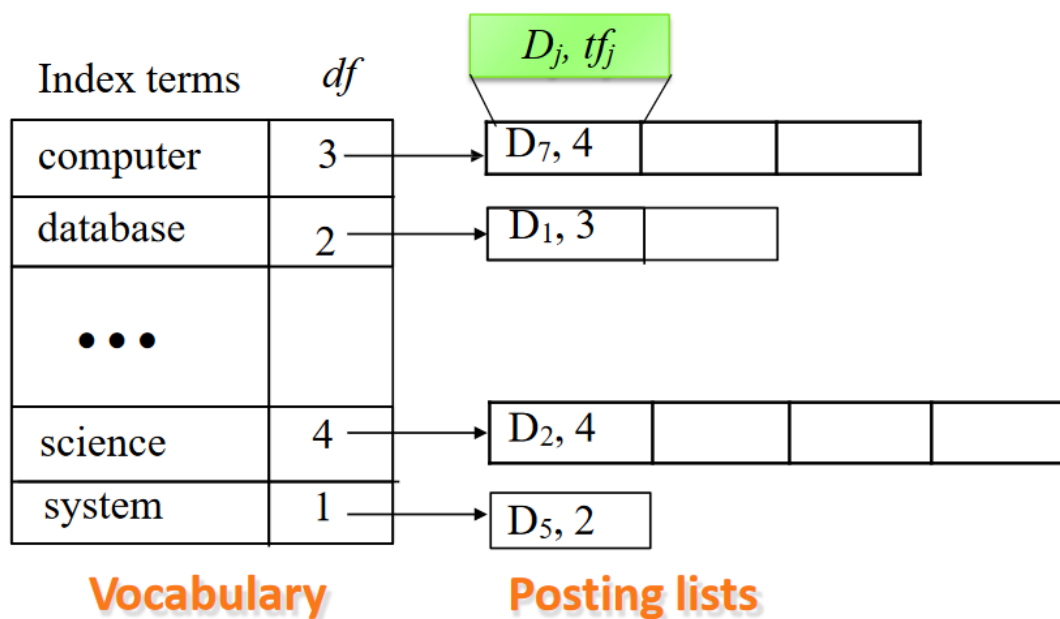
Un metodo più efficiente per rappresentare questa informazione è attraverso l'**Inverted Index**, che memorizza solo le posizioni dei 1 nella matrice, ossia i documenti in cui ogni termine appare. Invece di memorizzare una lunga lista di 0 e 1, per ogni termine si memorizza una lista di documenti associati a quel termine.

- **Keyword**: è la testa della lista puntata di documenti in cui compare quella parola.
 - *Esempio*: Brutus $\rightarrow$ [Doc1, Doc3, Doc5]

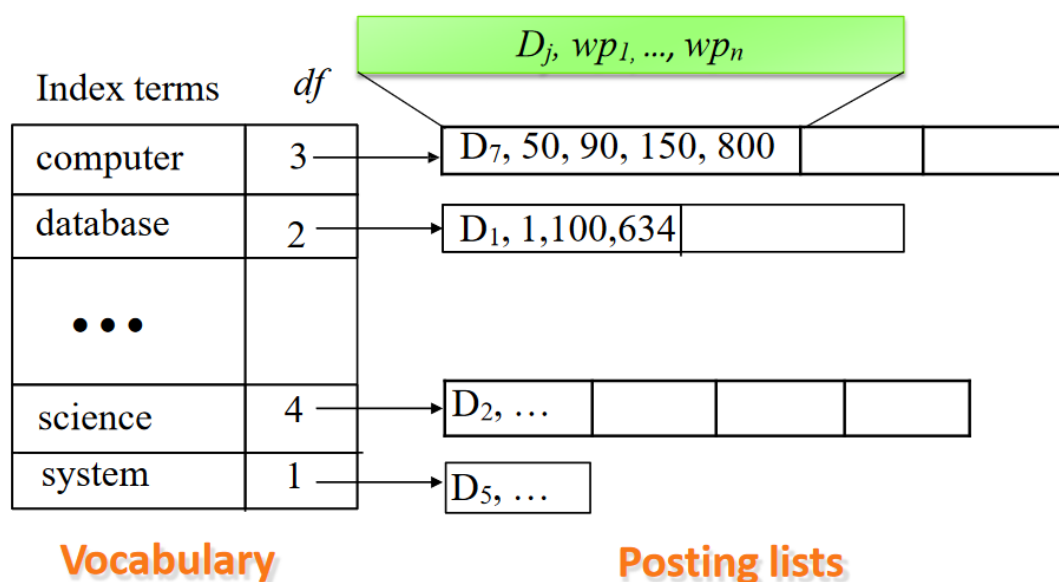
L'Inverted Index supporta sia:

- **Document-based indexing**: memorizza in quali documenti compare un termine e quante volte (tf).
- **Word-based indexing**: memorizza le posizioni precise del termine nel documento per supportare ricerche più sofisticate come quelle di prossimità e con operatori booleani.

Document Based Indexing



Word Based Indexing



Frequenze e Ranking

- **Document Frequency (df):** Indica il numero di documenti in cui un termine appare. Questo valore è utile per il processo di ranking, poiché termini che appaiono in molti documenti (e.g., stopwords) hanno meno peso rispetto a termini più rari.
- **Term Frequency (tf):** Indica quante volte un termine appare in un documento specifico. La lunghezza della lista di posizioni può essere utilizzata per calcolare questa frequenza.

Complessità spaziale dell'Inverted Index

L'Inverted Index, sebbene più efficiente della matrice di incidenza, ha ancora un impatto significativo sullo spazio di memorizzazione, specialmente con grandi quantità di documenti e parole.

- **HEAP's LAW:** La crescita del vocabolario (il numero di termini distinti) segue una legge sub-lineare del tipo $O(n^b)$, dove b è compreso tra 0.4 e 0.6. Questo implica che il vocabolario cresce lentamente rispetto alla dimensione della collezione di documenti.
- **Spazio per il vocabolario:** Il vocabolario richiede poco spazio (solitamente pochi MB anche per collezioni di grandi dimensioni) e può essere ulteriormente ridotto tramite tecniche di pre-processing come **stopword removal**, **stemming** e **lemmatization**. Queste riducono il numero di termini distinti nel vocabolario.
- **Spazio per le occorrenze:** Le occorrenze dei termini occupano molto più spazio. In un inverted index basato su documenti, lo spazio aggiuntivo richiesto può variare dal 20% al 40% della dimensione del testo originale. Con il **word-based indexing**, questo spazio può aumentare notevolmente, poiché oltre ai documenti vengono memorizzate anche le posizioni delle parole.

Problemi nell'uso di una hash table per l'indicizzazione

1. Ordinamento dei termini:

- Le hash table non mantengono l'ordinamento dei termini. Questo significa che non puoi fare **ranked queries** (ad esempio, ricerca di documenti contenenti termini compresi in un intervallo, o di tutti i documenti che contengono termini con un certo prefisso).

2. Problemi di dimensione e collisioni:

- Le hash table possono subire problemi di collisioni, specialmente con un numero elevato di chiavi (i termini). Quando molte chiavi si mappano allo stesso bucket, la ricerca all'interno di questi bucket diventa meno efficiente.

3. Problemi di dimensione:

- La dimensione complessiva di una **hash table** può crescere rapidamente, soprattutto se si cerca di ridurre le collisioni aumentando la dimensione della tabella. Inoltre, non tutte le posizioni della hash table potrebbero essere utilizzate efficientemente, generando un overhead di spazio.

Costruzione e aggiornamento dell'Inverted Index

La costruzione e l'aggiornamento di indici su larga scala è un processo complesso, che richiede ingegneria avanzata per ottimizzare l'uso delle risorse hardware, del sistema operativo e degli algoritmi impiegati. Un algoritmo di indicizzazione deve essere progettato considerando il comportamento su collezioni di documenti molto più grandi rispetto a quelle per cui è stato originariamente concepito.

Passaggi fondamentali per costruire un Inverted Index:

1. **Tokenizzazione:** Ogni documento viene convertito in una sequenza di termini (token). Questo è il primo passo per estrarre i termini dal testo e prepararli per l'indicizzazione.
2. **Aggiornamento dell'Inverted Index:** Per ogni token, si aggiunge il documento corrente alla lista invertita corrispondente a quel termine. Ad esempio, se un termine appare in più documenti, si aggiungono le posizioni in cui appare nel relativo elenco invertito.

Anche se questi passaggi sembrano semplici, la complessità cresce esponenzialmente man mano che la collezione di documenti e il vocabolario aumentano.

Ottimizzazione dell'inverted index

Uno degli approcci per ridurre il carico di memoria durante l'indicizzazione su larga scala è utilizzare una **struttura Trie** per gestire il vocabolario. I passaggi sono:

1. Leggere ogni parola del testo.
2. Cercare la parola nel Trie.
3. Se non esiste, aggiungere la parola al Trie con una lista vuota di occorrenze.
4. Se esiste, aggiungere la nuova posizione alla lista di occorrenze della parola.

Questo metodo assicura un'efficienza operativa, con una complessità $O(1)$ per carattere di testo per la ricerca e l'inserimento delle occorrenze.

Gestione della memoria

Quando l'indice diventa troppo grande per essere gestito in memoria, si scrivono delle liste parziali su disco. Una volta esaurita la memoria disponibile, l'indice in memoria viene svuotato, e una volta terminata l'indicizzazione, tutte le liste parziali vengono unite insieme in un unico indice invertito.

Split dell'indice in due file

Per ottimizzare l'uso della memoria durante la ricerca:

- **Posting file:** Contiene le liste di occorrenze dei termini.
- **Vocabulary file:** Contiene il vocabolario, con il numero di documenti associati ad ogni parola e un puntatore alla lista nel posting file.

Serializzazione

La serializzazione di una struttura dati è il processo con cui i dati presenti in memoria principale vengono convertiti in un formato sequenziale che può essere salvato su memoria secondaria (ad esempio, un disco). Questo processo consente di salvare e recuperare facilmente le strutture dati. Le due operazioni principali sono:

- **serialize:** converte la struttura dati in una sequenza di byte da salvare in memoria secondaria.
- **deserialize:** converte la sequenza di byte dalla memoria secondaria in una struttura dati in memoria principale.

Per quanto riguarda gli **Inverted Index**, il **vocabolario** può essere implementato utilizzando **array ordinati**, che possono essere memorizzati in memoria secondaria utilizzando degli **offset** (vedi esempio slides). Questo approccio permette di serializzare in modo efficiente i dati, mantenendo un accesso rapido ai termini durante la ricerca.

Vocabulary file implemented through a sorted array

Term	Start	n
boundary	0	1
case	1	2
computer	3	2
database	5	1
deliver	6	1
document	7	3
fan	10	3
play	13	1
position	14	1
science	15	2
System	17	1

Posting file

0	002 3	001 2	004 1	001 1	003 2
5	002 1	003 1	001 1	002 1	003 1
10	001 1	002 1	004 2	002 2	001 1
15	003 2	004 1	003 1		

The posting list of "document" starts at position 7 and contains 3 entries.

"document" is contained in documents 001002 003 with frequency ...

Tecniche alternative di indicizzazione

Rispetto ad altre tecniche, come i **Suffix Arrays** e i **Signature Files**, gli indici invertiti sono generalmente considerati la scelta migliore per molte applicazioni moderne. Oltre agli indici invertiti, esistono altre tecniche di indicizzazione, ciascuna con vantaggi e svantaggi specifici:

1. **Suffix Arrays**: strutture dati che permettono una ricerca più rapida per frasi o sequenze, ma risultano più complesse da costruire e mantenere rispetto agli indici invertiti.
2. **Signature Files**: utilizzati negli anni '80, sfruttano il hashing per creare un indice orientato alle parole. Tuttavia, oggi sono caduti in disuso a causa delle loro limitazioni in termini di efficienza e manutenibilità.

9.5 Searching

Il processo di retrieval di documenti rilevanti in un sistema di Information Retrieval (IR) prevede una serie di passaggi che permettono di identificare i documenti contenenti parole o frasi specifiche. Due approcci fondamentali sono il **Single-word retrieval** e il **Phrasal retrieval**, che differiscono nel tipo di query trattate.

Passaggi Generali per il Retrieval

Il processo di ricerca si articola in tre fasi principali:

1. Ricerca nel vocabolario

Viene effettuata un'analisi delle parole o dei pattern presenti nella query. Le query più complesse, come quelle contenenti frasi o condizioni di prossimità, sono suddivise in singole parole. Il processo si concentra quindi sul retrieval di parole singole.

2. Recupero delle occorrenze

Dopo aver isolato le parole o i pattern della query, il sistema recupera le liste delle loro occorrenze, cioè i documenti o le posizioni in cui tali parole sono presenti.

3. Manipolazione delle occorrenze

Le occorrenze recuperate vengono elaborate per rispondere a richieste più complesse, come la gestione di frasi, condizioni di prossimità tra parole o operazioni booleane.

9.6 Single-word Retrieval

Il **Single-word retrieval** si concentra sulla ricerca di singole parole nei documenti. E' il caso più semplice di query, in cui un solo termine è oggetto di ricerca.

- **Query di singole parole**

La ricerca di una singola parola può essere accelerata utilizzando strutture dati adatte, come indici invertiti. Queste strutture aiutano a localizzare rapidamente i documenti in cui la parola appare.

- **Query con prefissi e range**

In alcuni casi, può essere necessario cercare parole che iniziano con un certo prefisso (esempio: tutte le parole che iniziano con "auto") o parole che cadono entro un certo intervallo lessicografico.

Nota: Le query con prefissi o range non possono essere gestite con tecniche di hashing, poiché l'hashing non preserva l'ordine lessicografico necessario per queste ricerche.

- **Query booleane**

Le query booleane combinano più parole o termini con operatori logici come **AND**, **OR**, e **NOT** per effettuare ricerche più complesse:

- **AND:** Restituisce i documenti che contengono tutte le parole specificate (*UNION*).
- **OR:** Restituisce i documenti che contengono almeno una delle parole specificate (*INTERSECTION*).
- **NOT:** Esclude i documenti che contengono una parola specificata (*DIFFERENCE*).

La manipolazione delle occorrenze per queste query viene effettuata combinando le liste di occorrenze dei singoli termini, utilizzando operazioni su insiemi per ottenere il risultato corretto.

Term	Start	n
boundary	0	1
case	1	2
computer	3	2
database	5	1
deliver	6	1
document	7	3
fan	10	3
play	13	1
position	14	1
science	15	2
System	17	1

0	002 3	001 2	004 1	001 1	003 2
5	002 1	003 1	001 1	002 1	003 1
10	001 1	002 1	004 2	002 2	001 1
15	003 2	004 1	003 1		

Q: **database** OR **science** AND **case** ?

database OR **science** AND **case**

$\Rightarrow$ **database** OR ({003,004} $\cap$ {001,004})

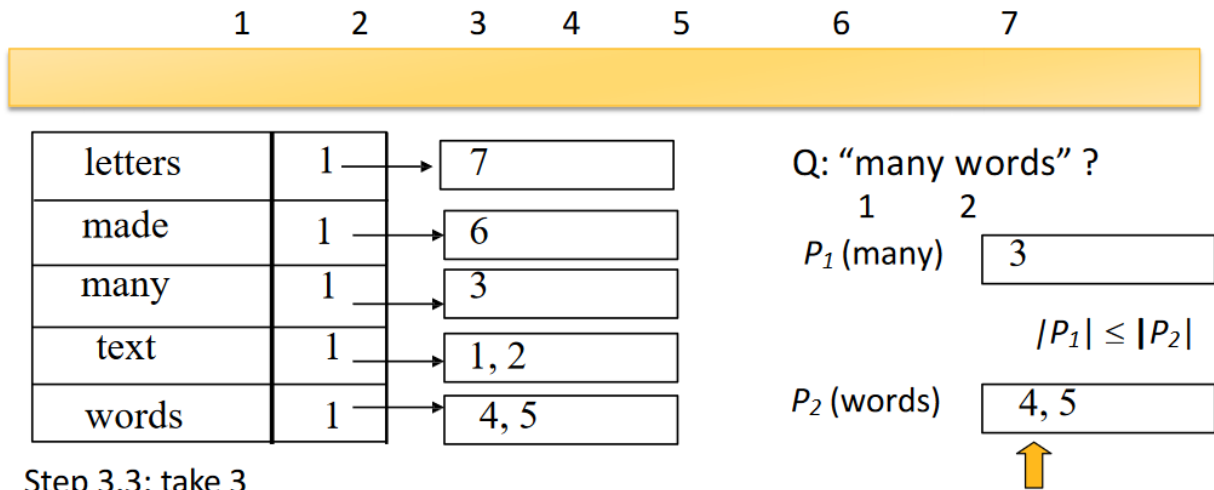
$\Rightarrow$ {002} $\cup$ {004}

$\Rightarrow$ {002,004}

9.7 Phrasal Retrieval

Il **Phrasal retrieval** riguarda la ricerca di frasi, ovvero sequenze di più parole in un ordine specifico.

- **Frazionamento delle frasi** Quando una query contiene una frase, il sistema suddivide la frase in singole parole e recupera le occorrenze di ciascuna parola nei documenti. Tuttavia, per confermare che una frase completa è presente, il sistema deve verificare che le parole appaiano nell'ordine corretto e siano adiacenti.
- **Query di prossimità** A differenza delle frasi esatte, le query di prossimità richiedono che le parole appaiano vicine, ma non necessariamente adiacenti. In questo caso, viene impostata una distanza massima tra le parole (es. "cat NEAR mouse").



Step 3.3: take 3

Step 3.3.1: Use binary search to find the position $(p + i - s) = (3 + 2 - 1)$ in the array P_2

Step 4: add the document to R

9.8 Boosting

Possiamo dare più **peso a una parte specifica della query** per influenzare il ranking dei risultati. Ad esempio, se stiamo cercando "ricetta torta al cioccolato facile" ma vogliamo enfatizzare "cioccolato", possiamo fare boosting di questa parola. Così, i risultati mostrati saranno ordinati per rilevanza, con le pagine che parlano maggiormente di "cioccolato" in cima alla lista, perché questa parola è stata resa più importante rispetto alle altre nella query. Questo avviene a **run-time**, cioè mentre si esegue la ricerca, permettendo all'utente di ottenere risultati più mirati.

10 Modeling

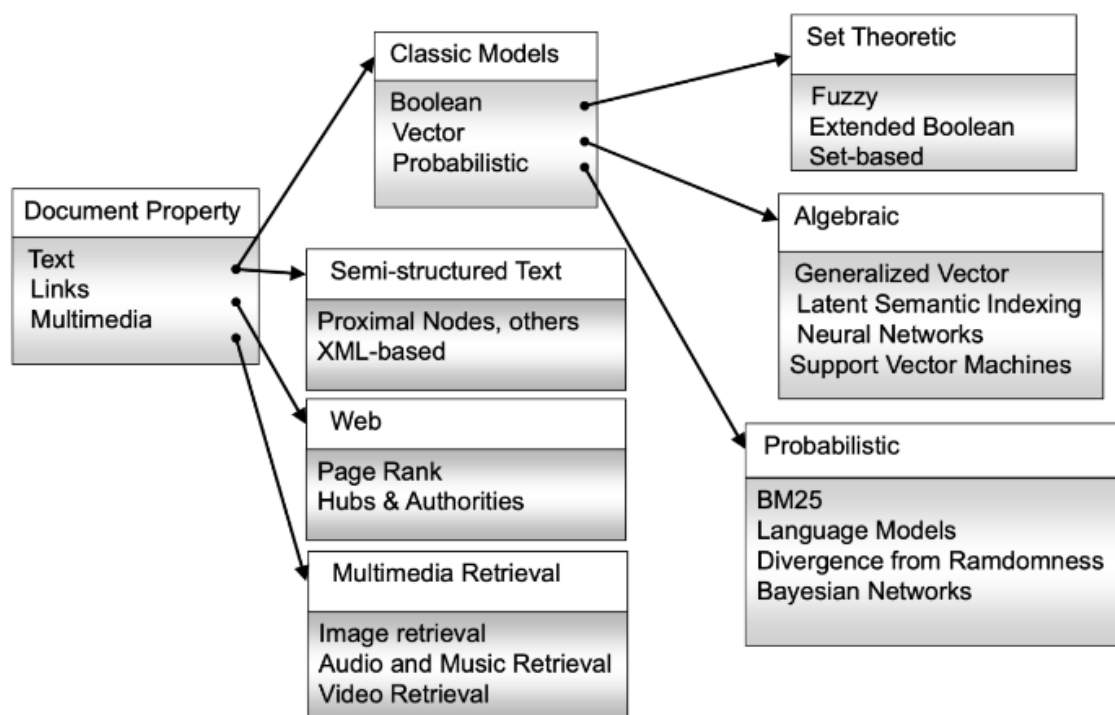
Nel campo dell'Information Retrieval (IR), gli algoritmi di ranking sono fondamentali per ordinare i documenti in base alla loro rilevanza rispetto a una query dell'utente. Questi algoritmi valutano e assegnano un punteggio ai documenti, determinando l'ordine in cui vengono presentati nei risultati di ricerca.

Fattori Considerati negli Algoritmi di Ranking

- **Pertinenza del Contenuto:** Valutazione di quanto il contenuto del documento corrisponda alla query dell'utente.
- **Autorità e Affidabilità:** Misura dell'affidabilità della fonte, spesso determinata da link in entrata da siti autorevoli.
- **Freschezza:** Importanza della data di pubblicazione, soprattutto per argomenti di attualità.
- **Usabilità:** Valutazione dell'esperienza utente, inclusi fattori come la velocità di caricamento e la compatibilità mobile.

10.1 IR Models

Nel campo dell'Information Retrieval (IR), i **modelli di recupero dell'informazione** sono astrazioni che rappresentano il processo ideale di recupero dei documenti rilevanti in risposta a una query dell'utente. Questi modelli forniscono una struttura teorica per comprendere e progettare sistemi di IR efficaci.



Definizione Formale di un Modello di IR

Un modello di IR può essere formalmente descritto come una tripla $(D, Q, R(q_i, d_j))$, dove:

- D : insieme delle rappresentazioni logiche dei documenti presenti nel sistema.
- Q : insieme delle rappresentazioni logiche delle necessità informative degli utenti (User Information Needs, UIN).
- $(D, Q, R(q_i, d_j))$: funzione di ranking che associa a ogni coppia query-documento un valore reale, quantificando la loro similarità o rilevanza.

Questa struttura permette di modellare il processo di recupero come un problema di valutazione della similarità tra la rappresentazione della query e quella dei documenti, al fine di ordinare i documenti in base alla loro pertinenza rispetto alla query.

Caratteristiche dei Modelli di IR

- **Astrazione del Processo di Recupero:** Un modello di IR idealizza il processo effettivo di recupero, fornendo una rappresentazione semplificata che facilita l'analisi e la progettazione del sistema.
- **Validità del Modello:** Le conclusioni derivate da un modello dipendono dalla sua capacità di approssimare accuratamente la situazione reale di recupero. Un modello efficace deve riflettere le caratteristiche essenziali del processo di recupero per fornire risultati utili.
- **Distinzione tra Modello e Implementazione:** E' importante notare che un modello di recupero non coincide con la sua implementazione pratica. Mentre il modello fornisce una struttura teorica, l'implementazione riguarda la realizzazione concreta del sistema basato su quel modello.

Principali Modelli di Information Retrieval

- **Modello Booleano:** Uno dei primi modelli sviluppati, si basa sull'algebra booleana e sulla teoria degli insiemi. I documenti e le query sono rappresentati come insiemi di termini, e il recupero si basa su operazioni logiche come AND, OR e NOT. Questo modello è semplice ma presenta limitazioni nella gestione della rilevanza parziale dei documenti.
- **Modello Vettoriale:** Rappresenta documenti e query come vettori in uno spazio multidimensionale, dove ogni dimensione corrisponde a un termine del vocabolario. La similarità tra una query e un documento è misurata tramite l'angolo tra i rispettivi vettori, spesso utilizzando la misura del coseno. Questo modello gestisce meglio la rilevanza parziale rispetto al modello booleano.
- **Modello Probabilistico:** Si basa sulla probabilità che un documento sia rilevante per una data query. Utilizza informazioni statistiche per stimare questa probabilità, considerando fattori come la frequenza dei termini e la distribuzione dei termini nei documenti rilevanti e non rilevanti.
- **Modelli Avanzati:** Con l'evoluzione della tecnologia, sono stati sviluppati modelli più sofisticati, come quelli basati su reti neurali e tecniche di machine learning. Questi modelli cercano di catturare relazioni più complesse tra documenti e query, migliorando la capacità del sistema di comprendere il linguaggio naturale e il contesto.

10.2 Similarità

Similarità e Dissimilarità

Similarità e **dissimilarità** sono misure fondamentali per valutare quanto due oggetti siano simili o diversi tra loro, ed entrambi trovano applicazione in molti campi, come il riconoscimento di pattern, l'analisi dei dati, e il recupero dell'informazione. Ecco alcune definizioni più dettagliate:

- **Similarità:** E' una misura che aumenta al crescere della somiglianza tra due oggetti. Un numero più alto indica maggiore similarità, quindi più i due oggetti sono "simili".
- **Dissimilarità** (o "distanza"): E' il concetto opposto e aumenta al diminuire della somiglianza. In pratica, un numero più alto indica una maggiore differenza tra gli oggetti. In molte applicazioni, il termine "distanza" viene utilizzato come sinonimo di dissimilarità.

Nota: La similarità e la dissimilarità sono spesso correlate, ma non sono esattamente l'inverso l'una dell'altra. Alcune misure specifiche devono essere definite in base al contesto.

Perché misurare la (dis)similarità?

Misurare la similarità o dissimilarità è utile in diversi ambiti, come:

1. **Classificazione:** Dato un oggetto non etichettato Q , lo si può assegnare a una classe tra quelle predefinite, confrontandolo con oggetti noti e classificati. Sebbene non sia l'unico modo per classificare, i metodi basati sulla similarità sono molto efficaci.
2. **Clustering:** Consiste nel trovare raggruppamenti naturali di oggetti. Gli oggetti all'interno dello stesso cluster risultano essere molto simili secondo una misura di similarità specifica.
3. **Query by Content:** Nell'Information Retrieval, spesso si cerca un oggetto nel sistema che somigli al massimo possibile all'oggetto di una query. Ad esempio, si potrebbe voler trovare l'immagine più simile a una data immagine, o il documento più pertinente a una query di testo.

Misurare la distanza tra oggetti

Per calcolare la dissimilarità tra due oggetti O_1 e O_2 , si utilizza una misura di distanza, spesso indicata come $D(O_1, O_2)$, che restituisce un numero reale. La scelta della distanza dipende dal tipo di oggetti e dal contesto in cui viene applicata. Alcuni esempi comuni di misure di distanza includono:

- **Distanza Euclidea:** Utilizzata per variabili numeriche, misura la distanza "lineare" tra due punti nello spazio.
- **Distanza di Manhattan:** Somma delle differenze assolute tra le coordinate. E' utile quando le variabili non sono continue o quando le coordinate rappresentano categorie.
- **Distanza Coseno:** Misura l'angolo tra due vettori e si usa spesso per dati testuali.

Rappresentazione logica (features) e misurazione della (dis)similarità

Per confrontare due oggetti, è necessario rappresentarli attraverso delle caratteristiche misurabili (chiamate anche **feature**). La (dis)similarità dipende:

- dalla rappresentazione logica degli oggetti (cioè quali caratteristiche scegliamo di considerare);
- dalla misura di distanza che adottiamo.

Esempio:

Se volessimo misurare la similarità tra due documenti, dovremmo rappresentarli come vettori di parole o termini (ad esempio, come vettori di frequenze di parole). La similarità tra i documenti sarà poi calcolata in base alle loro rappresentazioni vettoriali e alla misura di distanza scelta (come la distanza coseno o la distanza euclidea).

Estrazione delle caratteristiche (Feature Extraction)

L'estrazione delle caratteristiche è un passo cruciale perché determina quali informazioni useremo per confrontare gli oggetti. In base ai dati e al contesto, potremmo:

1. **Generare caratteristiche:** A volte le informazioni di base non sono sufficienti. Per esempio, se stiamo analizzando la similarità di persone dal punto di vista medico, altezza e peso potrebbero non essere abbastanza significativi separatamente. Invece, il BMI (Body Mass Index), calcolato come peso diviso per il quadrato dell'altezza, può essere una caratteristica utile.
2. **Pulire le caratteristiche:** I dati grezzi possono contenere errori o valori estremi (outlier) che devono essere corretti o rimossi. Un esempio è eliminare valori di peso o altezza impossibili nei dati medici.
3. **Normalizzare le caratteristiche:** A volte, è necessario ridimensionare le caratteristiche per renderle confrontabili. Ad esempio, se confrontiamo altezza (misurata in metri) e peso (in chili), i valori di peso potrebbero dominare il calcolo della distanza. La normalizzazione aiuta a bilanciare le caratteristiche, portandole su una scala comune.
4. **Ridurre le caratteristiche:** In alcuni casi, possiamo avere troppe caratteristiche, il che rende il confronto inefficiente e complesso. Ridurre la dimensionalità (ad esempio, tramite tecniche come l'analisi delle componenti principali - PCA) può rendere il calcolo della similarità più efficiente.

In sintesi

1. **Definizione di Similarità e Dissimilarità:** *La similarità misura quanto due oggetti siano simili, mentre la dissimilarità (o distanza) misura quanto siano diversi.*
2. **Uso delle misure di (dis)similarità:** *Sono usate per compiti come classificazione, clustering e query by content.*
3. **Misure di distanza:** *Possono includere la distanza euclidea, di Manhattan, e il coseno, tra le altre, e devono essere scelte in base ai dati e agli obiettivi.*
4. **Rappresentazione e Estrazione delle caratteristiche:** *Gli oggetti devono essere rappresentati tramite caratteristiche significative e pulite; in alcuni casi, queste devono essere **normalizzate** o **ridotte** per ottimizzare il processo di misurazione.*

10.3 Rappresentazione Logica dei Documenti

La rappresentazione logica dei documenti in IR si basa sul concetto di **bag of words** (o **bag of index terms**), un metodo in cui ogni documento è rappresentato come un insieme di termini chiave. Si tratta degli index terms ricavati dalla **fase di pre-processing**.

Bag of Index Terms (sacco di termini):

- I documenti vengono rappresentati come insiemi di parole chiave, dette anche index terms o termini di indicizzazione. **Non viene considerato l'ordine** di questi termini.
 - **Vantaggi:** Semplifica il modello e riduce la complessità del calcolo.
 - **Limitazioni:** Non cattura la struttura e l'ordine sintattico, che a volte può essere importante per il significato.
- Ogni documento viene scomposto in parole singole, o keywords, che rappresentano i temi principali.
- Questa rappresentazione ignora la struttura sintattica o l'ordine delle parole, concentrandosi solo sulla presenza e sulla frequenza delle parole.
- Questo approccio è comunemente usato dai motori di ricerca web per rappresentare il contenuto delle pagine.

10.4 Termini di Indicizzazione

Sono parole che rappresentano i temi principali di un documento. Spesso sono sostantivi, poiché questi hanno un significato intrinseco che aiuta a identificare l'argomento del testo. Ad esempio, in un articolo di biologia, termini come "cellula", "DNA" e "proteina" sarebbero buoni candidati come index terms.

Pesi dei Termini (Weights):

- Non tutti i termini sono ugualmente utili per descrivere il contenuto di un documento.
- I pesi indicano l'importanza di ogni termine per un documento specifico. Ad esempio, termini come "importante" e "molto" possono essere comuni in molti documenti, ma non descrivono bene il contenuto.
- I pesi sono generalmente determinati attraverso metriche come TF-IDF (Term Frequency-Inverse Document Frequency), che dà un peso maggiore ai termini frequenti nel documento ma rari nel resto della collezione.

Formalmente:

- **Set di Termini:** Sia $\mathbf{K} = \{\mathbf{k}_1, \dots, \mathbf{k}_t\}$ l'insieme di tutti i termini di indicizzazione.
- **Peso:** Per ogni termine $\mathbf{k}_i$ e documento $\mathbf{d}_j$, esiste un peso $\mathbf{w}_{ij} \geq 0$ che rappresenta l'importanza di $\mathbf{k}_i$ in $\mathbf{d}_j$.
- **Rappresentazione del Documento:** Un documento può essere rappresentato come un vettore di pesi:

$$\mathbf{d}_j = (w_{1j}, w_{2j}, \dots, w_{tj})$$

Questo vettore rappresenta il documento nello spazio delle caratteristiche, dove ogni componente corrisponde al peso di un termine.

Indipendenza dei Termini (Mutual Independence):

- Solitamente, si assume che i pesi dei termini siano **indipendenti** tra loro.
- Questo significa che il peso di un termine non influenza il peso di un altro termine nel documento.
- L'indipendenza dei pesi rende il calcolo della similarità più semplice e permette una **classificazione e ranking** più veloci.

10.5 Modello Booleano e Query Booleane

Il modello Booleano di Information Retrieval è un sistema di recupero di documenti basato su espressioni logiche e sulla teoria degli insiemi. E' uno dei modelli più semplici e storicamente uno dei primi utilizzati nei sistemi IR, specialmente in ambito commerciale.

Le query Booleane permettono di costruire espressioni logiche per cercare documenti contenenti parole chiave specifiche, utilizzando gli operatori logici **AND**, **OR** e **NOT**.

Caratteristiche del Modello Booleano

Il modello Booleano è costruito su basi di **teoria degli insiemi** e **algebra booleana**. Ogni documento può essere visto come un insieme di termini, e le query booleane determinano quali documenti sono rilevanti attraverso operazioni tra insiemi.

1. Rappresentazione Binaria dei Termini:

- I pesi dei termini nei documenti sono rappresentati in modo binario: se un termine t_i è presente in un documento $\mathbf{d}_j$, il peso w_{ij} è uguale a 1, altrimenti è uguale a 0.

2. Semantica degli Operatori Booleani:

- **AND:** Il documento deve contenere entrambi i termini. Formalmente, per due termini t_x e t_y :

$$t_x \text{ AND } t_y = \{d_j \mid w_{xj} = 1\} \cap \{d_j \mid w_{yj} = 1\}$$

- **OR:** Il documento può contenere almeno uno dei termini:

$$t_x \text{ OR } t_y = \{d_j \mid w_{xj} = 1\} \cup \{d_j \mid w_{yj} = 1\}$$

- **NOT:** Il documento non deve contenere un termine:

$$\text{NOT } t_x = \{d_j \mid w_{xj} = 0\}$$

Queste operazioni permettono di selezionare esattamente i documenti che soddisfano i criteri specificati, senza approssimazioni.

Vantaggi e Svantaggi del Modello Booleano

Il modello Booleano offre alcuni vantaggi e presenta anche delle limitazioni importanti.

Vantaggi:

- **Formalismo Chiaro:** Le query booleane hanno una semantica precisa, quindi l'utente può essere sicuro che i documenti recuperati corrispondano esattamente alla sua richiesta.
- **Semplicità di Implementazione:** Il modello è relativamente semplice da implementare e ha costituito il principale strumento di recupero di informazioni per diversi decenni.

Svantaggi:

- **Matching Esatto:** Il modello Booleano si basa su un confronto binario: un documento è rilevante o non lo è, senza gradazioni intermedie. Questo significa che non è possibile esprimere una corrispondenza parziale.
- **Recupero Troppo Ampio o Troppo Limitato:** Le query booleane possono restituire troppi o troppo pochi documenti. Una query troppo ampia può restituire molti documenti non rilevanti, mentre una query troppo ristretta può restituire pochi o nessun documento.
- **Mancanza di Ranking:** Il modello Booleano non fornisce un meccanismo per ordinare i documenti in base alla loro rilevanza. Questo è un limite rispetto ai modelli basati su ranking, come il modello vettoriale, che permette di ordinare i risultati.
- **Difficoltà di Espressione delle Richieste:** Per alcuni utenti, potrebbe risultare complicato tradurre una richiesta informativa complessa in un'espressione booleana efficace. Scrivere query booleane dettagliate può essere difficile e poco intuitivo.

10.6 Modello Vector Space

Il modello a spazio vettoriale (Vector Space Model - VSM) è uno dei pilastri dell'Information Retrieval (IR) moderno, ampiamente utilizzato per rappresentare documenti e query come vettori in uno spazio multidimensionale.

Questo modello permette di confrontare i contenuti in base alla **somiglianza** tra le loro **rappresentazioni vettoriali**, senza richiedere corrispondenze esatte tra i **termini presenti**. La flessibilità del VSM lo rende particolarmente adatto per gestire **documenti non strutturati** e **query di testo libero**.

Principi chiave del modello

1. Rappresentazione Vettoriale

- Ogni documento e query viene rappresentato come un vettore $\vec{d}_j$ o $\vec{q}_j$ rispettivamente, in uno spazio t -dimensionale.
- La dimensione t rappresenta il numero di termini distinti (vocaboli) che rimangono dopo il preprocessing (ad esempio, rimozione di stopwords e applicazione dello stemming).

2. Peso dei Termini

- Ogni termine i in un documento o in una query riceve un peso reale w_{ij} , che indica la sua rilevanza. Ad esempio:
 - $w_{ij} = 0$ se il termine non è presente o non ha rilevanza in un determinato documento.
- I pesi possono essere calcolati in vari modi, come con la frequenza dei termini (TF) o con misure combinate come $TF - IDF$. *Vedi dopo.*

3. Calcolo della Similarità

- La **similarità** tra una query e un documento (o tra due documenti) viene calcolata utilizzando una misura di similarità.
- La più comune è la **similarità coseno** (**cosine similarity**), che utilizza il prodotto scalare normalizzato per valutare l'angolo tra i vettori. *Vedi Dopo.*

Spazio Vettoriale

In questo modello, ogni termine viene rappresentato come un vettore in uno **spazio a t dimensioni**, con t che rappresenta il numero totale di **termini distinti** (dopo la fase di pre-processing).

$$\begin{aligned} T_1 &= (1, 0, \dots, 0) \\ T_2 &= (0, 1, \dots, 0) \\ &\dots \\ T_n &= (0, 0, \dots, 1) \end{aligned} \tag{1}$$

I vettori ortogonali $T_1, T_2, \dots, T_n$ rappresentano la base ortogonale di questo spazio, in cui ogni vettore rappresenta un **termine unico** e **non ha correlazione con gli altri**. Questa ortogonalità **implica che le features, ossia i termini, sono considerate mutualmente indipendenti**.

La **rappresentazione ortogonale semplifica il calcolo della similarità tra documenti e query**, perché si può supporre che l'**assenza di un termine in un vettore non influisca sugli altri**. La similarità tra i vettori si basa quindi sull'angolo tra di essi, calcolato con la similarità coseno, che sfrutta questa struttura ortogonale.

Calcolo della Similarità

Una volta rappresentati documenti e query come vettori, è possibile confrontarli utilizzando una **misura di similarità**. Tra le più diffuse in IR, la **similarità coseno** valuta l'angolo tra due vettori nello spazio vettoriale, quantificando il grado di somiglianza tra le loro rappresentazioni. La similarità coseno si calcola come:

$$\vec{q} = (w_{1q}, w_{2q}, \dots, w_{tq}) \quad w_{iq} \geq 0$$

$$\vec{d}_j = (w_{1j}, w_{2j}, \dots, w_{tj}) \quad w_{ij} \geq 0$$

$$\text{sim}(\vec{d}_j, \vec{q}) = \frac{\vec{d}_j \cdot \vec{q}}{|\vec{d}_j| \times |\vec{q}|} = \frac{\sum_{i=1}^t w_{ij} \cdot w_{iq}}{\sqrt{\sum_{i=1}^t w_{ij}^2} \cdot \sqrt{\sum_{i=1}^t w_{iq}^2}}$$

Questa formula calcola l'angolo tra i due vettori, dove:

- $\alpha = 0$ (*coseno* = 1) indica **massima similarità** (i vettori sono paralleli).
- $\alpha = 90$ (*coseno* = 0) indica **nessuna similarità** (i vettori sono ortogonali).

La radice quadrata nel **denominatore normalizza i vettori** alla loro lunghezza, rendendo la misura di **similarità indipendente dalla lunghezza** dei documenti. Questa proprietà è utile perché garantisce che i documenti più lunghi non siano avvantaggiati nella misura di similarità.

Il vettore normalizzato $|\vec{q}|$ non affetta il ranking, quindi può essere rimosso dalla formula in modo da semplificarla.

Example

Q: "gold silver truck"

D₁: "Shipment of gold damaged in a fire"

D₂: "Delivery of silver arrived in a silver truck"

D₃: "Shipment of gold arrived in a truck"

D₄: "Shipment of gold damaged arrived in a truck"

	arrive	damage	deliver	fire	gold	silver	shipment	truck
D₁	0	1	0	1	1	0	1	0
D₂	1	0	1	0	0	1	0	1
D₃	1	0	0	0	1	0	1	1
D₄	1	1	0	0	1	0	1	1
Q	0	0	0	0	1	1	0	1

$$\text{sim}(Q, D_1) = \frac{1}{\sqrt{4} \times \sqrt{3}} = 0.29$$

$$\text{sim}(Q, D_2) = \frac{2}{\sqrt{4} \times \sqrt{3}} = 0.58$$

$$\text{sim}(Q, D_3) = \frac{2}{\sqrt{4} \times \sqrt{3}} = 0.58$$

$$\text{sim}(Q, D_4) = \frac{2}{\sqrt{5} \times \sqrt{3}} = 0.52$$

Operazione di **AND** bit-a-bit tra il documento e la query, dove si conta quanti bit "1" risultano alla fine. La differenza tra i documenti **D₂**, **D₃** e **D₄** sta nel fatto che **D₄** contiene un termine in più che non corrisponde a nessun termine della query. Pertanto, un documento con più termini corrispondenti alla query e meno termini non corrispondenti sarà classificato più in alto nel ranking.

Alternative common measures

	Binary	Weighted
Cosine coefficient	$\frac{ \vec{d}_j \cap \vec{q} }{\sqrt{ \vec{d}_j } \times \sqrt{ \vec{q} }}$	$\frac{\sum_{i=1}^t w_{ij} \times w_{iq}}{\sqrt{\sum_{i=1}^t w_{ij}^2} \times \sqrt{\sum_{i=1}^t w_{iq}^2}}$
Dice's coefficient	$\frac{2 \vec{d}_j \cap \vec{q} }{ \vec{d}_j + \vec{q} }$	$\frac{2\sum_{i=1}^t w_{ij} \times w_{iq}}{\sum_{i=1}^t w_{ij}^2 + \sum_{i=1}^t w_{iq}^2}$
Jacquard measure	$\frac{ \vec{d}_j \cap \vec{q} }{ \vec{d}_j + \vec{q} - \vec{d}_j \cap \vec{q} }$	$\frac{\sum_{i=1}^t w_{ij} \times w_{iq}}{\sum_{i=1}^t w_{ij}^2 + \sum_{i=1}^t w_{iq}^2 - \sum_{i=1}^t w_{ij} \times w_{iq}}$
Minkowski distance	$\left(\vec{q} + \vec{d}_j - 2 \vec{d}_j \cap \vec{q} \right)^{1/r}$	$\left(\sum_{i=1}^t w_{ij} - w_{iq} ^r\right)^{1/r}$
Euclidian distance (r=2)		$\sqrt{\sum_{i=1}^t (w_{ij} - w_{iq})^2}$

Peso dei Termini

Il peso di un termine in un documento aiuta a distinguere tra termini che sono rilevanti per rappresentare il contenuto del documento da quelli meno rilevanti. In particolare:

- **Termini Rilevanti:** Vengono assegnati pesi elevati ai termini che sono prominenti e distintivi in un documento.
- **Termini Comuni:** I termini che compaiono in molti documenti solitamente ricevono pesi bassi perché non contribuiscono molto a differenziare il contenuto di un documento dagli altri.

I pesi sono quindi utilizzati per valorizzare l'importanza di un termine all'interno di un documento specifico rispetto alla sua importanza generale in tutta la collezione di documenti.

Term Frequency (tf)

La frequenza del termine, indicata come $TF(t, d)$, misura la frequenza relativa di un termine t all'interno di un documento d . Ci sono diverse varianti di calcolo per il TF :

1. **Frequenza Binaria:** Impostata a 1 se t è presente in d e a 0 altrimenti.
2. **Frequenza Grezza:** Calcolata come il conteggio $f_{t,d}$ delle occorrenze del termine t in d .

$$TermFrequency(t, d) = f_{t,d}$$

3. **Frequenza Logaritmica:** Usa il logaritmo della frequenza grezza per evitare valori estremi nei documenti lunghi.

$$TermFrequency(t, d) = \log(1 + f_{t,d})$$

4. **Frequenza Normalizzata:** Il numero di volte che t appare in d diviso per il numero totale di termini in d .

$$TermFrequency(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

5. **Doppia Normalizzazione:** Per evitare un bias verso documenti più lunghi, è possibile usare una doppia normalizzazione:

$$TermFrequency(t, d) = 0.5 + 0.5 \times \frac{f_{t,d}}{\max\{f_{t',d} : t' \in d\}}$$

Inverse Document Frequency (idf)

L'**Inverse Document Frequency (idf)** misura quanto un termine t è raro in una collezione globale di documenti, rappresentata come D . In altre parole la rarità di un termine t è dettata a partire dall'insieme totale di documenti, e non da singoli documenti specifici. E' calcolata come:

$$InverseDocumentFrequency(t, D) = \log \frac{N}{|\{d \in D : t \in d\}|}$$

dove:

- N è il numero totale di documenti nella collezione.
- $|\{d \in D : t \in d\}|$ è il numero di documenti in cui appare termine t .

Alcune **varianti** dell'idf includono:

- **Idf Smussato**: Aggiunge un valore al denominatore per evitare divisioni per zero.

$$InverseDocumentFrequency(t, D) = \log \left(\frac{N}{1 + |\{d \in D : t \in d\}|} \right) + 1$$

- **Idf Probabilistico**: Basato sulla probabilità inversa.

$$InverseDocumentFrequency(t, D) = \log \left(\frac{N - |\{d \in D : t \in d\}|}{|\{d \in D : t \in d\}|} \right)$$

Term Frequency-Inverse Document Frequency (tf-idf)

La misura **tf-idf** combina **tf** e **idf** per valutare il peso di un termine in relazione sia alla sua importanza in un singolo documento sia alla sua rarità nella collezione di documenti:

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Questa formula tende a dare un peso elevato ai termini che:

- Compiono molte volte nel documento (alto tf).
- Sono rari in altri documenti (alto idf).

Esempio d'uso: Nelle query, i termini rari come *"arachnocentric"* sono considerati importanti, poiché un documento contenente un termine raro è più probabile che sia rilevante per una query contenente quel termine.

Considerazioni sull'Idf nel Ranking dei Documenti

- L'idf non influenza il ranking dei documenti per query con **un solo termine**, poiché l'importanza è attribuita interamente a quel termine.
- Per query con più termini, l'idf permette di dare **maggior peso ai termini meno frequenti nella collezione**. Ad esempio, per una query come *"persona capricciosa"*, l'idf farà sì che *"capricciosa"* sia valutata più rilevante di *"persona"*, migliorando la precisione del ranking.

Vantaggi e Svantaggi del Vector Model

• Vantaggi

- **Miglioramento delle prestazioni di recupero:** L'uso di uno schema di ponderazione dei termini, come il tf-idf, permette di attribuire più importanza ai termini più rilevanti, migliorando l'efficacia nel recupero dei documenti pertinenti.
- **Strategia di matching parziale:** Consente di recuperare documenti che approssimano le condizioni della query, non solo i documenti che la soddisfano esattamente. Questo permette di ottenere risultati anche se non tutti i termini della query sono presenti nel documento.
- **Formula di ranking con il coseno:** Utilizza la similarità del coseno tra il vettore del documento e quello della query per ordinare i documenti in base al loro grado di somiglianza alla query.

• Svantaggi

- **Mancanza di informazioni semantiche e sintattiche:** Il modello non considera né la semantica né la sintassi, trattando i termini come unità isolate senza tenere conto del contesto.
- **Indipendenza dei termini:** Si assume che i termini dell'indice siano indipendenti tra loro, il che ignora potenziali relazioni semantiche o sintattiche tra i termini.
- **Limiti dello schema tf-idf:** Il tf-idf non considera le dipendenze tra i termini dell'indice. Anche se in teoria l'inclusione delle dipendenze tra termini potrebbe migliorare la rappresentazione, in pratica ciò potrebbe risultare controproducente, aumentando la complessità del modello senza migliorare significativamente i risultati.

10.7 Implementazione Modello Space Vector

In questo modello, ogni dimensione corrisponde a un termine del vocabolario, e i pesi associati riflettono l'importanza dei termini nei documenti e nelle query.

Utilizzo dell'Inverted Index nel Modello dello Spazio Vettoriale

Un **inverted index** è una struttura dati che associa a ciascun termine una lista di documenti in cui il termine appare. Questa struttura è cruciale per migliorare l'efficienza del recupero delle informazioni nel modello dello spazio vettoriale, specialmente quando si calcola la similarità tra query e documenti.

Gestione dei Vettori Sparsi

Poiché sia le query che i documenti contengono solo una piccola parte del vocabolario totale, i loro vettori sono **sparsi**, con molti elementi pari a zero. Calcolare direttamente la similarità coseno su vettori così grandi sarebbe inefficiente. L'inverted index permette di concentrarsi solo sui termini presenti sia nella query che nei documenti, riducendo significativamente il numero di calcoli necessari.

10.8 Modello Probabilistico

L'obiettivo principale di qualsiasi modello di Information Retrieval (IR) è determinare quanto un documento sia rilevante rispetto a una query data. L'approccio probabilistico utilizza la teoria della probabilità per affrontare l'incertezza nella stima della rilevanza.

Il **Probabilistic Ranking Principle (PRP)** afferma che:

- *Per ottenere il miglior risultato possibile, i documenti devono essere ordinati in base alla loro probabilità stimata di rilevanza rispetto alla query, in ordine decrescente.*

10.9 Binary Independence Model (BIM)

Definizione

Il **Binary Independence Model (BIM)** è un modello probabilistico di Information Retrieval. E' basato su due concetti principali:

1. **Rilevanza binaria:** un documento è rilevante ($R = 1$) o non rilevante ($R = 0$) rispetto a una query.
2. **Indipendenza dei documenti:** la rilevanza di un documento è considerata indipendente dalla rilevanza degli altri documenti.

Scopo

Stimare la probabilità che un documento d sia rilevante rispetto a una query q , indicata come:

$$P(R = 1 | \vec{d}, \vec{q})$$

Questa probabilità è usata per ordinare i documenti: quelli con probabilità più alta di rilevanza vengono mostrati per primi.

Come funziona il BIM

1. Assunzioni principali:
 - La rilevanza di ciascun documento è binaria ($R = 1$ o $R = 0$).
 - I termini nei documenti sono considerati indipendenti tra loro.
 - La rilevanza di un documento non dipende dalla rilevanza degli altri documenti.
2. Calcolo della probabilità di rilevanza:
 - Si stimano le statistiche che influenzano la rilevanza, come:
 - **Term Frequency (TF):** quante volte un termine appare nel documento.
 - **Document Frequency (DF):** in quanti documenti appare un termine.
 - **Document Length:** lunghezza complessiva del documento.
 - Queste statistiche vengono combinate per calcolare:

$$P(R = 1 | \vec{d}, \vec{q})$$

3. Matching query-document:
 - La query viene confrontata con i documenti attraverso un calcolo basato sui termini d'indice.
 - Il sistema stima in modo incerto quanto un documento soddisfi la query, usando le probabilità calcolate.
4. Ordinamento dei documenti:
 - I documenti vengono ordinati in base a $P(R = 1 | \vec{d}, \vec{q})$, in ordine decrescente.

PROCEDIMENTO E FORMULE

Notazioni e Definizioni

1. $P(\mathbf{R} = 1|\tilde{\mathbf{d}}_j, \tilde{\mathbf{q}}) = P(\mathbf{R}|\tilde{\mathbf{d}}_j, \tilde{\mathbf{q}})$: rappresenta la probabilità che il documento $\tilde{\mathbf{d}}_j$ sia rilevante per la query $\tilde{\mathbf{q}}$.
2. $P(\mathbf{R} = 0|\tilde{\mathbf{d}}_j, \tilde{\mathbf{q}}) = P(\bar{\mathbf{R}}|\tilde{\mathbf{d}}_j, \tilde{\mathbf{q}})$: rappresenta la probabilità che il documento $\tilde{\mathbf{d}}_j$ **NON** sia rilevante per la query $\tilde{\mathbf{q}}$.

In altre parole:

- $\mathbf{R} = 1$ o $\mathbf{R}$ significa che il documento è rilevante rispetto alla query.
 - $\mathbf{R} = 0$ o $\bar{\mathbf{R}}$ significa che il documento **non** è rilevante rispetto alla query.
3. $\text{sim}(\mathbf{d}_j, \mathbf{q})$: la funzione di **similarità** che calcola quanto un documento d_j sia simile alla query q . In questa slide è definita come il rapporto tra la probabilità che il documento sia rilevante e la probabilità che non sia rilevante.

Calcolo della Similarità $\text{sim}(\mathbf{d}_j, \mathbf{q})$

La formula di **similarità** è data da:

$$\text{sim}(d_j, q) = \frac{P(\mathbf{R}|\vec{d}_j, \vec{q})}{P(\bar{\mathbf{R}}|\vec{d}_j, \vec{q})}$$

Questa espressione confronta la probabilità che un documento sia rilevante rispetto a quella che non lo sia.

Uso della Regola di Bayes

Utilizzo la **regola di Bayes** per riformulare le probabilità condizionate:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

$$P(A|B) \cdot P(B) = P(B|A) \cdot P(A)$$

Usando la **regola di Bayes**, possiamo riscrivere $\text{sim}(\mathbf{d}_j, \mathbf{q})$ come:

$$\text{sim}(d_j, q) = \frac{P(\vec{d}_j|\mathbf{R}, \vec{q}) \cdot P(\mathbf{R}|\vec{q})}{P(\vec{d}_j|\bar{\mathbf{R}}, \vec{q}) \cdot P(\bar{\mathbf{R}}|\vec{q})}$$

In questo caso:

- $P(\vec{d}_j|\mathbf{R}, \vec{q})$: rappresenta la probabilità di osservare il documento $\vec{d}_j$ dato che è rilevante per la query $\vec{q}$.
- $P(\mathbf{R}|\vec{q})$: rappresenta la probabilità a priori che un documento sia rilevante per la query $\vec{q}$.
- $P(\vec{d}_j|\bar{\mathbf{R}}, \vec{q})$: rappresenta la probabilità di osservare il documento $\vec{d}_j$ dato che **non** è rilevante per la query $\vec{q}$.
- $P(\bar{\mathbf{R}}|\vec{q})$: rappresenta la probabilità a priori che un documento **non** sia rilevante per la query $\vec{q}$.

Interpretazione

La formula $\frac{P(\vec{d}_j|\mathbf{R}, \vec{q})}{P(\vec{d}_j|\bar{\mathbf{R}}, \vec{q})}$ appresenta il rapporto tra le probabilità di osservare il documento $\vec{d}_j$ tra i documenti rilevanti e quelli non rilevanti. Questo rapporto ci dice quanto è probabile che un documento sia rilevante rispetto a non esserlo.

In conclusione, questa formula permette al sistema di ranking di ordinare i documenti in base alla probabilità stimata di rilevanza rispetto alla query, così da restituire per primi i documenti con maggiore probabilità di essere rilevanti.

Passaggio 1: Assunzione di uguaglianza delle probabilità

Nella formula, si fa un'approssimazione assumendo che:

$$P(R|\vec{q}) = P(\bar{\mathbf{R}}|\vec{q})$$

Questo significa che **la probabilità a priori che un documento sia rilevante** ($P(R|\vec{q})$) è **uguale alla probabilità che non sia rilevante** ($P(\bar{\mathbf{R}}|\vec{q})$). Dato che queste probabilità non cambiano il ranking (ordinamento) dei documenti, possiamo ignorarle nel calcolo del ranking, semplificando così la formula.

Passaggio 2: Similarità come rapporto di probabilità

Dato quanto detto sopra, la funzione di similarità tra il documento d_j e la query q può essere approssimata come:

$$\text{sim}(d_j, q) \approx \frac{P(\vec{d}_j|R, \vec{q})}{P(\vec{d}_j|\bar{\mathbf{R}}, \vec{q})}$$

Questa è la probabilità di trovare il documento d_j tra quelli rilevanti rispetto a quella di trovarlo tra quelli non rilevanti.

Passaggio 3: Assunzione di indipendenza dei termini

Ora si assume che i termini k_i siano indipendenti tra loro, quindi possiamo calcolare la probabilità di un documento d_j come una **produttoria delle probabilità dei singoli termini**.

Passaggio 4: Formula finale con la produttoria

Rappresentiamo ogni documento d_j come un vettore binario di termini. Ogni termine ha valore $w_{ij} = 1$ se il termine k_i è presente nel documento, e $w_{ij} = 0$ se non è presente.

Dato che i termini sono indipendenti, possiamo riscrivere la similarità come:

$$\text{sim}(d_j, q) \approx \frac{\left(\prod_{w_{ij}=1} P(k_i|R, \vec{q}) \right) \cdot \left(\prod_{w_{ij}=0} P(\bar{k}_i|R, \vec{q}) \right)}{\left(\prod_{w_{ij}=1} P(k_i|\bar{\mathbf{R}}, \vec{q}) \right) \cdot \left(\prod_{w_{ij}=0} P(\bar{k}_i|\bar{\mathbf{R}}, \vec{q}) \right)}$$

Questa formula sfrutta la probabilità che ogni termine sia presente o assente in modo indipendente per calcolare la probabilità complessiva di rilevanza di d_j .

Interpretazione della Formula

- $P(k_i|R, \vec{q})$: probabilità che il termine k_i sia presente in un documento rilevante.
- $P(\bar{k}_i|R, \vec{q})$: probabilità che il termine k_i **non** sia presente in un documento rilevante.
- $P(k_i|\bar{\mathbf{R}}, \vec{q})$: probabilità che il termine k_i sia presente in un documento **non** rilevante.
- $P(\bar{k}_i|\bar{\mathbf{R}}, \vec{q})$: probabilità che il termine k_i **non** sia presente in un documento **non** rilevante.

In pratica, questa formula stima la rilevanza del documento d_j in base alla presenza o assenza dei termini della query in d_j e nei documenti rilevanti.

Passaggio 5: Semplificazione tramite Logaritmi

La formula iniziale della similarità tra il documento d_j e la query q è scritta come una frazione di produttorie. Ricordando che:

- $P(\bar{k}_i|R, \vec{q}) = 1 - P(k_i|R, \vec{q})$: cioè la probabilità che k_i **non** sia presente in un documento rilevante.
- $P(\bar{k}_i|\bar{\mathbf{R}}, \vec{q}) = 1 - P(k_i|\bar{\mathbf{R}}, \vec{q})$: cioè la probabilità che k_i **non** sia presente in un documento **non** rilevante.

- w_{ij} è un bit che indica se il termine k_i è presente ($w_{ij} = 1$) o assente ($w_{ij} = 0$) nel documento d_j .

La formula diventa:

$$\text{sim}(d_j, q) \approx \log \left(\prod_{i=1}^t \frac{P(k_i|R, \vec{q})^{w_{ij}} \cdot (1 - P(k_i|R, \vec{q}))^{1-w_{ij}}}{P(k_i|\bar{R}, \vec{q})^{w_{ij}} \cdot (1 - P(k_i|\bar{R}, \vec{q}))^{1-w_{ij}}} \right)$$

Utilizzando la proprietà del logaritmo $\log(A \cdot B) = \log(A) + \log(B)$, la formula diventa una somma di logaritmi, che consente di passare dalla produttoria alla sommatoria:

$$\text{sim}(d_j, q) \approx \sum_{i=1}^t \left(w_{ij} \log \frac{P(k_i|R, \vec{q})}{P(k_i|\bar{R}, \vec{q})} + (1 - w_{ij}) \log \frac{1 - P(k_i|R, \vec{q})}{1 - P(k_i|\bar{R}, \vec{q})} \right)$$

Abbiamo anche portato gli esponenti fuori dai logaritmi, moltiplicandoli per il logaritmo stesso.

Passaggio 6: Considerare i Termini Rilevanti per la Query

La slide mostra che, **ignorando le costanti** e concentrandosi solo sui **termini rilevanti per la query e i documenti**, la formula diventa:

$$\text{sim}(d_j, q) = \sum_{i=1}^t w_{iq} \times w_{ij} \times \log \frac{P(k_i|R, \vec{q})(1 - P(k_i|\bar{R}, \vec{q}))}{(1 - P(k_i|R, \vec{q}))P(k_i|\bar{R}, \vec{q})}$$

In questo caso:

- w_{iq} : indica se il termine k_i è rilevante per la query.
- w_{ij} : indica se il termine k_i è presente nel documento d_j .

L'uso dei pesi in forma di esponente consente di semplificare ulteriormente, annullando la prima parte della formula se $w_{ij} = 0$ e annullando la seconda parte se $w_{ij} = 1$, evitando quindi di dover gestire condizioni per la presenza o assenza del termine in ogni documento.

Passaggio 7: Punteggio di Similarità

Viene introdotto il concetto di "odds ratio" (Rapporto di Probabilità) e come questo sia utilizzato per calcolare il **punteggio di similarità** c_i di un termine k_i rispetto alla **rilevanza di un documento** d_j dato un insieme di query q .

$$c_i = \log \frac{P(k_i | R, \vec{q})(1 - P(k_i | \bar{R}, \vec{q}))}{(1 - P(k_i | R, \vec{q}))P(k_i | \bar{R}, \vec{q})}$$

Questa formula rappresenta il logaritmo del **rapporto tra le probabilità** di presenza del termine k_i nei documenti rilevanti rispetto ai non rilevanti.

Passaggio 8: Scomposizione del logaritmo

La formula si espande ulteriormente in due componenti, separando il numeratore e il denominatore:

$$c_i = \log \frac{P(k_i | R, \vec{q})}{1 - P(k_i | R, \vec{q})} + \log \frac{1 - P(k_i | \bar{R}, \vec{q})}{P(k_i | \bar{R}, \vec{q})}$$

Questa separazione permette di analizzare ciascuna parte individualmente, evidenziando il contributo del termine k_i per i documenti rilevanti e non rilevanti. Applicando la proprietà del logaritmo si nota meglio la differenza:

$$c_i = \log \frac{P(k_i | R, \vec{q})}{1 - P(k_i | R, \vec{q})} - \log \frac{P(k_i | \bar{R}, \vec{q})}{1 - P(k_i | \bar{R}, \vec{q})}$$

In questo caso:

- $P(k_i | R, \vec{q}) / 1 - P(k_i | R, \vec{q})$: rappresenta l'**odds** (o probabilità relativa) del termine k_i nei documenti rilevanti.
- $P(k_i | \bar{R}, \vec{q}) / 1 - P(k_i | \bar{R}, \vec{q})$: rappresenta invece l'**odds** del termine k_i nei documenti **non** rilevanti.

Interpretazione di C_i :

- c_i è **zero** se le probabilità relative di apparizione del termine k_i nei documenti rilevanti e non rilevanti sono uguali, suggerendo che il termine non fornisce informazioni utili per distinguere la rilevanza.
- c_i è **positivo** se è più probabile che k_i appaia nei documenti rilevanti, indicando che il termine ha una certa importanza per identificare la rilevanza.
- c_i è **negativo** se è meno probabile che k_i appaia nei documenti rilevanti, indicando che il termine ha poca importanza per identificare la rilevanza.
- c_i funziona come un **peso per il termine**, perché viene sommato nei calcoli complessivi di similarità per i termini presenti nella query.

In sintesi, c_i quantifica l'importanza di ogni termine della query rispetto alla rilevanza del documento, e la somma di questi c_i costituisce una misura di similarità che può essere usata per ordinare i documenti in base alla loro rilevanza rispetto alla query.

Conclusione del Risultato

Questo risultato è una funzione di ranking che ci permette di valutare la rilevanza di un termine rispetto alla query in un documento, in base alla probabilità che tale termine appaia in documenti rilevanti piuttosto che in documenti non rilevanti.

Ottenimento delle Probabilità Iniziali

Il problema iniziale: non conosciamo R

- R : Insieme dei documenti rilevanti per la query.
- Non sapendo inizialmente quali documenti sono rilevanti, dobbiamo fare un'ipotesi iniziale per calcolare $P(k_i | R, \vec{q})$ (probabilità che il termine k_i sia presente in un documento rilevante) e $P(k_i | \bar{R}, \vec{q})$ (probabilità che k_i sia presente in un documento **non** rilevante).

Cosa facciamo quando non ci sono documenti recuperati (inizialmente):

- $P(k_i, R, \vec{q})$: Si assume costante. Per semplicità, si assegna il valore 0.5 (ipotizziamo uguale probabilità di presenza o assenza di un termine in documenti rilevanti).
- $P(k_i | \bar{R}, \vec{q})$: Si calcola usando la distribuzione dei termini nella collezione:

$$P(k_i | \bar{R}, \vec{q}) = \frac{n_i}{N}$$

- n_i : Numero di documenti in cui appare k_i .
- N : Numero totale di documenti nella collezione.

Queste stime iniziali permettono di ordinare i documenti in base alla loro probabilità di rilevanza. Una volta recuperati documenti, le probabilità iniziali possono essere migliorate in base alle nuove informazioni.

Migliorando le Probabilità Iniziali

Come **migliorare le probabilità iniziali** nel Binary Independence Model (BIM) una volta che abbiamo recuperato un sottoinsieme di documenti V (esempio: i top r documenti ordinati in base alla probabilità di rilevanza iniziale):

Definizioni principali

- V : Sottoinsieme di documenti recuperati (esempio: top r documenti).
- V_i : Sottoinsieme di V che contiene il termine k_i .
- n_i : Numero di documenti nell'intera collezione N che contengono k_i .

Per documenti rilevanti (R):

$$P(k_i|R, \vec{q}) = \frac{V_i}{V} = \frac{\text{Documenti **Rilevanti** che **Contengono** } k_i}{\text{Documenti **Rilevanti**}}$$

La probabilità è data dal rapporto tra i documenti recuperati che contengono k_i (V_i) e tutti i documenti recuperati (V).

Per documenti non rilevanti (R):

$$P(k_i|\bar{R}, \vec{q}) = \frac{n_i - V_i}{N - V} = \frac{\text{Documenti **Non Rilevanti** che **Contengono** } k_i}{\text{Documenti **Non Rilevanti**}}$$

La probabilità è calcolata considerando i documenti che contengono k_i , ma non fanno parte di V , rispetto ai documenti **non** inclusi in V .

```
0 | DX |
  | DY | Documenti Rilevanti = V
  | DZ |
----- r (arbitrario)
  | DA |
  | DE | Documenti non Rilevanti = N - V
N | DO |
```

Coefficiente di discriminazione

c_i rappresenta un punteggio che misura quanto k_i contribuisce a discriminare i documenti rilevanti dai non rilevanti. La formula è:

$$c_i = \log \frac{\frac{V_i}{V-V_i}}{\frac{(n_i-V_i)}{(N-n_i)-(V-V_i)}} \quad c_i = \log \frac{\frac{\text{Documenti **Rilevanti** che **Contengono** } k_i \text{ (favorisce)}}{\frac{\text{Documenti **Rilevanti** che **Non Contengono** } k_i \text{ (penalizza)}}}{\frac{\text{Documenti **Non Rilevanti** che **Contengono** } k_i \text{ (penalizza)}}{\frac{\text{Documenti **Non Rilevanti** che **Non Contengono** } k_i \text{ (favorisce)}}}$$

- **Numeratore:** Rapporto tra i documenti rilevanti contenenti k_i e quelli rilevanti che non lo contengono.
- **Denominatore:** Rapporto tra i documenti non rilevanti contenenti k_i e quelli non rilevanti che non lo contengono.

Gestione dei valori zero (Smoothing)

Per evitare divisioni per zero o valori nulli (ad esempio, se k_i appare in tutti o in nessuno dei documenti), si applicano tecniche di smoothing.

Le formule aggiornate aggiungono piccoli valori costanti:

- Per $P(k_i|R)$:

$$P(k_i|R) = \frac{V_i + 0.5}{V + 1}$$

- Per $P(k_i|\bar{R})$:

$$P(k_i|\bar{R}) = \frac{n_i - V_i + 0.5}{N - V + 1}$$

Passaggi principali:

- Dopo aver recuperato un sottoinsieme V di documenti, possiamo aggiornare le probabilità iniziali per migliorare l'ordine dei documenti.
- Le nuove probabilità tengono conto della distribuzione dei termini nei documenti rilevanti (R) e non rilevanti ($\bar{R}$).
- Lo smoothing evita problemi con valori nulli o estremi.

Example

Q: "gold silver truck"

D1: "Shipment of gold damaged in a fire."

D2: "Delivery of silver arrived in a silver truck."

D3: "Shipment of gold arrived in a truck."

► Assume $V = \{D2, D3\}$

N : number of documents in the collection

V : number of relevant documents

n_i : number of documents that contain term t_i

V_i : number of relevant documents that contain term t_i

variable	Gold	Silver	Truck
N	3	3	3
V	2	2	2
n_i	2	1	2
V_i	1	1	2

Example (cont.)

► Computing Term Relevance Weight $P(k_i | R) = \frac{V_i + 0.5}{V + 1}$ $P(k_i | \bar{R}) = \frac{n_i - V_i + 0.5}{N - V + 1}$

$$c_i = \log\left(\frac{V_i + 0.5}{V - V_i + 0.5} \div \frac{n_i - V_i + 0.5}{(N - n_i) - (V - V_i) + 0.5}\right)$$

$$gold : \log\left(\frac{1 + 0.5}{2 - 1 + 0.5} \div \frac{2 - 1 + 0.5}{3 - 2 - 2 + 1 + 0.5}\right) = \log \frac{1}{3} = -0.477$$

$$silver : \log\left(\frac{1 + 0.5}{2 - 1 + 0.5} \div \frac{1 - 1 + 0.5}{3 - 1 - 2 + 1 + 0.5}\right) = \log \frac{1}{0.333} = 0.477$$

$$truck : \log\left(\frac{2 + 0.5}{2 - 2 + 0.5} \div \frac{2 - 2 + 0.5}{3 - 2 - 2 + 2 + 0.5}\right) = \log \frac{5}{0.333} = 1.176$$

► Similarity Coefficient for each document

D1: -0.477, D2: 0.477+1.176 = 1.653, D3: -0.477+1.176 = 0.699

Vantaggi e Svantaggi

Vantaggi:

- I documenti vengono ordinati in base alla probabilità decrescente di essere rilevanti rispetto alla query, facilitando il recupero dei risultati più pertinenti.

Svantaggi:

- **Incertezza iniziale:** E' necessario effettuare una stima iniziale per separare i documenti in due categorie (rilevanti e non rilevanti), che potrebbe essere inaccurata.
- **Pesi binari:** Il modello utilizza pesi binari per i termini di indicizzazione, ignorando informazioni più dettagliate.
- **Frequenza ignorata:** Non considera quante volte un termine di indicizzazione compare in un documento, trascurando un aspetto potenzialmente rilevante.
- **Indipendenza dei termini:** Presuppone che i termini di indicizzazione siano indipendenti tra loro, un'ipotesi che non è sempre realistica.

10.10 Introduzione alla Fuzzy Set Theory (Teoria degli Insiemi Fuzzy)

La **Fuzzy Set Theory** è un'estensione della logica booleana classica ed è stata introdotta da Lotfi Zadeh nel 1965. E' particolarmente utile per gestire situazioni in cui le informazioni sono incerte, vaghe o imprecise, come spesso accade nella vita reale.

Concetto Base

Nella logica booleana tradizionale, ogni elemento appartiene o non appartiene a un insieme: un'affermazione è o **vera** o falsa (logica True/False). Questo approccio rigido non è sempre adeguato per descrivere la realtà, dove i confini tra vero e falso possono essere sfumati.

La **Fuzzy Set Theory** permette invece di definire appartenenze "parziali". Un elemento può appartenere a un insieme con un certo **grado di appartenenza**, che è rappresentato da un valore compreso tra 0 e 1:

- **0:** L'elemento non appartiene all'insieme.
- **1:** L'elemento appartiene completamente all'insieme.
- **Valori intermedi:** L'elemento appartiene parzialmente.

Caratteristiche Principali

- **Estensione della logica booleana:**

E' un superset della logica booleana, che diventa un caso particolare (quando i gradi di appartenenza sono solo 0 o 1). Supera i problemi di rigidità del "vero/falso" assoluto.

- **Gestione dell'incertezza:**

E' particolarmente efficace nel trattare situazioni imprecise o incerte. Ad esempio, può descrivere concetti come "alto", "veloce" o "caldo", che non hanno confini chiari.

- **Descrizione linguistica:**

E' utile quando i problemi possono essere descritti con linguaggio naturale, usando parole anziché numeri precisi. Ad esempio, "una temperatura moderatamente alta".

- **Significato e precisione:**

La teoria consente un compromesso tra il significato dei dati e la loro precisione, adattandosi meglio a contesti complessi.

Esempio Pratico

Consideriamo l'insieme "persone alte":

- **Nella logica booleana:** una persona di altezza 1,80 m potrebbe essere considerata alta (1), mentre una persona di 1,79 m potrebbe essere considerata non alta (0).
- **Nella logica fuzzy:** una persona di 1,80 m potrebbe avere un grado di appartenenza pari a 0,9 all'insieme "persone alte", mentre una persona di 1,79 m potrebbe avere un grado di 0,8.

Funzioni Caratteristiche

1. Crisp Set Representation (Rappresentazione degli Insiemi Classici):

- Funzione caratteristica:

$$f_A(x) : X \rightarrow \{0, 1\}$$

Una funzione caratteristica assegna a ogni elemento x un valore binario:

$$f_A(x) = \begin{cases} 1, & \text{if } x \in A \\ 0, & \text{if } x \notin A \end{cases}$$

- 1 se l'elemento appartiene all'insieme A ,
- 0 se non appartiene.

- Definizione dell'insieme:

$$A = \{x | x \in X \cap f_A(x) = 1\}$$

L'insieme A include tutti gli x per cui $f_A(x)$ vale 1.

2. Fuzzy Set Representation (Rappresentazione degli Insiemi Fuzzy):

- **Funzione di appartenenza:**

$$\mu_A(x) : X \rightarrow [0, 1]$$

Qui, ogni elemento x ha un **grado di appartenenza** che può essere un **valore reale** tra 0 e 1:

$$\mu_A(x) = \begin{cases} 1, & \text{if } x \text{ is totally in } A \\ 0, & \text{if } x \text{ is not in } A \\ \text{value } x \text{ between } 0 \text{ and } 1, & \text{if } x \text{ is partially in } A \end{cases}$$

- 1: l'elemento appartiene completamente all'insieme A ,
- 0: l'elemento non appartiene affatto,
- *valori intermedi*: indicano un grado di appartenenza parziale.

- **Definizione dell'insieme fuzzy:**

$$A = \{x | x \in X \cup \mu_A(x) > 0\}$$

L'insieme fuzzy include tutti gli x che hanno un grado di appartenenza maggiore di 0.

Funzione di Appartenenza

La **funzione di appartenenza** $\mu_A(x)$ in un insieme fuzzy può essere una funzione sigmoide. Le **funzioni sigmoidi** sono una scelta comune per modellare la gradualità di appartenenza in molti contesti pratici, grazie alla loro **natura continua** e alla capacità di gestire **transizioni morbide tra appartenenza e non appartenenza**.

Una funzione sigmoide ha la forma generale:

$$\mu_A(x) = \frac{1}{1 + e^{-k(x-c)}}$$

Dove:

- x è il valore dell'elemento in esame.
- c è il valore al centro della transizione, ossia il punto in cui la funzione vale 0.5 (la "soglia").
- k controlla la pendenza della curva: un valore maggiore rende la transizione più ripida, mentre un valore minore la rende più graduale.

Vantaggi di usare una funzione sigmoide:

- **Morbidezza:** gestisce in modo naturale transizioni graduali, evitando soglie rigide.
- **Semplicità:** è facile da parametrizzare e calcolare.
- **Flessibilità:** il comportamento può essere regolato con i parametri k e c , adattandosi a diverse applicazioni.

Logica del Fuzzy Set e Semantica degli Operatori Fuzzy

1. Fuzzy Intersection, o Fuzzy AND (A AND B)

$$\mu_{A \cap B} = \min[\mu_A(x), \mu_B(X)]$$

2. Fuzzy Union, o Fuzzy OR (A OR B)

$$\mu_{A \cup B} = \max[\mu_A(x), \mu_B(X)]$$

3. Operatore NOT (NOT A)

$$\mu_{\neg A}(x) = 1 - \mu_A(x)$$

Operazione	Fuzzy
AND(x, y)	$\min(x, y)$
OR(x, y)	$\max(x, y)$
NOT(x)	$1 - x$

Recupero di informazioni fuzzy:

- La rappresentazione di documenti e query tramite insiemi di parole chiave produce descrizioni che sono solo parzialmente correlate ai contenuti semantici reali dei rispettivi documenti e query.
- Data una query basata su parole chiave, ad esempio "information retrieval", il modello dello spazio vettoriale associa a ciascun documento un valore di somiglianza:

$$\mu_q(d_j) = \text{sim}(q, d_j)$$

Fuzzy information retrieval

- ▶ Each *query term* A_i defines a *fuzzy set* D_{A_i}
- ▶ Each *document* d has a *degree of membership* in this set D : $\mu_{A_i}(d)$

$$\text{sim}(Q_{AND}, d) = \mu_{D_{A_1} \cap \dots \cap D_{A_n}}(d) = \min \{ \mu_{A_i}(d) \mid i = 1 \dots n \}$$

$$\text{sim}(Q_{OR}, d) = \mu_{D_{A_1} \cup \dots \cup D_{A_n}}(d) = \max \{ \mu_{A_i}(d) \mid i = 1 \dots n \}$$

Example For the *query term gold* $D_{gold} = \{(d_1, 0.8), (d_2, 0.5)\}$
For the *query term silver* $D_{silver} = \{(d_1, 0.5), (d_2, 0.4)\}$

$$D_{gold \wedge silver} = \{(d_1, 0.5), (d_2, 0.4)\}$$

$$D_{gold \vee silver} = \{(d_1, 0.8), (d_2, 0.5)\}$$

10.11 Il Modello BM25 (Okapi BM25)

Il **modello BM25** (acronimo di *Best Match 25*), noto anche come **Okapi BM25**, è un metodo di ranking sviluppato per migliorare il recupero delle informazioni in collezioni di testi di grandi dimensioni. E' un'evoluzione del *Binary Independence Model (BIM)*, originariamente progettato per cataloghi con record brevi e lunghezze omogenee. Sebbene il BIM fosse efficace in contesti specifici, non era ideale per le moderne raccolte di documenti a testo completo, che richiedono un'analisi più approfondita di caratteristiche come la frequenza dei termini e la lunghezza del documento.

Introdotta nel 1994, il BM25 è stato progettato per superare queste limitazioni, incorporando la **sensibilità alla frequenza dei termini (term frequency, TF)** e alla **lunghezza del documento**, due fattori critici per il recupero di informazioni. Questi miglioramenti hanno reso il BM25 uno dei modelli più diffusi e robusti per il ranking dei documenti, trovando applicazione in motori di ricerca, sistemi di interrogazione avanzati e altri strumenti di recupero delle informazioni.

La popolarità del BM25 risiede nella sua capacità di calcolare un punteggio di rilevanza bilanciato e flessibile, tenendo conto **sia dell'importanza di un termine in un documento sia della distribuzione globale del termine nella collezione**. A oggi, è ampiamente considerato uno standard di riferimento per i modelli di ranking e recupero delle informazioni.

RSV - Formula Semplificata

Il **RSV (Retrieval Status Value)** rappresenta il **punteggio di rilevanza** del documento d rispetto alla query q . Più è alto, più il documento è rilevante.

$$RSV_d = \sum_{t \in q \cap d} \log \frac{N}{df_t}$$

- $\sum_{t \in q \cap d}$: somma calcolata per ogni termine t presente sia nella query q sia nel documento d .
- N : numero totale di documenti nella collezione.
- df_t (*document frequency*): numero di documenti nella collezione che contengono il termine t .

- $\log \frac{N}{df_t}$ calcola l'**Inverse Document Frequency (IDF)**, che misura l'importanza di un termine nella collezione. Termini rari (con df_t basso) avranno un IDF più alto.

RSV - Formula Migliorata

$$RSV_d = \sum_{t \in q \cap d} \left[\log \frac{N}{df_t} \right] \cdot \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b \times (L_d/L_{average})) + tf_{td}}$$

Questa formula tiene conto sia della **frequenza dei termini** (tf_{td}) sia della **lunghezza del documento** (L_d).

1. $\log \frac{N}{df_t}$: **IDF** del termine t , come nella formula semplificata.
2. tf_{td} (*term frequency*): numero di volte che il termine t compare nel documento d . **Più è alto, più il termine è importante nel documento.**
3. $L_d/L_{average}$: normalizza la lunghezza del documento rispetto alla media
 - L_d : lunghezza del documento d (numero di termini).
 - $L_{average}$: lunghezza media dei documenti nella collezione.
4. k_1 : Parametro di tuning che controlla la scala dell'importanza della frequenza dei termini (tf_{td}).
 - **Valori tipici:** $k_1 \approx 1.2 - 2.0$.
5. b : Parametro di tuning che regola l'impatto della lunghezza del documento.
 - $b = 0$: nessuna normalizzazione per lunghezza.
 - $b = 1$: piena normalizzazione.
 - **Valori tipici:** $b \approx 0.75$.

Interpretazione

$$\frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b \times (L_d/L_{average})) + tf_{td}}$$

La frazione regola l'importanza del termine t in base alla sua frequenza e alla lunghezza del documento.

- **Numeratore:** moltiplica tf_{td} per $(k_1 + 1)$ per enfatizzarlo.
- **Denominatore:** normalizza tf_{td} in base alla lunghezza del documento e al parametro b .

Il punteggio **RSV** (RSV_d) combina l'**importanza globale del termine** (IDF) e la sua **rilevanza locale** (TF) nel documento, **normalizzando per lunghezza**.

Termini rari (IDF alto) e frequenti nel documento (TF alto) contribuiscono di più al punteggio. Documenti più lunghi hanno un peso ridotto per evitare bias (grazie alla normalizzazione con $L_d/L_{average}$).

Formula Estesa

$$RSV_d = \sum_{t \in q \cap d} \left[\log \frac{N}{df_t} \right] \cdot \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b \times (L_d/L_{average})) + tf_{td}} \cdot \frac{(k_3 + 1)tf_{tq}}{k_3 + tf_{tq}}$$

Novità rispetto alla versione precedente:

1. **Query Term Frequency (tf_{tq}):**

- Misura la frequenza del termine t nella query q .
- E' **importante per query lunghe**, in cui i termini possono comparire più volte.
- Il peso è regolato dalla frazione:

$$\frac{(k_3 + 1)tf_{tq}}{k_3 + tf_{tq}}$$

dove k_3 è un parametro di tuning che controlla l'impatto del tf_{tq} .

2. **Lunghezza delle Query:**

- La normalizzazione della lunghezza è assente, perché si considera una query fissa e il focus è sulla sua rilevanza rispetto ai documenti.

Questa variante del BM25 introduce il fattore tf_{tq} per gestire query lunghe, migliorando la precisione del ranking. I parametri k_1 , k_3 e b possono essere **ottimizzati su una collezione di test** per massimizzare le performance del modello.

11 Evaluation

11.1 User Happiness

Definizione di "User Happiness" in un sistema IR

La **User Happiness** in un sistema di recupero dell'informazione (IR) rappresenta il livello di soddisfazione dell'utente rispetto ai risultati ottenuti dalle ricerche. E' influenzata dalla **velocità di risposta**, dalla **rilevanza dei risultati** e dall'**efficacia del sistema** nel permettere all'utente di raggiungere il proprio obiettivo, sia esso trovare informazioni, completare un acquisto o aumentare la produttività in un contesto lavorativo.

Prerequisiti alla misurazione della soddisfazione dell'utente

La misura più comune è la rilevanza dei risultati di ricerca. Per misurare la rilevanza, si utilizzano tre elementi fondamentali:

- Una **collezione di documenti** di riferimento (*benchmark*).
- Un **insieme di query standardizzate** per il test.
- Una **valutazione binaria** (*rilevante/non rilevante*) per ogni query e ogni documento.

Valutazione di un Sistema IR

L'utente esprime un **bisogno informativo**, che viene tradotto in una **query**. La **rilevanza** di un documento viene valutata rispetto al **bisogno informativo**, non semplicemente alla query formulata.

Esempio:

- **Bisogno informativo:** sapere se il vino rosso è più efficace del vino bianco nel ridurre il rischio di infarto.
- **Query digitata:** wine red white heart attack effective.
- La **valutazione** della rilevanza deve basarsi su quanto il documento risponde alla domanda originale, non solo sulla presenza delle parole chiave.

11.2 Precision e Recall

Nei sistemi di **Information Retrieval (IR)**, l'efficacia del recupero delle informazioni viene valutata utilizzando due metriche fondamentali: **Precision** e **Recall**. Queste misure permettono di capire quanto bene un sistema IR riesca a restituire documenti rilevanti rispetto a una richiesta informativa dell'utente.

Definizioni di Precision e Recall

Recall

La **Recall** (o *richiamo*) misura la capacità del sistema di recuperare tutti i documenti rilevanti per una determinata query. Essa viene calcolata come la frazione dei documenti rilevanti effettivamente recuperati rispetto all'insieme totale di documenti rilevanti. La formula per la **Recall** è:

$$Recall = \frac{|Ra|}{|R|}$$

Dove:

- R è l'insieme dei documenti rilevanti per la query.
- A è l'insieme dei documenti effettivamente recuperati dal sistema IR.
- Ra è il numero di documenti che sono sia rilevanti che recuperati (intersezione tra R e A).

In altre parole, **la Recall misura la completezza del sistema**, ovvero quanto è efficace nel trovare tutti i documenti rilevanti disponibili.

Precision

La **Precision** (o *precisione*) indica invece la qualità dei risultati recuperati, valutando la frazione di documenti rilevanti tra quelli restituiti dal sistema. La formula per la **Precision** è:

$$Precision = \frac{|Ra|}{|A|}$$

Dove:

- A è il numero totale di documenti recuperati.
- Ra è il numero di documenti rilevanti tra quelli recuperati.

In altre parole, **la Precision misura l'accuratezza del sistema**, ovvero quanti documenti restituiti sono effettivamente utili per l'utente.

Analisi della relazione tra Precision e Recall

La Precision e la Recall sono **inversamente proporzionali**:

- Un sistema che restituisce **molte documenti aumenta la Recall** (probabilità di trovare i documenti rilevanti), ma **diminuisce la Precision** (perché includerà anche molti documenti non rilevanti).
- Un sistema che restituisce **pochi documenti altamente rilevanti** avrà una **Precision alta**, ma una **Recall bassa**, perché potrebbe tralasciare molti documenti rilevanti.

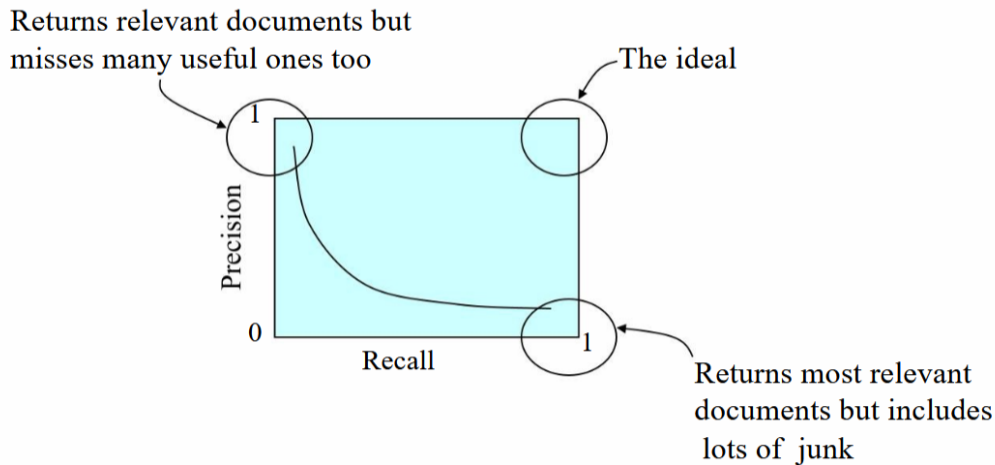
Questo trade-off è inevitabile:

- **Alta Recall, bassa Precision** → Il sistema restituisce troppi documenti, inclusi molti irrilevanti.
- **Alta Precision, bassa Recall** → Il sistema restituisce pochi documenti, rischiando di escludere informazioni importanti.

Precision-Recall Curve e valutazione di sistemi IR

Per analizzare il comportamento di un sistema IR, si utilizza la **curva Precision-Recall**, che mostra come varia la Precision al variare della Recall.

- La *Recall* è una **funzione non decrescente**: aumenta con l'aumento del numero di documenti recuperati.
- La *Precision* **tende a decrescere con l'aumento della Recall**, poiché aumentando il numero di documenti restituiti si rischia di includere elementi irrilevanti.
- Un buon sistema IR mantiene una *Precision elevata anche per alti livelli di Recall*.

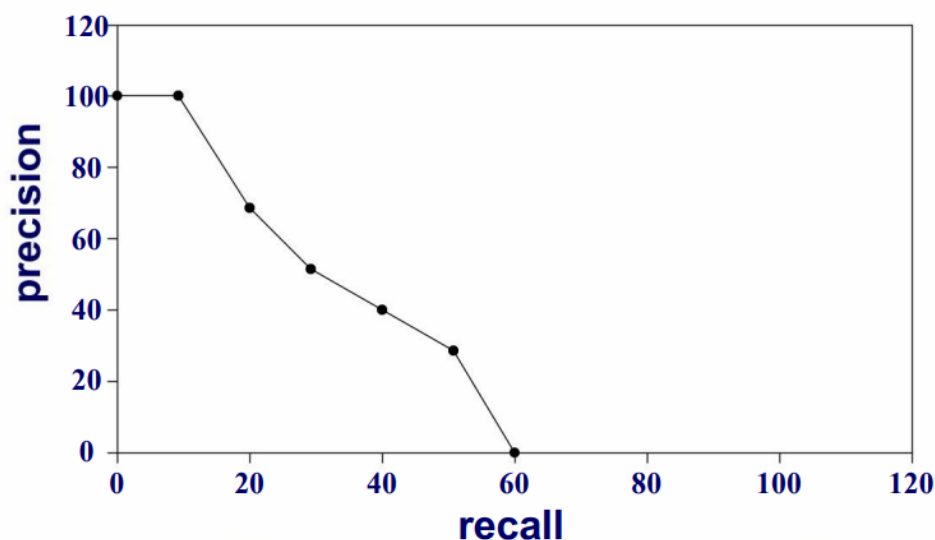


L'immagine mostra una **Precision-Recall Curve** standard, dove si nota il *tipico andamento decrescente della Precision all'aumentare della Recall*.

11.3 Livelli Standard di Recall

Per valutare le prestazioni di un sistema di Information Retrieval (IR), si utilizza la Precision-Recall Curve, che mostra come la precisione varia al crescere del recall. Tuttavia, per comparare diversi sistemi, è necessario un metodo standardizzato: si utilizzano **11 livelli standard di recall**.

- Per misurare la precisione a diversi livelli di recall, si adottano **11 punti standard: 0%, 10%, 20%, 30%, ..., 90%, 100%**.
- L'idea è semplice: invece di considerare ogni punto della curva, ci si concentra solo su questi livelli prestabiliti.
- Ad esempio, se un sistema ha recuperato il **30% dei documenti rilevanti**, si misura la precisione in quel punto. Questo permette di confrontare facilmente più sistemi sulla stessa scala.



Example

- 10 relevant docs: $\{d_3, d_5, d_9, d_{25}, d_{39}, d_{44}, d_{56}, d_{71}, d_{389}, d_{123}\}$
- Ranking for query q : $d_{123}, d_{84}, d_{56}, d_{6}, d_8, d_9, d_{511}, d_{129}, d_{187}, d_{25}, d_{36}, d_{48}, d_{250}, d_{113}, d_2$

Level	P()	Level	P()	Level	P()
0	??%	0.4	40%	0.8	0%
0.1	100%	0.5	33%	0.9	0%
0.2	66%	0.6	0%	1	0%
0.3	50%	0.7	0%		

Confronto tra livelli standard di recall

Un problema nell'uso diretto dei livelli standard di recall è che diversi sistemi di IR potrebbero recuperare i documenti in un ordine differente, generando livelli di recall che non corrispondono esattamente ai valori standard.

Per risolvere questo problema si utilizza una tecnica detta **normalizzazione dei livelli di recall**.

Esempio:

- Se per una query q il sistema restituisce risultati con recall a 0.33, 0.66 e 1.0, mentre un'altra query q' produce recall a 0.5, 0.75 e 1.0, non possiamo confrontarli direttamente.
- Per rendere il confronto possibile, interpoliamo i valori per ottenere precisioni a livelli standard di recall.

11.4 Precision Interpolata (a Soffitto)

Poiché i livelli standard di recall potrebbero non essere coperti direttamente dai risultati di un sistema, si utilizza la **precisione interpolata** per garantire che ogni sistema sia valutato agli stessi livelli.

$$P(r_j) = \max_{r_j \leq r \leq r_{j+1}} P(r)$$

Questa formula indica che la precisione interpolata a un dato livello di recall r_j è il massimo valore di precisione tra r_j e il livello successivo r_{j+1} . In pratica, se in un punto specifico la precisione scende, si considera il valore massimo precedente.

Esempio con livelli di recall non standard:

Recall	Precision	Recall	Interpolated Precision
0.33	0.33	1.0	0
0.66	0.4	0.9	0
1.0	0	0.8	0
		0.7	0
		0.6	0.4
		0.5	0.4
		0.4	0.4
		0.3	0.4
		0.2	0.4
		0.1	0.4
		0.0	0.4

11.5 Average Precision (Level) e Confronto tra Sistemi

Un singolo grafico precision-recall per una query non è particolarmente utile per valutare le prestazioni di un sistema. Per una valutazione più completa, si calcolano le **precisioni medie** su un insieme di query.

La **Precisione Media** ($AP(r)$) al livello r di recall è ottenuta calcolando la precisione al livello di recall r per ciascuna query e facendo poi la media di queste precisioni su tutte le query:

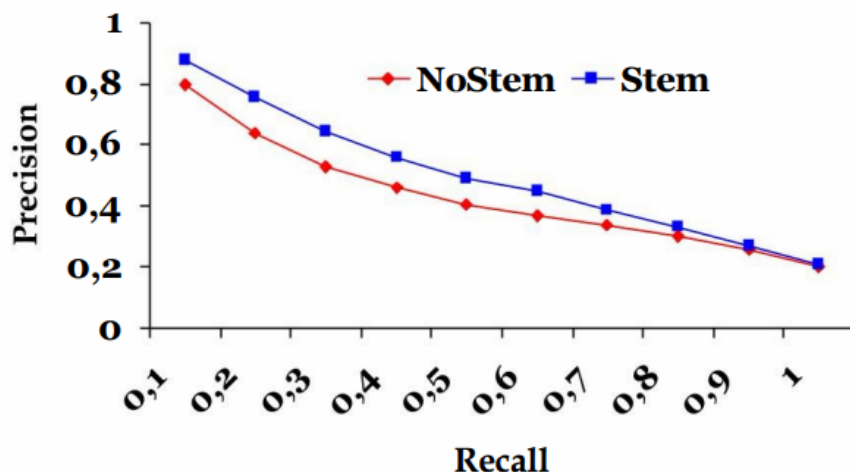
$$AP(r) = \frac{1}{N_q} \cdot \sum_{i=1}^{N_q} P_i(r)$$

Dove:

- N_q è il numero totale di query.
- $P_i(r)$ è la **precisione interpolata** al livello r rispetto alla query q_i .
- $AP(r)$ è la **precisione media** al livello r rispetto a tutte le query.

Confronto tra Sistemi

Calcolando la precisione media per ciascun livello di recall r , otteniamo un **insieme delle precisioni medie** $\{AP(r) \mid r \in [0.0, 1.0], r \text{ con passo } 0.1\}$. Questi valori possono essere utilizzati per realizzare **una curva** che è utile per valutare le prestazioni complessive di un sistema e per confrontarle con le prestazioni di altri sistemi:



Per confrontare sistemi di IR, si **analizzano** le loro **precision-recall curve**:

- Il sistema con la curva più vicina all'angolo in alto a destra del grafico ha prestazioni migliori, poiché indica alta precisione e alto recall.
- Il sistema **Stem** (blu) ha una precisione più alta rispetto a **NoStem** (rosso) per quasi tutti i livelli di recall. Questo indica che l'uso dello stemming migliora la precisione del recupero delle informazioni.

11.6 Single Value Average Precision

A volte è utile avere un **singolo valore di precisione per ogni query**, piuttosto che una media su tutte le query, per *evitare che anomalie nei risultati vengano mascherate dalla media*. Quando si confrontano due algoritmi, può essere importante analizzare se uno supera l'altro per ciascuna query, piuttosto che basarsi su un confronto globale.

11.7 Average Precision (Query) e Confronto tra Sistemi

La Precisione Media ($AP(q)$) è una misura utilizzata per valutare le prestazioni di un sistema di recupero delle informazioni in relazione a una singola query q , basandosi sulla precisione ottenuta a vari livelli di recall.

- **Precisione Media Interpolata (Query)**: calcolata considerando i **valori interpolati di precisione** per ciascun livello di recall, garantendo che la precisione non diminuisca mentre il recall aumenta.

$$AP(q) = \frac{1}{n} \cdot \sum_{r=0}^n P_q(r)$$

- n è il numero **totale di livelli** di recall.
- $P_q(r)$ è la **precisione interpolata** al livello r per la query q .

- **Precisione Media Non-Interpolata (Query)**: Questa misura favorisce sistemi che recuperano documenti rilevanti (R_q) per la query q velocemente (es. all'inizio del ranking). $AP(q) = \sum_{r=0}^n \frac{P_q(r/|R_q|)}{|R_q|}$

Confronto tra Sistemi

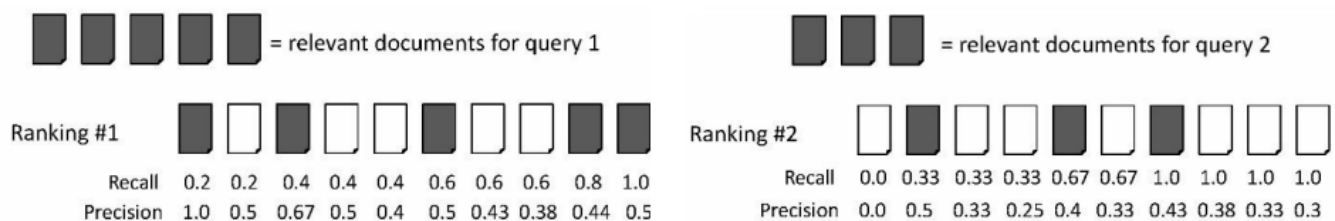
Calcolando la precisione media per ogni query q , otteniamo un **insieme di valori di precisione media** $\{AP(q) \mid q \in Q\}$. Questi valori possono essere utilizzati per confrontare le prestazioni di diversi sistemi, eseguendo un *confronto diretto tra i singoli risultati di precisione media per ciascuna query*.

Per ottenere una valutazione complessiva delle prestazioni di un sistema su tutte le query, si può calcolare la **Mean Average Precision (MAP)**. La MAP è ottenuta normalizzando la somma delle precisioni medie per il numero totale di query.

$$MAP = \sum_{q \in Q} \frac{AP(q)}{|Q|}$$

Questo fornisce un **singolo valore** che rappresenta la **performance globale del sistema** su un insieme di query.

Esempio



$$\text{average precision query 1} = (1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$$

$$\text{average precision query 2} = (0.5 + 0.4 + 0.43)/3 = 0.44$$

$$\text{mean average precision} = (0.62 + 0.44)/2 = 0.53$$

Ultime considerazioni su MAP

- Se un documento rilevante non viene mai recuperato, la sua precisione è considerata pari a zero.
- MAP utilizza la macro-media, dando lo stesso peso a ogni query indipendentemente dal numero di documenti rilevanti.
- E' una delle metriche più utilizzate nella ricerca accademica per valutare i sistemi di Information Retrieval.
- E' adatta per la ricerca sul web? Dipende: MAP presuppone che l'utente voglia trovare molti documenti rilevanti per ogni query.
- La sua applicazione richiede una grande quantità di giudizi di rilevanza all'interno della collezione testuale.

11.8 R-Precision (a.k.a. Precision@R)

La **R-Precision** misura la precisione del sistema nel recuperare esattamente R documenti rilevanti, dove R è il numero totale di documenti rilevanti per una specifica query. In pratica, si calcola considerando i primi R documenti restituiti dal sistema e determinando quanti di essi sono effettivamente rilevanti.

$$R\text{-Precision} = \frac{\# \text{ documenti rilevanti nei primi } R \text{ risultati}}{R}$$

Questa metrica è particolarmente utile per confrontare le prestazioni di diversi algoritmi di recupero informazioni, utilizzando la differenza tra le loro R-Precision per una stessa query:

$$RP_{A/B}(i) = RP_A(i) - RP_B(i)$$

$RP_A(i)$ e $RP_B(i)$ rappresentano la R-Precision dei due algoritmi per q_i .

Esempio

n	doc #	relevant
1	588	x
2	589	x
3	576	
4	590	x
5	986	
6	592	x
7	984	
8	988	
9	578	
10	985	
11	103	
12	591	
13	772	x
14	990	

Assume $R = \#$ of relevant docs = 6

6-Precision

or

Precision@6 = $4/6 = 0.67$

11.9 Problemi di precisione e recall

1. La corretta stima del recall massimo per una query richiede una conoscenza dettagliata di tutti i documenti nella collezione. Con collezioni di grandi dimensioni, tale conoscenza non è disponibile.
2. Precisione e recall sono misure correlate che catturano diversi aspetti dell'insieme dei documenti recuperati. In molte situazioni, l'uso di una singola misura che combini precisione e recall potrebbe essere più appropriato.
3. Precisione e recall misurano l'efficacia su un insieme di query elaborate in modalità batch. Con i sistemi moderni, l'interattività (e non l'elaborazione batch) è l'aspetto chiave del processo di recupero.
4. Precisione e recall sono facili da definire quando viene imposta un'ordinamento lineare dei documenti recuperati. Per i sistemi che richiedono un ordinamento debole, però, precisione e recall potrebbero risultare inadeguati.

11.10 Harmonic Mean e E-Measure

Le metriche **F-Measure** e **E-Measure** combinano precision e recall in un unico valore per valutare le prestazioni di un sistema di recupero dell'informazione ([Soluzione al 2° problema](#)).

F-Measure (Harmonic Mean)

L'**F-Measure** è la media armonica tra precision e recall, espressa dalla formula:

$$F = \frac{2}{\frac{1}{prec.} + \frac{1}{recall}} = \frac{2 \cdot prec. \cdot recall}{prec. + recall}$$

Questa metrica è utile perché assume un valore elevato solo quando sia precision che recall sono alte, penalizzando gli squilibri tra le due.

E-Measure

L'**E-Measure** è una generalizzazione dell'**F-Measure** che permette di assegnare maggiore importanza alla precisione o al recall a seconda delle esigenze dell'utente. La formula è:

$$E = 1 - \frac{1 + b^2}{\frac{b^2}{prec.} + \frac{1}{recall}}$$

Dove b è un parametro che regola l'importanza relativa tra le due misure:

- $b = 1 \rightarrow$ Equivalente all'**F-Measure** (media armonica).
- $b > 1 \rightarrow$ Enfatisza la precisione.
- $b < 1 \rightarrow$ Enfatisza il recall.

L'**F-Measure** è particolarmente utile in situazioni in cui precision e recall devono essere bilanciate, mentre l'**E-Measure** consente una maggiore flessibilità adattandosi alle esigenze specifiche dell'utente.

11.11 Discounted Cumulative Gain (DCG)

La **Discounted Cumulative Gain (DCG)** è una metrica molto utilizzata nella valutazione dei motori di ricerca e dei sistemi di recupero dell'informazione. Il suo obiettivo è misurare la qualità dell'ordine dei risultati restituiti rispetto alla loro rilevanza per l'utente.

Principi di base

DCG si basa su due assunzioni fondamentali:

- **I documenti più rilevanti sono più utili** rispetto a quelli con rilevanza marginale.
- **I documenti posizionati più in basso nella classifica sono meno utili**, poiché è meno probabile che vengano esaminati dall'utente.

Per quantificare l'utilità di un documento, viene utilizzata una scala di **rilevanza graduata**, dove ad esempio:

- 0 $\rightarrow$ Documento non utile.
- 1 $\rightarrow$ Marginalmente utile.
- 2 $\rightarrow$ Utile.
- 3 $\rightarrow$ Molto utile.

Formula della DCG

Il **guadagno (gain)** di un documento viene accumulato dall'alto verso il basso nella lista dei risultati, ma i documenti nelle posizioni inferiori ricevono un peso ridotto tramite uno sconto logaritmico. La formula generale della **DCG** fino alla posizione p è:

$$DCG_p = rel_1 + \sum_{j=2}^p \frac{rel_j}{\log_2(j)} \quad \text{alternativamente} \quad DCG_p = \sum_{i=1}^p \frac{2^{rel_i} - 1}{\log(1 + i)}$$

Dove:

- rel_i è la rilevanza del documento alla posizione i .
- $\log_2(i)$ è il fattore di sconto per la posizione i .

Esempio 1 di calcolo della DCG

- Alla posizione 1, il documento mantiene il suo valore di rilevanza.
- Alla posizione 2, la rilevanza è divisa per $\log_2(2) = 1$ (quindi non cambia).
- Alla posizione 3, la rilevanza è divisa per $\log_2(3) \approx 1.59$, riducendo il guadagno.
- Alla posizione 4, la rilevanza è divisa per $\log_2(4) = 2$, e così via.

Questa tecnica riflette il comportamento dell'utente: i documenti in cima alla classifica sono più importanti rispetto a quelli posizionati più in basso.

Esempio 2 di calcolo della DCG

Supponiamo di avere **10 documenti classificati** con i seguenti livelli di rilevanza: $[3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$. Il guadagno scontato (discounted gain) per documento è calcolato così:

Posizione i	Rilevanza rel_i	Fattore di sconto $\log_2(i)$	Discounted Gain $\frac{rel_i}{\log_2(i)}$
1	3	1.00	3.00
2	2	1.00	2.00
3	3	1.59	1.89
4	0	2.00	0.00
5	0	2.32	0.00
6	1	2.59	0.39
7	2	2.81	0.71
8	2	3.00	0.67
9	3	3.17	0.95
10	0	3.32	0.00

La **DCG cumulativa** viene calcolata sommando progressivamente i valori della colonna Discounted Gain. Questa sequenza mostra l'accumulo del guadagno fino a una certa posizione nella classifica:

$$[3, 5, 6.89, 6.89, 6.89, 7.28, 7.99, 8.66, 9.61, 9.61]$$

11.12 Normalized Discounted Cumulative Gain (NDCG)

Poiché il valore della **DCG** dipende dal numero e dalla distribuzione dei documenti rilevanti, per confrontare diverse query si utilizza una versione **normalizzata**, chiamata **NDCG**.

La **NDCG** si ottiene dividendo la DCG di una classifica per la **DCG ideale (IDCG)**, che è la *DCG calcolata su una classifica perfettamente ordinata* (con i documenti più rilevanti in cima).

$$NDCG_p = \frac{DCG_p}{IDCG_p}$$

Questa normalizzazione permette di confrontare query con numeri diversi di documenti rilevanti, rendendo la misura più affidabile.

4 documents: d_1, d_2, d_3, d_4

i	Ground Truth		Ranking Function ₁		Ranking Function ₂	
	Document Order	r_i	Document Order	r_i	Document Order	r_i
1	d4	2	d3	2	d3	2
2	d3	2	d4	2	d2	1
3	d2	1	d2	1	d4	2
4	d1	0	d1	0	d1	0
NDCG _{GT} =1.00			NDCG _{RF1} =1.00		NDCG _{RF2} =0.9203	

$$DCG_{GT} = 2 + \left(\frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.6309$$

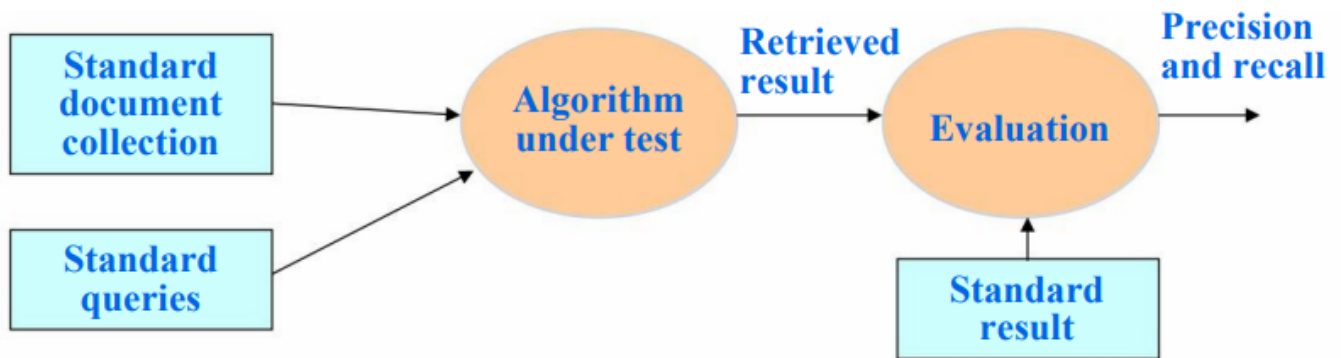
$$DCG_{RF1} = 2 + \left(\frac{2}{\log_2 2} + \frac{1}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.6309$$

$$DCG_{RF2} = 2 + \left(\frac{1}{\log_2 2} + \frac{2}{\log_2 3} + \frac{0}{\log_2 4} \right) = 4.2619$$

$$MaxDCG = DCG_{GT} = 4.6309$$

11.13 Benchmarks

I **benchmark** nell'Information Retrieval (IR) sono raccolte standardizzate di documenti, query e giudizi di rilevanza utilizzate per valutare le prestazioni dei sistemi di recupero delle informazioni. Queste collezioni permettono di confrontare diversi algoritmi su dati condivisi, garantendo una valutazione oggettiva e riproducibile.



TREC

Uno dei benchmark più importanti è il **TREC (Text Retrieval Conference)**, organizzato dal **National Institute of Standards and Technology (NIST)** degli Stati Uniti. TREC è un insieme di benchmark di rilevanza che include milioni di documenti, principalmente articoli di giornale, e centinaia di query informative. Tuttavia, non fornisce giudizi di rilevanza esaustivi per tutti i documenti, poiché sarebbe troppo costoso. Invece, i valutatori del NIST giudicano solo i documenti restituiti dai sistemi partecipanti all'esperimento.

BEIR

Un altro benchmark significativo è **BEIR (Benchmarking IR)**, una raccolta eterogenea che copre una vasta gamma di task di recupero delle informazioni. BEIR fornisce anche un framework standardizzato per la valutazione di modelli NLP-based, semplificando il confronto tra approcci diversi nel recupero di informazioni testuali.

12 Tolerant Retrieval

Tolerant retrieval si riferisce a tecniche utilizzate per gestire le discrepanze tra i termini di ricerca e i termini presenti nei documenti. Quando non c'è una corrispondenza esatta, le seguenti tecniche possono essere utilizzate:

- **Wildcard queries:** Permettono di cercare parole che corrispondono a schemi con caratteri jolly, come "*" o "?", per gestire variazioni di termini.
- **Spelling correction:** Corregge automaticamente errori di ortografia nel termine di ricerca per migliorare i risultati della ricerca.
- **Soundex:** Una tecnica fonetica che converte le parole in codici che rappresentano il loro suono, permettendo di trovare parole simili foneticamente.

12.1 Wildcard Queries

Le **Wildcard Queries** sono interrogazioni che utilizzano caratteri speciali (come `*`) per rappresentare uno o più caratteri in un termine di ricerca. L'implementazione efficiente di queste ricerche può essere complessa e richiede strategie ottimizzate per evitare operazioni troppo costose in termini di calcolo.

Tipologie di Wildcard Queries

1. **Prefix Queries (Wildcard alla fine del termine)** Una query di tipo **prefix** utilizza `*` alla fine della parola, consentendo di trovare tutti i termini che iniziano con una determinata sequenza di caratteri.
 - **Esempio:** `mon*` recupera parole come *monday*, *monitor*.
 - Questa operazione è facilmente implementabile con strutture dati come **B-tree** o **dizionari ordinati**, cercando tutti i termini compresi nell'intervallo $mon \leq t < moo$.
2. **Postfix Queries (Wildcard all'inizio del termine)** Una query di tipo **postfix** utilizza `*` all'inizio della parola, trovando tutti i termini che terminano con una certa sequenza di caratteri.
 - **Esempio:** `*mon` recupera parole come *lemon*, *summon*, *common*.
 - Una soluzione efficiente consiste nel mantenere **un indice invertito aggiuntivo per i termini scritti al contrario** (es. *gammon* $\rightarrow$ *nommag*). In questo modo, la ricerca `*mon` diventa equivalente a cercare `nom*` nell'indice invertito.
3. **Midfix Queries (Wildcard in mezzo al termine)** Quando `*` è presente all'interno della parola, la ricerca diventa più complessa perché richiede di trovare termini che abbiano una parte iniziale e una parte finale specifica.
 - **Esempio:** `m*nchen` potrebbe restituire *München* e *Manschen*.
 - Una possibile strategia è **dividere la ricerca in due wildcard separate** (`m*` e `*nchen`) e poi intersecare i due insiemi di termini trovati.
 - Tuttavia, questa operazione è **molto costosa** in termini computazionali; richiede la gestione di una grande quantità di dati.

12.1.1 Permutation Index

Un **Permutation Index** è una struttura dati progettata per gestire in modo efficiente le **wildcard queries**, in particolare quelle che contengono il carattere `*` all'interno della parola (**midfix queries**).

L'idea di base è trasformare una query con wildcard in una **prefix query**, rendendo possibile il recupero delle informazioni tramite strutture di ricerca ottimizzate, come i B-tree.

Uno dei metodi più utilizzati per implementare un Permutation Index è il **Permuterm Index**, che memorizza tutte le rotazioni di una parola in un dizionario, aggiungendo un carattere speciale `$` alla fine del termine.

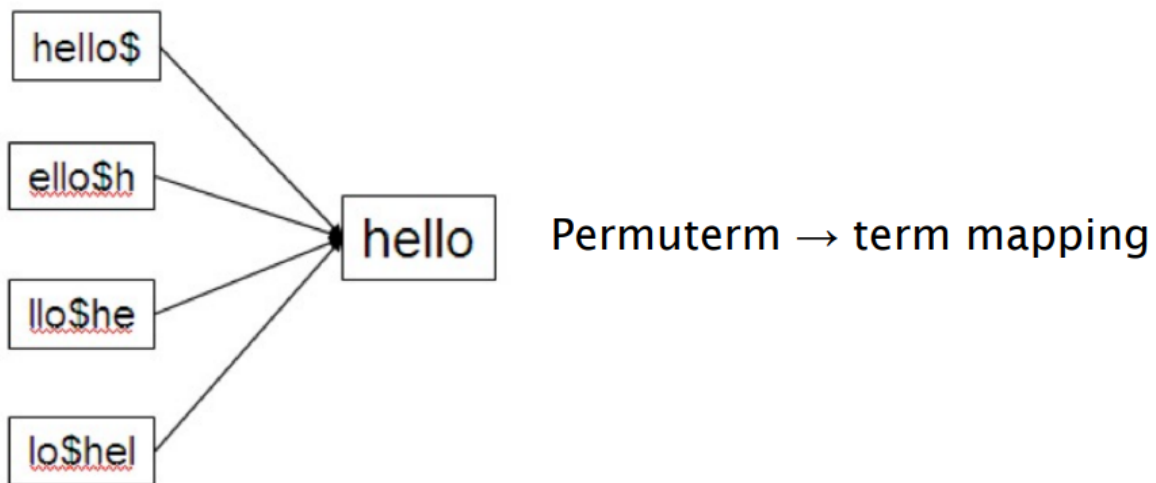
Come funziona il Permuterm Index

1. **Creazione dell'indice:** Per ogni parola nel lessico, vengono generate tutte le sue rotazioni e archiviate in un **B-tree**.

Ad esempio, per il termine **HELLO**, il permuterm index memorizza:

- hello\$
- ello\$h
- llo\$he
- lo\$hel
- o\$hell

Ciascun permuterm punta alla parola originale nell'indice invertito.



2. **Elaborazione delle wildcard queries:** Le query con wildcard vengono trasformate per essere risolte come prefix queries, seguendo queste regole:

- $X \rightarrow$ Ricerca $X\$$ (es. hello $\rightarrow$ hello\$)
- $X^* \rightarrow$ Ricerca $\$X^*$ (es. hel* $\rightarrow$ \$hel*)
- $*X \rightarrow$ Ricerca $X\* (es. *lo $\rightarrow$ lo\$*)
- $*X^* \rightarrow$ Ricerca X^* (es. *ell* $\rightarrow$ ell*)
- $X^*Y \rightarrow$ Ricerca $Y\$X^*$ (es. hel*o $\rightarrow$ o\$hel*)

Una volta riformulata la query, si effettua una ricerca nel B-tree per individuare i permuterm corrispondenti e quindi si recuperano i documenti nell'indice invertito.

Vantaggi e Svantaggi del Permuterm Index

• Vantaggi del Permuterm Index

- Supporto efficiente per **midfix queries** ($*X^*$) e **postfix queries**.
- Utilizzo di **B-tree**, che permette **ricerche rapide**.
- Elimina la necessità di scansioni complete del lessico, a differenza di altre tecniche.

• Svantaggi del Permuterm Index

- **Aumento delle dimensioni del lessico:** Per ogni parola vengono memorizzate molte rotazioni, il che quadrupla (o più) il numero di voci nel dizionario.
- **Costi di memoria elevati**, specialmente con parole lunghe.
- **Maggiore complessità nell'aggiornamento:** ogni nuova parola richiede la generazione di tutte le sue permutazioni.

12.1.2 Q-grammi

I **q-grammi** sono sequenze di q caratteri estratte da una stringa, utilizzate per la ricerca approssimata di termini in database testuali e motori di ricerca. Servono come alternativa ai Permuterm Index, offrendo un compromesso tra efficienza di ricerca e spazio occupato.

Il numero di caratteri che compongono un q-gramma è determinato dal valore prefissato di q (ad esempio, $q = 2$ per **bigrammi** e $q = 3$ per **trigrammi**). Per ottimizzare l'efficacia nella ricerca, vengono aggiunti **simboli speciali**: $\#$ per indicare l'inizio di una parola e $\$$ per segnalarne la fine.

Esempi

- Consideriamo la parola "**vacations**" con $q = 3$ (trigrammi):
 $\{\# \# v, \# va, vac, aca, cat, ati, tio, ion, ons, ns \$, s \$ \$\}$
- Per la frase "**April is the cruelest month**" con $q = 2$ (bigrammi):
 $\{\# a, ap, pr, ri, il, l \$, \# i, is, s \$, \# t, th, he, e \$, \# c, cr, ru, ue, el, le, es, st, t \$, \# m, mo, on, nt, th, h \$\}$
- Il numero di q-grammi risultanti è ricavato tramite la seguente formula:

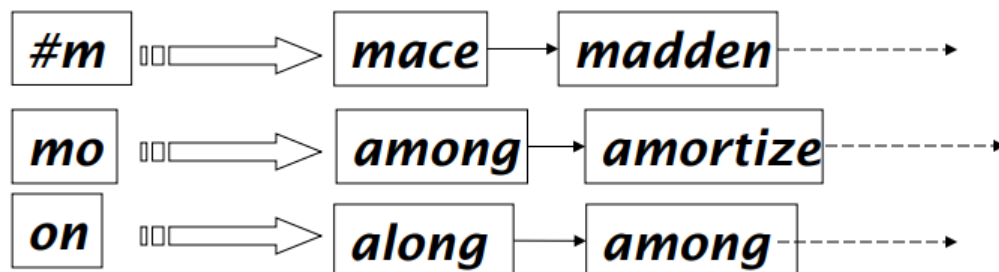
$$\text{Numero di q-grammi} = L + q - 1$$

dove L rappresenta la lunghezza della stringa originale e q è il valore prefissato per il q-gramma.

Come funzionano gli indici basati su Q-grammi

1. Costruzione dell'indice

- Si enumerano tutti i q-grammi presenti nei termini del lessico.
- Si mantiene un indice invertito secondario, che associa ogni q-gramma ai termini che lo contengono.



2. Ricerca tramite wildcard queries

- Per elaborare una wildcard query, la parola viene suddivisa in q-grammi;
 - Query: "mon*"
 - Q-grammi: $\{\# m, mo, on\}$, con $q = 2$
- Si esegue un **AND** tra questi q-grammi per recuperare tutte le parole che contengono questa sequenza;
 - es. "monaco", "monitor", "money", "moon".
- Successivamente, viene applicato un **post-filtraggio** per **eliminare i falsi positivi**;
 - es. "moon" potrebbe apparire perchè contiene "mo" e "on".
- I termini validi vengono poi usati per cercare i documenti nell'indice invertito principale.

Confronto tra Q-gram Index e Permuterm Index

Caratteristica	Q-gram Index	Permuterm Index
Spazio occupato	Più efficiente (meno ingombrante)	Più spazio richiesto (molte rotazioni per parola)
Wildcard supportate	Funziona bene per prefix e infix queries	Più efficace per tutte le wildcard queries
Post-filtraggio	Necessario per eliminare falsi positivi	Non richiesto
Efficienza nella ricerca	Buona, ma richiede più passaggi	Veloce, ma con elevato costo di memoria

Il **q-gram index** rappresenta quindi una soluzione più **compatta** e **scalabile** rispetto al **permuterm index**, specialmente in contesti con vocabolari molto grandi. Tuttavia, richiede un passaggio di post-filtraggio per eliminare risultati errati.

12.2 Spelling Correction

La **correzione ortografica** è una tecnica fondamentale nei sistemi di **recupero dell'informazione (IR)**, utilizzata per migliorare la qualità dei documenti indicizzati e delle query degli utenti. L'obiettivo è ridurre gli errori di battitura o trascrizione per garantire che il sistema restituisca risultati più accurati.

Esistono due principali ambiti di applicazione:

1. **Correzione dei documenti indicizzati** → Assicura che i documenti presenti nel database siano il più possibile privi di errori.
2. **Correzione delle query degli utenti** → Aiuta l'utente a trovare i documenti desiderati, correggendo automaticamente gli errori nella query o suggerendo alternative corrette.

Tipologie di Correzione Ortografica

Esistono due approcci principali per la correzione ortografica:

1. Correzione Isolata (Isolated Word Correction)

- Ogni parola viene controllata **singolarmente**, senza considerare il contesto in cui si trova. Se una parola non è presente nel dizionario, viene segnalata come errata.
- **Limite principale:** non rileva errori in cui la parola digitata esiste ma non è quella giusta nel contesto della frase.
- Esempio di errore non rilevato:
 - "I want to **form** a club" (corretto)
 - "I want to **from** a club" (errato ma non segnalato, perché "from" è una parola valida).

2. Correzione Sensibile al Contesto (Context-sensitive)

- Analizza le parole circostanti per determinare se la parola è usata correttamente nel contesto.
- Esempio:
 - "I flew **form** Heathrow to Narita." (Frase errata)
 - "I flew **from** Heathrow to Narita." (Correzione suggerita)
- Questo approccio è più avanzato e spesso utilizzato nei motori di ricerca e nei correttori grammaticali intelligenti.

Correzione dei Documenti Indicizzati

La correzione ortografica è particolarmente utile nella gestione di documenti digitalizzati tramite OCR (Optical Character Recognition) e in quelli che contengono errori di digitazione (typos).

Esempi di errori comuni:

- "O" e "D" → possono essere confusi nella scansione di testi stampati.
- "O" e "I" → più facilmente scambiati durante la digitazione, perché vicini sulla tastiera QWERTY.

Tuttavia, correggere direttamente i documenti può essere impraticabile per grandi collezioni di testo. **Alternativa:** Aniché correggere i documenti, si può migliorare il sistema di ricerca affinché sia in grado di riconoscere automaticamente varianti errate delle parole e mapparle ai termini corretti.

Correzione degli Errori nelle Query degli Utenti

Nella maggior parte dei casi, i sistemi di IR si concentrano sulla **correzione delle query**, piuttosto che sulla modifica dei documenti indicizzati.

1. **Ricerca con correzione automatica:** Se l'utente scrive male un termine nella query, il sistema recupera comunque i documenti indicizzati con la versione corretta della parola.
 - Query errata: "*Alanis Morisett*" → Il sistema può recuperare automaticamente documenti contenenti "*Alanis Morissette*".
2. **Suggerimento di query alternative (Did You Mean...?):** Il sistema suggerisce all'utente una versione corretta della query, ma lascia la decisione finale all'utente.
 - L'utente digita "*Alanis Morisett*" → Il sistema suggerisce la possibile versione corretta della parola: "*Did You Mean: Alanis Morissette?*"

12.3 Spelling Correction - Isolated Word Correction

Le due premesse

1° premessa: Esistenza di un lessico di riferimento

- Il sistema deve disporre di un lessico da cui derivare le parole corrette.

Questo può essere:

- Un dizionario standard (es. Webster's English Dictionary).
- Un dizionario specifico per un settore, mantenuto manualmente.
- Il lessico del corpus indicizzato (es. l'intero web), includendo anche nomi propri, acronimi e perfino errori comuni.

2° premessa: Definizione di una misura di distanza tra parole

- Deve esserci un metodo per calcolare la distanza tra una parola errata e una parola corretta. Dato un lessico e una parola q il sistema deve restituire i termini più vicini a q .
 - Ad esempio, "*informaton*" → "*information*".

La nozione di "*vicinanza*" può essere definita con diversi criteri.

12.3.1 Edit distance

L'**Edit Distance** è una misura che quantifica la differenza tra due stringhe, calcolando il numero minimo di operazioni necessarie per trasformare una stringa nell'altra.

Tipologie di Edit Distance

1. **Levenshtein Distance**: L'edit distance più comune è la **Distanza di Levenshtein**, che utilizza tre operazioni fondamentali a livello di carattere:

- **Inserzione** di un carattere
- **Cancellazione** di un carattere
- **Sostituzione** di un carattere

Esempi:

- "*dof*" $\rightarrow$ "*dog*" (1 operazione: sostituzione di "f" con "g")
- "*cat*" $\rightarrow$ "*dog*" (3 operazioni: sostituzione di ogni lettera)

2. **Damerau-Levenshtein Distance**: Una variante della distanza di Levenshtein è la Distanza di **Damerau-Levenshtein**, che aggiunge un'operazione in più:

- **Trasposizione** (scambio di due caratteri adiacenti)

Esempi:

- "*cat*" $\rightarrow$ "*act*" (1 operazione con trasposizione)
- "*form*" $\rightarrow$ "*from*" (1 operazione con trasposizione)

Calcolo dell'Edit Distance: Programmazione Dinamica

L'edit distance è generalmente calcolata tramite **programmazione dinamica**, utilizzando una matrice C di dimensione $(|S_1| + 1) \cdot (|S_2| + 1)$, dove ogni cella $C_{i,j}$ rappresenta il costo minimo per trasformare i primi i caratteri della stringa S_1 nei primi j caratteri di S_2 .

Formula ricorsiva

La matrice è costruita nel seguente modo:

- Casi base:

$$C_{i,0} = i, \quad C_{0,j} = j$$

- Passo ricorsivo:

$$C_{i,j} = \begin{cases} C_{i-1,j-1} & \text{se } x_i = y_j \\ 1 + \min(C_{i-1,j}, C_{i,j-1}, C_{i-1,j-1}) & \text{altrimenti} \end{cases}$$

- $C_{i-1,j}$ rappresenta la cancellazione
- $C_{i,j-1}$ rappresenta la inserzione
- $C_{i-1,j-1}$ rappresenta la sostituzione

Questa tecnica consente di trovare l'edit distance ottimale, ma ha una complessità temporale di $O(n^2)$ risultando **poco efficiente** per grandi insiemi di dati.

		s	u	r	g	e	r	y
	0	1	2	3	4	5	6	7
s	1	0	1	2	3	4	5	6
u	2	1	0	1	2	3	4	5
r	3	2	1	0	1	2	3	4
v	4	3	2	1	1	2	3	4
e	5	4	3	2	2	1	2	3
y	6	5	4	3	3	2	2	2

Weighted Edit Distance

Una variante della distanza di Levenshtein è la **Weighted Edit Distance**, che assegna pesi diversi alle operazioni di modifica a seconda della probabilità che un errore avvenga.

- **Utilità:** utile per errori di battitura o OCR.
- **Esempio:** la sostituzione di "m" con "n" (caratteri vicini sulla tastiera) può avere un costo inferiore rispetto alla sostituzione con "q".
- **Implementazione:** richiede una matrice di pesi e un algoritmo modificato di programmazione dinamica.

Applicazioni dell'Edit Distance

1. **Correzione delle query errate** → Dato un termine errato, si individuano tutte le parole corrette nel dizionario entro una distanza massima predefinita. Ad esempio, una query come "Alanis Morisett" potrebbe suggerire "Alanis Morissette".
2. **Suggerimenti automatici** → L'utente riceve una lista di parole corrette tra cui scegliere (Did you mean...?).
3. **Ricerca nei documenti** → Possibilità di cercare nei documenti le versioni corrette delle parole errate.

Vantaggi e Svantaggi dell'Algoritmo

Vantaggi:

- **Flessibilità:** può essere adattato a diversi tipi di distanza tra stringhe.
- **Efficienza in termini di memoria:** richiede solo $O(|P|)$ spazio.
- **Precisione:** risultati accurati basati su una metrica ben definita.

Svantaggi:

- **Bassa efficienza computazionale:** complessità temporale $O(|P||T|)$
- **Scalabilità:** calcolo della distanza per ogni parola nel dizionario.
- **Alternative necessarie:** sono stati sviluppati metodi per ottimizzarne le prestazioni.

12.3.2 Q-Gram Overlap

Il **Q-Gram Overlap** è una tecnica utilizzata per misurare la somiglianza tra stringhe di testo, confrontando insiemi di sottostringhe (q-grammi) estratte dalle parole. Questo metodo consente di individuare termini simili anche in presenza di errori di battitura o variazioni minime.

Come Funziona il Q-Gram Overlap

1. **Un q-gramma è una sequenza di q caratteri consecutivi estratti da una stringa.** Esempio con $q = 3$ (trigrammi):
 - Query: "december"
 - Trigrammi: $\{\#\#d, \#de, dec, ece, cem, emb, mbe, ber, er\$, r\$\$ \}$
 - Dove $\#$ e $\$$ sono caratteri speciali che indicano l'inizio e la fine della parola.
2. **Ricerca nel dizionario tramite q-gram index**
 - Per ogni q-gramma della query, si cercano nel dizionario tutte le parole che contengono almeno uno di quei q-grammi.
 - Si utilizza una struttura dati come un indice invertito per q-grammi per velocizzare la ricerca.
3. **Calcolo della similarità tra stringhe**
 - Si confrontano gli insiemi di q-grammi tra la query e le parole candidate per determinare il grado di sovrapposizione.

Esempio di Q-Gram Overlap con Trigrammi

Caso: confronto tra "december" e "november"

1. **Trigrammi di "december"** $\rightarrow \{\#\#d, \#de, dec, ece, cem, emb, mbe, ber, er\$, r\$\$ \}$
2. **Trigrammi di "november"** $\rightarrow \{\#\#n, \#no, nov, ove, vem, emb, mbe, ber, er\$, r\$\$ \}$
3. **Q-Gram Overlap:** 5 trigrammi in comune $\rightarrow \{emb, mbe, ber, er\$, r\$\$ \}$

Misura della Similarità: Coefficiente di Jaccard

Per quantificare il grado di sovrapposizione tra due insiemi di q-grammi, si utilizza il **coefficiente di Jaccard**, definito come:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Dove:

- A e B sono gli insiemi di q-grammi delle due stringhe
- $|A \cap B|$ è il numero di q-grammi in comune
- $|A \cup B|$ è il numero totale di q-grammi distinti tra le due stringhe

Esempio con "december" e "november"

$$J(december, november) = \frac{5}{15} = 0.333$$

Il valore di Jaccard è compreso tra 0 e 1:

- 1 $\rightarrow$ Le due stringhe hanno gli stessi q-grammi (identiche).
- 0 $\rightarrow$ Nessun q-gramma in comune (totalmente diverse).

Per stabilire se due parole sono abbastanza simili, si può **fissare una soglia** (es. $J > 0.8$ per considerarle un match).

Q-Gram Overlap con Q-Gram Index

Un **q-gram index** è una struttura che permette di effettuare rapidamente la ricerca di parole simili sfruttando un indice invertito per q-grammi.

1. Query = "lord"
2. Bigrammi della query: {*lo, or, rd*}
3. Q-Gram Index (parole associate a ogni bigramma)

lo	--> alone	--> [lore]	--> sloth
or	--> [border]	--> [lore]	--> morbid
rd	--> ardent	--> [border]	--> card

4. Ricerca parallela nei posting list:
 - Le parole candidate devono corrispondere ad almeno 2 dei 3 bigrammi.
 - Si possono utilizzare misure come il coefficiente di Jaccard per decidere quali parole sono abbastanza simili.

Vantaggi e Svantaggi del Q-Gram Overlap

Vantaggi

- **Resistente agli errori di battitura** → Utile per fuzzy search e correzione ortografica.
- **Supporta wildcard search** → Aiuta a gestire ricerche con caratteri jolly (* e ?).
- **Funziona bene per termini brevi** → Più efficace rispetto a tecniche basate su Edit Distance quando si confrontano stringhe corte.

Svantaggi

- **Maggiore spazio richiesto** → L'indice per q-grammi può essere molto grande.
- **Post-filtraggio necessario** → La ricerca può restituire falsi positivi (es. "mon*" può restituire anche "moon").
- **Prestazioni variabili** → Dipende dalla scelta di q (valori piccoli portano a più corrispondenze, valori grandi possono perdere simili validi).

12.3.3 Advanced q-gram filtering

L'**Advanced Q-Gram Filtering** è una tecnica che migliora le prestazioni del calcolo della **Edit Distance**, riducendo il numero di confronti necessari. Questo approccio non risolve direttamente il problema, ma **filtra le coppie di stringhe improbabili**, limitando la ricerca alle sole stringhe potenzialmente simili.

L'obiettivo è stabilire rapidamente se due stringhe **non possono** essere simili entro una soglia di errore k , ($ED(S_1, S_2) \leq k$), evitando confronti costosi. I principali filtri proposti sono:

1. **Length Filter** → Esclude le stringhe con differenze di lunghezza troppo grandi per soddisfare la soglia di errore k . Se $|S_1 - S_2| > k$ allora $ED(S_1, S_2) > k$ e possiamo scartarle subito.
2. **Count Filter** → Confronta la frequenza dei q-grammi nelle due stringhe. Se il numero di q-grammi comuni è troppo basso, la somiglianza è improbabile e la stringa può essere scartata.
3. **Position Filter** → Considera la posizione dei q-grammi nelle stringhe. Se la distribuzione è troppo diversa, anche con gli stessi q-grammi, le stringhe sono probabilmente diverse.

Length filtering

Due stringhe S_1 e S_2 possono avere una **Edit Distance** $\leq k$ solo se la loro differenza di lunghezza è al massimo k . Matematicamente

$$|length(S_1) - length(S_2)| \leq k$$

Se questa condizione non è soddisfatta, possiamo escludere immediatamente la coppia senza calcolare la Edit Distance.

Count filtering

Il **Count Filtering** è una tecnica di filtraggio che si basa sul numero di **q-grammi condivisi** tra due stringhe. L'idea principale è che **stringhe simili condividono un numero minimo di q-grammi**, e questo numero diminuisce con il numero di modifiche (edit operations) necessarie per trasformare una stringa nell'altra.

Se due stringhe S_1 e S_2 hanno una **Edit Distance** $\leq k$ allora devono avere **almeno** un certo numero di q-grammi in comune. Questo numero minimo è calcolato con la formula:

$$[\max(length(S_1), length(S_2)) + q - 1] - kq$$

Position Filtering

Il **Position Filtering** è una tecnica che migliora l'accuratezza del confronto tra stringhe basato sui q-grammi, evitando falsi positivi dovuti a corrispondenze casuali di q-grammi in posizioni diverse.

Quando due stringhe hanno un **Edit Distance** alta, possono comunque condividere molti q-grammi, ma questi potrebbero essere **disallineati**. Per risolvere questo problema, il Position Filtering **registra la posizione di ogni q-gramma** e scarta quelli che si trovano a una distanza superiore a k rispetto alla loro posizione nella stringa originale.

Formalmente, dati due q-grammi corrispondenti q_1 e q_2 appartenenti a due stringhe S_1 e S_2 , si accettano valori validi solo se:

$$|pos(q_1) - pos(q_2)| \leq k$$

Dove $pos(q)$ indica la posizione del q-gramma all'interno della relativa stringa. Questa tecnica aiuta a **ridurre confronti inutili** e migliora la precisione nei sistemi di **ricerca testuale** e **correzione ortografica**.

Approximate String Join

L'Approximate Join tra due **Text Collections** $TC1$ e $TC2$ richiede di confrontare tutte le possibili coppie di testi nel loro **prodotto cartesiano** $TC1 \times TC2$. Tuttavia, il calcolo dell'Edit Distance per ogni coppia è computazionalmente costoso, con una complessità di $O(n^2)$.

Per migliorare l'efficienza, si applicano **tecniche di filtering** (*Length filtering, Count filtering, Position filtering*) per ridurre il numero di confronti effettivi. Si costruisce un **Candidate Set** S , che include solo le coppie di testi che soddisfano determinati filtri:

$$S = \{(t_1, t_2) | (t_1, t_2) \in TC1 \times TC2 \wedge \text{satisfies all filters}\}$$

Questo approccio garantisce sia la **correttezza** (nessuna esclusione di coppie valide) sia una maggiore **efficienza** (riducendo i falsi positivi).

12.4 Spelling Correction - Context Sensitive Word Correction

La **correzione contestuale degli errori ortografici** non si limita a correggere parole isolate, ma considera il contesto della frase. Ad esempio, nella query *"flew form Heathrow"*, il termine *"form"* è un errore e dovrebbe essere corretto in *"from"*, poiché la frase corretta *"flew from Heathrow"* ha maggiore probabilità di esistere nei documenti.

12.4.1 Hit-Based Correction

Un approccio comune consiste nel generare varianti della query con singole parole corrette e selezionare la frase che restituisce più risultati. Questo metodo, detto **hit-based spelling correction**, sfrutta il numero di documenti recuperati per suggerire la correzione più plausibile. Tuttavia, il numero di combinazioni può crescere rapidamente, rendendo necessario un compromesso tra **efficacia** e **costo computazionale**.

Problemi generali nella correzione ortografica

- **Interfaccia utente:** si può applicare una correzione automatica o suggerire alternative (Did you mean...?). Se ci sono più correzioni possibili, occorre decidere quale mostrare.
- **Efficienza:** la correzione ortografica è costosa, quindi potrebbe essere applicata solo alle query che restituiscono pochi documenti. I motori di ricerca moderni, però, sono abbastanza ottimizzati da poterla eseguire su ogni query.

12.5 Soundex

Soundex è un **algoritmo fonetico** progettato per indicizzare parole in base alla loro pronuncia, anziché alla loro esatta ortografia. Il suo obiettivo principale è associare **parole omofone** (*che suonano simili*) a una stessa **rappresentazione codificata**, permettendo di identificare nomi scritti in modi diversi ma con la stessa pronuncia, come *Chebyshev* e *Tchebycheff*. Tuttavia, essendo basato sulla lingua inglese, il suo utilizzo è limitato ad alcuni contesti, specialmente nella ricerca di nomi propri.

Funzionamento dell'algoritmo

Soundex converte ogni parola in una **stringa di quattro caratteri**, composta dalla prima lettera della parola seguita da tre numeri. Il processo avviene in questi passaggi:

1. Si conserva la prima lettera della parola.
2. Si rimuovono le vocali (*A, E, I, O, U*) e alcune consonanti mute (*H, W, Y*).
3. Le altre lettere vengono convertite in cifre in base a un set di regole fonetiche:
 - B, F, P, V $\rightarrow$ 1
 - C, G, J, K, Q, S, X, Z $\rightarrow$ 2
 - D, T $\rightarrow$ 3
 - L $\rightarrow$ 4
 - M, N $\rightarrow$ 5
 - R $\rightarrow$ 6
4. Si eliminano i numeri consecutivi duplicati.
5. Si rimuovono gli zeri.
6. Si aggiungono zeri finali per ottenere una stringa di quattro caratteri.

Esempio

Se applichiamo Soundex al nome *Herman*, il risultato sarà: H655.

Utilità e limiti di Soundex

Sebbene Soundex sia utile per la ricerca di nomi con elevata tolleranza agli errori, la sua efficacia nella recupero di informazioni (IR) è limitata. Studi come quello di Zobel e Dart (1996) hanno dimostrato che esistono algoritmi fonetici più performanti. Inoltre, Soundex è influenzato dalla nazionalità dei nomi e funziona male con lingue diverse dall'inglese.

13 Ranking the Web

13.1 Link Analysis

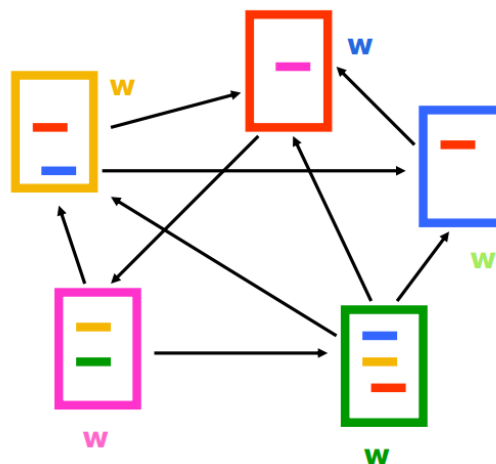
La **Link Analysis** è una tecnica utilizzata per valutare l'importanza di una pagina web sfruttando la **struttura dei collegamenti ipertestuali** tra le pagine. L'idea di base è che un **link da una pagina A a una pagina B rappresenti una forma di endorsement**, ovvero A considera B una fonte autorevole su un determinato argomento. Questo concetto è stato alla base dell'algoritmo di ranking sviluppato da Google, noto come **PageRank**.

Principi della Link Analysis

1. **I link in entrata (inlink) determinano l'autorità di una pagina** → Più pagine rimandano a una determinata risorsa, maggiore sarà la sua rilevanza.
2. **Si può rappresentare il web come un grafo**, dove le pagine web sono nodi e i link rappresentano archi diretti tra i nodi.
3. **Esistono algoritmi di ranking basati sulla Link Analysis:**
 - **PageRank (1998, Brin & Page)** → Classifica l'intero web indipendentemente dalla query dell'utente.
 - **HITS (1998, Kleinberg)** → Determina l'autorità delle pagine solo su un sottoinsieme rilevante rispetto a una query specifica.

Funzionamento degli algoritmi LAR (Link Analysis Ranking)

1. Si parte da una collezione di pagine web.
2. Si estrae il **grafo dei collegamenti ipertestuali**.
3. Si esegue un algoritmo di ranking basato sulla struttura dei link.
4. Si assegna un **peso di autorità** a ciascuna pagina, utile per il posizionamento nei motori di ricerca.



13.2 PageRank

L'algoritmo **PageRank** è stato sviluppato da **Brin e Page (1998)** per determinare l'importanza di una pagina web in base alla sua **connessione con altre pagine**. L'idea chiave è che una pagina sia **autorevole** non per il suo contenuto, ma perché altre pagine la citano con un collegamento ipertestuale (**link in entrata**).

Principi di base

- Il PageRank sfrutta la **topologia del web** come indicatore di autorevolezza.
- L'algoritmo **non analizza il contenuto delle pagine**, ma utilizza il numero e la qualità dei link in entrata per calcolarne l'importanza.
- Una pagina p ottiene un punteggio $R(p)$ proporzionale ai punteggi delle pagine che la citano ($q \rightarrow p$).

Formula di base

L'algoritmo assegna un punteggio iniziale a ciascuna pagina e aggiorna iterativamente i punteggi seguendo la formula:

$$R(p) = c \cdot \sum_{q:q \rightarrow p} \frac{R(q)}{N_q}$$

Dove:

- $R(p)$ è il **PageRank** della pagina p .
- q sono le pagine che linkano p .
- N_q è il numero totale di link in uscita dalla pagina q .
- c è una costante di normalizzazione per far sì che la somma totale dei PageRank sia **sempre 1**.

L'idea è che ogni pagina **divida equamente la sua autorevolezza tra tutte le pagine che cita**.

Problema del “Rank Sink” e soluzione

Un problema della formula iniziale è che un **gruppo di pagine che si linkano tra loro senza puntare all'esterno può assorbire tutto il PageRank**, creando un **ciclo chiuso**.

Per risolvere il problema, si introduce un **fattore di damping** α , che aggiunge una probabilità di **saltare casualmente a una qualsiasi pagina del web**:

$$R(p) = (1 - \alpha) \sum_{q:q \rightarrow p} \frac{R(q)}{N_q} + \alpha E(p)$$

Dove:

- α (solitamente 0.15) è il **fattore di salto casuale**.
- $E(p)$ è un valore che rappresenta un **rank di origine esterna**, tipicamente distribuito uniformemente tra tutte le pagine.

Procedura Iterativa

L'algoritmo calcola i valori di PageRank in modo iterativo:

1. Inizializzazione:

- Si assegna a ogni pagina $E(p) = \frac{\alpha}{|S|}$, dove $|S|$ è il numero totale di pagine, mentre $0 < \alpha < 1$ è il fattore di salto casuale (es. $\alpha = 0.15$).
- Si assegna a ogni pagina un valore iniziale $R_0(p) = \frac{1}{|S|}$.
- Si imposta un contatore $i = 0$.

2. Calcolo iterativo:

- Per ogni pagina p , si calcola il valore provvisorio:

$$R'_i(p) = \left[(1 - \alpha) \sum_{q:q \rightarrow p} \frac{R_{i-1}(q)}{N_q} \right] + E(p)$$

- Si normalizza:

$$R_i(p) = c \cdot R'_i(p), \quad c = \frac{1}{\sum_{p \in S} R'_i(p)}$$

3. Convergenza:

- Si ripete il calcolo, incrementando i di 1 ad ogni iterazione, finché i valori di PageRank **non cambiano più significativamente**:

$$|R_i(p) - R_{i-1}(p)| < \epsilon$$

- ϵ è un valore di tolleranza per la convergenza.

PageRank Convergence Speed

La **computazione del PageRank** deve essere efficiente poiché il web è altamente dinamico: ogni settimana il 25% dei link cambia, e dopo un anno l'80% viene sostituito. Fortunatamente, l'algoritmo converge rapidamente, con un numero di iterazioni empiricamente pari a $O(\log n)$, rendendo il calcolo gestibile anche su grandi scale. Google, nei primi esperimenti, ha ottenuto convergenza in circa **52 iterazioni** su 322 milioni di link. Tecniche più recenti, come il **Distributed PageRank Computation (DPC)**, accelerano ulteriormente il processo, riducendo i tempi di convergenza di **5-7 volte** rispetto ai metodi tradizionali.

Data la continua evoluzione della struttura del web, avere un metodo rapido ed efficiente per il calcolo del PageRank è essenziale per mantenere aggiornati i risultati dei motori di ricerca.

13.3 Hubs and Authorities

L'algoritmo **HITS (Hyperlink-Induced Topic Search)**, proposto da **Kleinberg nel 1998**, è un metodo di analisi dei link che identifica due tipi di pagine web:

- **Authorities**: pagine che contengono informazioni rilevanti su un argomento.
- **Hubs**: pagine che non forniscono direttamente informazioni, ma contengono link verso buone Authorities.

L'idea di base è che **buoni hub puntano a buone authorities**, e **buone authorities sono linkate da buoni hub**. Questo porta a una valutazione iterativa dei nodi in un grafo del web.

Calcolo dei Valori di Hub e Authority

Ogni pagina v ha due valori:

- $h(v) \rightarrow$ misura l'importanza della pagina come hub.
- $a(v) \rightarrow$ misura l'autorità della pagina.

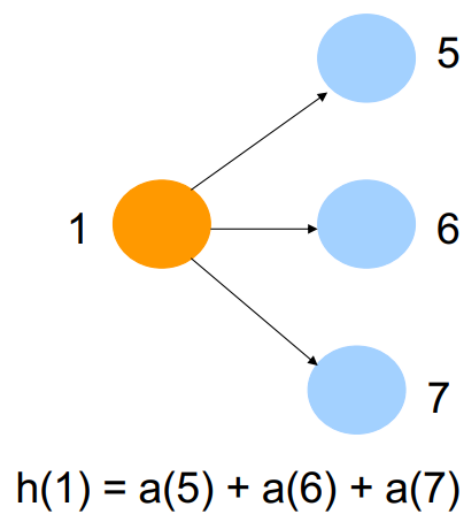
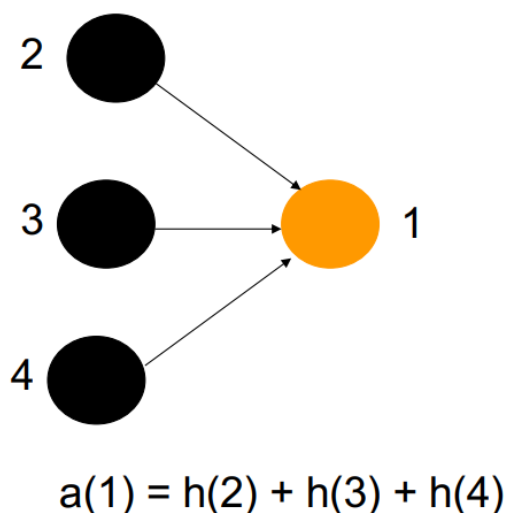
Il calcolo avviene in due passaggi:

1. Costruzione del Base Set:

- Si identificano le pagine contenenti un termine di ricerca (**Root Set**).
- Si aggiungono tutte le pagine che puntano a queste pagine (**backward neighbors**).
- Si aggiungono tutte le pagine a cui queste pagine puntano (**forward neighbors**).
- Si ottiene così il Base Set, un sotto-grafo del web su cui calcolare i punteggi.

2. Iterazione dei punteggi:

- Si inizializzano tutti i valori $h(v)$ e $a(v)$ a 1.
- Ad ogni iterazione:
 - Il punteggio **authority** di una pagina è la somma dei punteggi hub delle pagine che la linkano.
 - Il punteggio **hub** di una pagina è la somma dei punteggi authority delle pagine a cui punta.
- I valori vengono normalizzati e il processo si ripete fino alla convergenza.



Limitazioni e Vulnerabilità

- **Facile da manipolare:** un utente può creare una pagina con molti link verso authorities per farla apparire come un buon hub.
- **Dipendenza dalla query:** il Base Set deve essere costruito ogni volta in base alla ricerca.
- **Topic Drift:** possono essere incluse nel Base Set pagine autorevoli ma non pertinenti alla ricerca.
- **Sensibile ai cambiamenti del web:** piccole modifiche nei link possono alterare drasticamente i punteggi.

13.4 Spam

Lo **spam nei motori di ricerca** è un insieme di tecniche utilizzate per manipolare i risultati dei motori di ricerca e ottenere una posizione più alta nelle classifiche senza offrire contenuti di reale valore. Questo fenomeno è emerso con i primi motori di ricerca che si basavano principalmente sulla **frequenza delle parole chiave** (TF-IDF), permettendo a chiunque di migliorare il proprio ranking ripetendo in modo massiccio termini specifici nella propria pagina web.

Tecniche di Spam

- **Keyword Stuffing:** ripetizione eccessiva di parole chiave, spesso nascoste con colori di sfondo uguali al testo.
- **Cloaking:** presentare contenuti diversi ai crawler dei motori di ricerca rispetto a quelli visibili agli utenti.
- **Doorway Pages:** pagine ottimizzate per una parola chiave che reindirizzano l'utente a un'altra pagina.
- **Link Spamming:** utilizzo di reti di link artificiali per aumentare la popolarità di una pagina (es. link nascosti, scambi di link, siti con domini multipli che puntano a un'unica pagina).
- **Domain Flooding:** registrazione di numerosi domini che puntano alla stessa pagina per aumentare la sua visibilità.
- **Uso di bot:** programmi che simulano ricerche o generano milioni di richieste per alterare il ranking.

Contromisure contro lo Spam

- **Analisi dei link:** valutazione della qualità e dell'autenticità dei collegamenti tra le pagine.
- **Filtri basati su machine learning:** utilizzo di modelli addestrati per riconoscere contenuti spam.
- **Controlli manuali:** blacklist, auditing delle query più frequenti e verifica delle segnalazioni degli utenti.
- **Limiti su meta-keywords** e strumenti per rilevare pattern sospetti.

Nonostante le continue evoluzioni delle tecniche di spam, i motori di ricerca migliorano costantemente i loro algoritmi per garantire che i risultati restituiti siano sempre più pertinenti e affidabili.